

```
! pip install obspy
```

```
Collecting obspy
  Downloading obspy-1.5.1-cp313-cp313-manylinux_2_27_x86_64.manylinu
Requirement already satisfied: numpy>=1.21 in /usr/local/lib/python3
Requirement already satisfied: scipy>=1.7 in /usr/local/lib/python3.
Requirement already satisfied: matplotlib>=3.3 in /usr/local/lib/pyt
Requirement already satisfied: lxml in /usr/local/lib/python3.13/dis
Requirement already satisfied: setuptools in /usr/local/lib/python3.
Requirement already satisfied: sqlalchemy>=1.4 in /usr/local/lib/pyt
Requirement already satisfied: decorator in /usr/local/lib/python3.1
Requirement already satisfied: requests in /usr/local/lib/python3.13
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/py
Requirement already satisfied: cyciler>=0.10 in /usr/local/lib/python
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/p
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/p
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/pyt
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.1
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/py
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/li
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3
Requirement already satisfied: typing-extensions>=4.6.0 in /usr/loca
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/loca
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13
Downloading obspy-1.5.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_
15.3/15.3 MB 30.9 MB/s e
Installing collected packages: obspy
Successfully installed obspy-1.5.1
WARNING: The following packages were previously imported in this
runtime:
  [signal]
You must restart the runtime in order to use newly installed
versions.
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import obspy
print("obspy version: ",obspy.__version__)
```

```
obspy version: 1.5.1
```

```
from obspy.clients.fdsn import Client
earthscope=Client("EARTHSCOPE")
print(earthscope)
```

FDSN Webservice Client (base url: <https://service.earthscope.org>)
Available Services: 'dataselect' (v1.1.78), 'station' (v1.1.57)

Use e.g. `client.help('dataselect')` for the parameter description of the individual services or `client.help()` for parameter description of all webservices.

```
from obspy.clients.fdsn import Client
from obspy import UTCDateTime
```

```
usgs = Client("USGS")
```

```
start = UTCDateTime("2026-01-01")
end = UTCDateTime("2026-09-21")
```

```
catalog = usgs.get_events(
    starttime=start,
    endtime=end,
    minmagnitude=6.0
)
```

```
print(catalog)
```

```
100 Event(s) in Catalog:
2026-09-20T09:17:35.624000Z | -5.902, +146.092 | 6.4 mww | manual
2026-09-17T14:19:52.210000Z | +52.859, -171.376 | 6.5 mww | manual
...
2026-01-02T13:58:15.741000Z | +16.798, -99.394 | 6.5 mww | manual
2026-01-01T01:53:01.809000Z | -45.612, +96.359 | 6.0 mww | manual
To see all events call 'print(CatalogObject.__str__(print_all=True))'
```

```
for event in catalog:
    origin = event.preferred_origin() or event.origins[0]
    magnitude = event.preferred_magnitude() or event.magnitudes[0]

    print(
        "Time:",
        origin.time,
        "| Lat:", round(origin.latitude, 2),
        "| Lon:", round(origin.longitude, 2),
        "| Depth:", round(origin.depth / 1000, 1), "km",
        "| M:", magnitude.mag
    )
```

Time: 2026-06-08T18:00:27.517000Z	Lat: 22.79	Lon: -85.09	Dep:
Time: 2026-06-08T00:55:12.546000Z	Lat: 5.47	Lon: 125.22	Depth
Time: 2026-06-07T23:49:13.534000Z	Lat: 5.66	Lon: 125.26	Depth
Time: 2026-06-07T23:48:07.276000Z	Lat: 5.46	Lon: 125.22	Depth
Time: 2026-06-07T23:37:41.970000Z	Lat: 5.6	Lon: 125.06	Depth
Time: 2026-06-07T10:41:56.538000Z	Lat: 49.98	Lon: 157.25	Depth
Time: 2026-06-01T22:12:36.092000Z	Lat: 39.25	Lon: 15.79	Depth
Time: 2026-05-31T21:34:17.953000Z	Lat: -32.83	Lon: -71.89	Depth
Time: 2026-05-27T15:01:11.833000Z	Lat: -50.41	Lon: 139.26	Depth
Time: 2026-05-27T14:51:04.926000Z	Lat: -50.51	Lon: 139.49	Depth
Time: 2026-05-25T21:52:20.140000Z	Lat: -22.37	Lon: -68.6	Depth
Time: 2026-05-20T17:43:01.865000Z	Lat: -56.0	Lon: -122.3	Depth
Time: 2026-05-16T14:50:05.533000Z	Lat: 17.42	Lon: -61.29	Depth
Time: 2026-05-15T11:22:01.646000Z	Lat: 38.94	Lon: 142.16	Depth
Time: 2026-05-14T17:53:13.718000Z	Lat: -6.15	Lon: 130.38	Depth
Time: 2026-05-09T04:42:21.975000Z	Lat: 50.98	Lon: 179.21	Depth
Time: 2026-05-04T06:09:47.129000Z	Lat: 11.71	Lon: 125.33	Depth
Time: 2026-04-20T07:56:49.716000Z	Lat: 39.77	Lon: 143.1	Depth
Time: 2026-04-20T07:52:58.908000Z	Lat: 39.97	Lon: 143.06	Depth
Time: 2026-04-19T17:34:47.284000Z	Lat: -15.22	Lon: -173.48	Depth
Time: 2026-04-16T13:30:03.496000Z	Lat: -32.14	Lon: -178.32	Depth
Time: 2026-04-04T10:34:28.266000Z	Lat: 4.87	Lon: 126.14	Depth
Time: 2026-04-02T03:23:52.561000Z	Lat: 1.17	Lon: 126.48	Depth
Time: 2026-04-01T22:48:13.063000Z	Lat: 1.09	Lon: 126.24	Depth
Time: 2026-04-01T11:20:27.264000Z	Lat: -7.05	Lon: -76.22	Depth
Time: 2026-03-30T08:44:13.780000Z	Lat: -15.29	Lon: 167.58	Depth
Time: 2026-03-26T14:18:51.927000Z	Lat: 39.44	Lon: 143.38	Depth
Time: 2026-03-24T04:37:51.627000Z	Lat: -18.54	Lon: -175.48	Depth
Time: 2026-03-22T15:30:36.570000Z	Lat: -15.34	Lon: -172.65	Depth
Time: 2026-03-22T15:27:55.084000Z	Lat: -15.28	Lon: -172.6	Depth
Time: 2026-03-22T06:15:36.222000Z	Lat: -15.25	Lon: -172.93	Depth
Time: 2026-03-21T12:16:46.629000Z	Lat: 23.86	Lon: -45.8	Depth
Time: 2026-03-20T02:30:34.150000Z	Lat: -19.28	Lon: 168.38	Depth
Time: 2026-03-20T00:22:04.661000Z	Lat: -61.19	Lon: -56.4	Depth
Time: 2026-03-13T13:39:21.201000Z	Lat: -28.69	Lon: -71.48	Depth
Time: 2026-03-09T23:03:54.034000Z	Lat: 40.57	Lon: 13.98	Depth
Time: 2026-03-08T09:28:26.478000Z	Lat: -15.98	Lon: -173.85	Depth
Time: 2026-03-06T14:27:48.050000Z	Lat: -11.47	Lon: 163.1	Depth
Time: 2026-03-04T17:54:36.723000Z	Lat: 52.33	Lon: 176.37	Depth
Time: 2026-03-03T04:56:45.857000Z	Lat: 2.01	Lon: 96.69	Depth
Time: 2026-03-02T03:55:23.534000Z	Lat: 22.97	Lon: 144.21	Depth
Time: 2026-03-01T05:44:41.923000Z	Lat: -21.8	Lon: -179.45	Depth
Time: 2026-02-23T05:11:48.410000Z	Lat: 52.33	Lon: -169.87	Depth
Time: 2026-02-22T16:57:46.738000Z	Lat: 6.85	Lon: 116.29	Depth
Time: 2026-02-22T07:43:25.409000Z	Lat: -21.79	Lon: 179.63	Depth
Time: 2026-02-16T21:42:52.469000Z	Lat: 51.25	Lon: 157.7	Depth
Time: 2026-02-14T02:27:42.554000Z	Lat: -14.99	Lon: 166.62	Depth
Time: 2026-02-12T13:34:31.172000Z	Lat: -30.77	Lon: -71.42	Depth
Time: 2026-02-10T20:44:52.845000Z	Lat: -21.03	Lon: -178.38	Depth
Time: 2026-02-04T10:39:28.490000Z	Lat: -29.53	Lon: -178.6	Depth
Time: 2026-01-29T07:30:39.458000Z	Lat: -57.77	Lon: -25.6	Depth
Time: 2026-01-22T12:42:34.200000Z	Lat: 51.79	Lon: 158.53	Depth

```

Time: 2026-01-21T16:37:43.141000Z | Lat: 23.2 | Lon: 142.6 | Depth
Time: 2026-01-19T13:02:19.436000Z | Lat: -22.44 | Lon: 170.39 | Depth
Time: 2026-01-16T03:25:53.034000Z | Lat: 43.69 | Lon: -128.06 | Depth
Time: 2026-01-13T07:34:07.739000Z | Lat: 44.55 | Lon: 149.32 | Depth
Time: 2026-01-10T14:58:23.726000Z | Lat: 3.78 | Lon: 126.99 | Depth
Time: 2026-01-07T03:02:53.890000Z | Lat: 7.24 | Lon: 126.9 | Depth

```

```
event_time = UTCDateTime("2026-07-17T14:48:39.881000Z")
```

```
print("Event time:", event_time)
```

```
Event time: 2026-07-17T14:48:39.881000Z
```

```

stations = earthscope.get_stations(
    starttime=event_time - 60,
    endtime=event_time + 600,
    level="channel",
    latitude=14.57,
    longitude=-92.96,
    maxradius=10
)

print(stations)

```

```

SY.LAFE.S1.LXE, SY.LAFE.S3.MXZ, SY.LAFE.S3
SY.MASN.S1.LXZ, SY.MASN.S1.LXN, SY.MASN.S1
SY.MASN.S3.MXN, SY.MASN.S3.MXE, SY.MASN.S1
SY.MASN.S1.LXE, SY.MASN.S3.MXZ, SY.MASN.S3
SY.MGAN.S1.LXZ, SY.MGAN.S1.LXN, SY.MGAN.S1
SY.MGAN.S3.MXN, SY.MGAN.S3.MXE, SY.MOIG.S1
SY.MOIG.S1.LXE, SY.MOIG.S3.MXZ, SY.MOIG.S3
SY.MT03.S1.LXZ, SY.MT03.S1.LXN, SY.MT03.S1
SY.MT03.S3.MXN, SY.MT03.S3.MXE, SY.NANN.S1
SY.NANN.S1.LXE, SY.NANN.S3.MXZ, SY.NANN.S3
SY.OCM.S1.LXZ, SY.OCM.S1.LXN, SY.OCM.S1.LX
SY.OCM.S3.MXN, SY.OCM.S3.MXE, SY.ORTG.S1.L
SY.ORTG.S1.LXE, SY.ORTG.S3.MXZ, SY.ORTG.S3
SY.OXBV.S1.LXZ, SY.OXBV.S1.LXN, SY.OXBV.S1
SY.OXBV.S3.MXN, SY.OXBV.S3.MXE, SY.OXEC.S1
SY.OXEC.S1.LXE, SY.OXEC.S3.MXZ, SY.OXEC.S3
SY.OXET.S1.LXZ, SY.OXET.S1.LXN, SY.OXET.S1
SY.OXET.S3.MXN, SY.OXET.S3.MXE, SY.OXNC.S1
SY.OXNC.S1.LXE, SY.OXNC.S3.MXZ, SY.OXNC.S3
SY.OXPL.S1.LXZ, SY.OXPL.S1.LXN, SY.OXPL.S1
SY.OXPL.S3.MXN, SY.OXPL.S3.MXE, SY.OXSV.S1
SY.OXSV.S1.LXE, SY.OXSV.S3.MXZ, SY.OXSV.S3
SY.OXTT.S1.LXZ, SY.OXTT.S1.LXN, SY.OXTT.S1
SY.OXTT.S3.MXN, SY.OXTT.S3.MXE, SY.RCON.S1
SY.RCON.S1.LXE, SY.RCON.S3.MXZ, SY.RCON.S3
SY.RIF0.S1.LXZ, SY.RIF0.S1.LXN, SY.RIF0.S1

```

```

SY.RIF0.S3.MXN, SY.RIF0.S3.MXE, SY.SAB1.S1
SY.SAB1.S1.LXE, SY.SAB1.S3.MXZ, SY.SAB1.S3
SY.SAB2.S1.LXZ, SY.SAB2.S1.LXN, SY.SAB2.S1
SY.SAB2.S3.MXN, SY.SAB2.S3.MXE, SY.SAJU.S1
SY.SAJU.S1.LXE, SY.SAJU.S3.MXZ, SY.SAJU.S3
SY.SJQ.S1.LXZ, SY.SJQ.S1.LXN, SY.SJQ.S1.LX
SY.SJQ.S3.MXN, SY.SJQ.S3.MXE, SY.SNET.S1.L
SY.SNET.S1.LXE, SY.SNET.S3.MXZ, SY.SNET.S3
SY.SOMN.S1.LXZ, SY.SOMN.S1.LXN, SY.SOMN.S1
SY.SOMN.S3.MXN, SY.SOMN.S3.MXE, SY.TBCA.S1
SY.TBCA.S1.LXE, SY.TBCA.S3.MXZ, SY.TBCA.S3
SY.TBHS.S1.LXZ, SY.TBHS.S1.LXN, SY.TBHS.S1
SY.TBHS.S3.MXN, SY.TBHS.S3.MXE, SY.TBHY.S1
SY.TBHY.S1.LXE, SY.TBHY.S3.MXZ, SY.TBHY.S3
SY.TBMR.S1.LXZ, SY.TBMR.S1.LXN, SY.TBMR.S1
SY.TBMR.S3.MXN, SY.TBMR.S3.MXE, SY.TBPV.S1
SY.TBPV.S1.LXE, SY.TBPV.S3.MXZ, SY.TBPV.S3
SY.TBTN.S1.LXZ, SY.TBTN.S1.LXN, SY.TBTN.S1
SY.TBTN.S3.MXN, SY.TBTN.S3.MXE, SY.TEIG.S1
SY.TEIG.S1.LXE, SY.TEIG.S3.MXZ, SY.TEIG.S3
SY.TGUH.S1.LXZ, SY.TGUH.S1.LXN, SY.TGUH.S1
SY.TGUH.S3.MXN, SY.TGUH.S3.MXE, SY.TLIG.S1
SY.TLIG.S1.LXE, SY.TLIG.S3.MXZ, SY.TLIG.S3
SY.TRT1.S1.LXZ, SY.TRT1.S1.LXN, SY.TRT1.S1
SY.TRT1.S3.MXN, SY.TRT1.S3.MXE, SY.UNM.S1.L
SY.UNM.S1.LXE, SY.UNM.S3.MXZ, SY.UNM.S3.MX
SY.VMAR.S1.LXZ, SY.VMAR.S1.LXN, SY.VMAR.S1
SY.VMAR.S3.MXN, SY.VMAR.S3.MXE, SY.ZACA.S1
SY.ZACA.S1.LXE, SY.ZACA.S3.MXZ, SY.ZACA.S3
TC.ELI1..EHZ, TC.ELI1..EHN, TC.ELI1..EHE,
TC.ELI1..ENN, TC.ELI1..ENE, TC.LCR2..HHZ,
TC.LCR2..HHE, TC.MARA..HHZ, TC.MARA..HHN,

```

```

from math import radians, sin, cos, asin, sqrt

def distance_deg(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(
        radians, [lat1, lon1, lat2, lon2]
    )

    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
    c = 2 * asin(sqrt(a))

    return c * 180 / 3.141592653589793

station_list = []

```

```

for net in stations:
    # Ignore synthetic stations
    if net.code == "SY":
        continue

    for sta in net:
        channels = [cha.code for cha in sta]

        # We want a vertical component
        if "BHZ" in channels or "HHZ" in channels:

            distance = distance_deg(
                14.57,
                -92.96,
                sta.latitude,
                sta.longitude
            )

            station_list.append(
                (
                    distance,
                    net.code,
                    sta.code,
                    sta.latitude,
                    sta.longitude,
                    "BHZ" if "BHZ" in channels else "HHZ"
                )
            )

station_list.sort()

for row in station_list[:20]:
    print(row)

```

```

(1.225342816869424, 'GI', 'RETA', 14.524621, -91.694939, 'HHZ')
(1.28807518966878, 'GI', 'HUCU', 15.407847, -91.947188, 'HHZ')
(1.3579464204685345, 'GI', 'SUCU', 14.56, -91.557, 'HHZ')
(1.5945701343951102, 'GI', 'HUEH', 15.318176, -91.502541, 'HHZ')
(1.7611583096415253, 'GI', 'ESTQ', 14.15158, -91.19408, 'HHZ')
(1.8867712357167563, 'MX', 'CCIG', 16.281799, -92.136803, 'BHZ')
(2.0340201632723227, 'GI', 'HUBA', 15.816578, -91.294484, 'HHZ')
(2.1063852799680585, 'GI', 'CMSM', 14.782826, -90.793703, 'HHZ')
(2.177596334653952, 'GI', 'SASJ', 14.483898, -90.712236, 'HHZ')
(2.285366854927362, 'GI', 'GUMI', 14.621, -90.599, 'HHZ')
(2.3833211768029425, 'GI', 'GUVU', 14.532, -90.498, 'HHZ')
(2.4721610116484958, 'GI', 'SRIX', 14.171699, -90.441307, 'HHZ')
(2.530577222427194, 'GI', 'GUNB', 14.595731, -90.345306, 'HHZ')
(2.5470758760092433, 'GI', 'QCIX', 16.003414, -90.777245, 'HHZ')
(2.558350770467267, 'GI', 'GUSP', 14.552, -90.3168, 'HHZ')

```

```
(2.696203216057268, 'GI', 'SROR', 14.231059, -90.198398, 'HHZ')
(2.8735136766104588, 'GI', 'AVCS', 15.815492, -90.276573, 'HHZ')
(3.021562555910097, 'GI', 'JAMO', 14.495609, -89.839486, 'HHZ')
(3.1539081983444133, 'SV', 'NUBE', 13.90216, -89.779907, 'HHZ')
(3.207851123216648, 'SV', 'RBDL', 14.113, -89.6827, 'HHZ')
```

```
import requests

url = "https://service.earthscope.org/irisws/fedcatalog/1/query"

params = {
    "targetservice": "dataselect",
    "format": "text",
    "cha": "HHZ",
    "latitude": 14.57,
    "longitude": -92.96,
    "maxradius": 10,
    "starttime": str(event_time - 10),
    "endtime": str(event_time + 120),
    "includeavailability": "true",
    "matchtimeseries": "true",
    "nodata": 404
}

response = requests.get(url, params=params)

print("Status code:", response.status_code)
print(response.text)
```

```
OV|DUNO|HHZ|10.0791|-85.493698|174.0|0.0|0.0|-90.0|Nanometrics Tr
OV|GRZA|HHZ|9.9155|-85.635597|39.0|0.0|0.0|-90.0|Guralp CMG3ESP_1
OV|HATI|HHZ|10.2986|-85.699997|56.0|0.0|0.0|-90.0|CMG-3ESP, 120 s
OV|HDC3|HHZ|10.0021|-84.111397|1175.0|0.0|0.0|-90.0|Streckeisen S
OV|HZTE|HHZ|10.7137|-85.595398|194.0|0.0|0.0|-90.0|Streckeisen ST
OV|INDI|HHZ|9.8647|-85.502197|74.0|0.0|0.0|-90.0|Streckeisen STS-
OV|JACO|HHZ|9.6624|-84.6595|85.0|0.0|0.0|-90.0|Streckeisen STS-2
OV|JUDI|HHZ|10.1659|-85.538803|702.0|0.0|0.0|-90.0|Guralp CMG3ESP
OV|LAPE|HHZ|9.807095|-84.960281|77.8|0.0|0.0|-90.0|CMG-6T, 30s-10
OV|MURC|HHZ|10.9009|-85.730103|52.0|0.0|0.0|-90.0|Guralp CMG6TD S
OV|NARJ|HHZ|9.972|-85.543701|576.0|0.0|0.0|-90.0|Guralp CMG6TD Se
OV|OCM|HHZ|9.8941|-83.962303|1607.0|0.0|0.0|-90.0|Nanometrics Tri
OV|ORTG|HHZ|10.3623|-85.459|760.0|0.0|0.0|-90.0|Nanometrics Trill
OV|PNCB|HHZ|9.5895|-85.091797|31.0|0.0|0.0|-90.0|Streckeisen STS-
OV|PNUA|HHZ|10.5955|-83.839897|49.0|0.0|0.0|-90.0|trillium compac
OV|POPE|HHZ|10.0635|-85.263397|37.0|0.0|0.0|-90.0|Guralp CMG3ESP_
OV|RIFO|HHZ|10.3172|-83.922798|139.0|0.0|0.0|-90.0|Nanometrics Tr
OV|SAJU|HHZ|10.0673|-85.710503|90.0|0.0|0.0|-90.0|Nanometrics Tri
OV|SARO|HHZ|10.843065|-85.615723|303.0|0.0|0.0|-90.0|Streckeisen
OV|TAM|HHZ|10.2635|-85.848198|5.0|0.0|0.0|-90.0|trillium compact
OV|TBLN|HHZ|9.8237|-83.999298|1596.0|0.0|0.0|-90.0|trillium compa
```

```

OV|TJAR| |HHZ|9.8504|-83.930199|1414.0|0.0|0.0|-90.0|trillium_compa
OV|VACR| |HHZ|10.472|-84.675003|370.0|0.0|0.0|-90.0|Guralp CMG3ESPC
OV|VAVL| |HHZ|10.4373|-84.709099|749.0|0.0|0.0|-90.0|trillium_compa
OV|VICI1| |HHZ|9.9732|-83.860497|3424.0|0.0|0.0|-90.0|trillium_compa
OV|VICI2| |HHZ|9.9741|-83.864197|3402.0|0.0|0.0|-90.0|trillium_compa
OV|VIMO| |HHZ|10.0242|-83.943199|1754.0|0.0|0.0|-90.0|Guralp CMG3ES
OV|VIRE| |HHZ|9.9668|-83.897499|2971.0|0.0|0.0|-90.0|trillium_compa
OV|VMAR| |HHZ|10.8125|-85.120499|580.0|0.0|0.0|-90.0|Nanometrics Tr
OV|VPCC| |HHZ|10.268144|-84.204591|1135.0|0.0|0.0|0.0|2.9947E8|1.0
OV|VPNC| |HHZ|10.201533|-84.23505|2393.0|1.0|0.0|-90.0|Compact,Tril
OV|VPPC| |HHZ|10.199833|-84.22765|2462.0|1.0|0.0|-90.0|Trillium Com
OV|VPRS| |HHZ|10.23363|-84.276914|1419.0|0.0|0.0|0.0|2.9947E8|1.0|
OV|VPTE| |HHZ|10.1718|-84.198502|2109.0|0.0|0.0|-90.0|Guralp CMG3ES
OV|VPVF| |HHZ|10.2045|-84.229698|2431.0|0.0|0.0|-90.0|Guralp CMG3ES
OV|VRBA| |HHZ|10.8664|-85.326|732.0|0.0|0.0|-90.0|Nanometrics Trill
OV|VRLE| |HHZ|10.7883|-85.365196|921.0|0.0|0.0|-90.0|Guralp CMG3ESP
SV|ARTIL|00|HHZ|13.837375|-89.365123|615.0|1.8|0.0|-90.0|Trillium
SV|BARRE|00|HHZ|13.759228|-89.268371|1383.0|1.8|0.0|-90.0|Trillium
SV|CEDA| |HHZ|13.8028|-89.394852|0.0|0.0|0.0|-90.0|Trillium-120P|4.
SV|CNCH| |HHZ|13.27633|-87.833672|1056.0|0.0|0.0|-90.0|Trillium Com
SV|JAYA| |HHZ|13.65425|-89.448898|0.0|0.0|0.0|-90.0|Trillium-120P|4
SV|MT03| |HHZ|14.39886|-89.360626|1830.0|0.0|0.0|-90.0|Trillium-120
SV|NUBE| |HHZ|13.90216|-89.779907|1306.4|0.0|0.0|-90.0|Trillium-120
SV|P5N0| |HHZ|13.985917|-88.759003|210.0|0.0|0.0|-90.0|Trillium Com
SV|PACA| |HHZ|13.469|-88.32333|1123.0|0.0|0.0|-90.0|Trillium Compac
SV|PAVA| |HHZ|13.71299|-88.937119|1019.8|0.0|0.0|-90.0|Trillium-120
SV|RBDL| |HHZ|14.113|-89.6827|1431.0|0.0|0.0|-90.0|Trillium Compact
SV|SBLS| |HHZ|13.8393|-89.623|1949.0|2.2|0.0|-90.0|Trillium Compact
SV|SNET| |HHZ|13.6868|-89.231499|775.0|0.0|0.0|-90.0|Streckeisen ST
SV|TAC0| |HHZ|13.97029|-89.353943|357.0|0.0|0.0|-90.0|Trillium-120P
SV|VSM2|00|HHZ|13.442371|-88.273294|1613.0|2.0|0.0|-90.0|Trillium
TC|LCR2| |HHZ|9.7415|-84.005333|1704.0|1.0|0.0|-90.0|OSOP|6.2483E8|
TC|MARA| |HHZ|10.0195|-85.425499|629.0|1.0|0.0|-90.0|CMG-6T, 30s-10
TC|TCS1| |HHZ|10.0421|-84.299797|629.0|1.0|0.0|-90.0|CMG-6T, 30s-10
TC|TEXA| |HHZ|10.383|-84.618|450.0|1.0|0.0|-90.0|RSN,RSN,RSN|6.2483
TC|TRT2| |HHZ|10.55736|-83.73526|46.0|0.0|0.0|-90.0|CMG-3ESP, 120 s

```

```
import requests

url = "https://service.earthscope.org/irisws/fedcatalog/1/query"

params = {
    "net": "GI",
    "sta": "RETA",
    "loc": "--",
    "cha": "HHZ",
    "starttime": str(event_time - 60),
    "endtime": str(event_time + 600),
    "targetservice": "dataselect",
    "format": "request",
    "includeoverlaps": "true"
}

response = requests.get(url, params=params)

print("Status:", response.status_code)
print(response.text)
```

Status: 200

DATACENTER=IRISDMC,<https://www.earthscope.org/data/>
 DATASELECTSERVICE=<https://service.earthscope.org/fdsnws/dataselect/1>
 GI RETA -- HHZ 2026-07-17T14:47:39 2026-07-17T14:58:39

```
from obspy import UTCDateTime

candidate_stations = [
    ("GI", "RETA"),
    ("GI", "HUCU"),
    ("GI", "SUCU"),
    ("GI", "HUEH"),
    ("GI", "ESTQ"),
    ("MX", "CCIG"),
    ("GI", "HUBA"),
    ("GI", "CMSM"),
    ("GI", "SASJ"),
    ("GI", "GUMI"),
    ("GI", "GUVU"),
    ("GI", "SRIX"),
    ("GI", "GUNB"),
    ("GI", "QCIX"),
    ("CT", "CUCB")
]
```

```

    ("GI", "GUSF"),
    ("GI", "SROR"),
    ("GI", "AVCS"),
    ("GI", "JAMO"),
    ("SV", "NUBE"),
    ("SV", "RBDL")
]

successful = []

for network, station in candidate_stations:

    try:
        st = earthscope.get_waveforms(
            network=network,
            station=station,
            location="--",
            channel="HHZ",
            starttime=event_time - 60,
            endtime=event_time + 600
        )

        print("DATA FOUND:", network, station)
        print(st)

        successful.append((network, station, st))

    except Exception:
        print("No data:", network, station)

print("\nSuccessful stations:", len(successful))

```

```

No data: GI RETA
DATA FOUND: GI HUCU
1 Trace(s) in Stream:
GI.HUCU..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880
DATA FOUND: GI SUCU
1 Trace(s) in Stream:
GI.SUCU..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880
No data: GI HUEH
No data: GI ESTQ
No data: MX CCIG
No data: GI HUBA
No data: GI CMSM
No data: GI SASJ
No data: GI GUMI
DATA FOUND: GI GUVU
1 Trace(s) in Stream:
GI.GUVU..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880
No data: GI SRIX

```

```

No data: GI GUNB
DATA FOUND: GI QCIX
1 Trace(s) in Stream:
GI.QCIX..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z
DATA FOUND: GI GUSP
1 Trace(s) in Stream:
GI.GUSP..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z
DATA FOUND: GI SROR
1 Trace(s) in Stream:
GI.SROR..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z
DATA FOUND: GI AVCS
1 Trace(s) in Stream:
GI.AVCS..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880000Z
No data: GI JAMO
DATA FOUND: SV NUBE
1 Trace(s) in Stream:
SV.NUBE..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z
DATA FOUND: SV RBDL
1 Trace(s) in Stream:
SV.RBDL..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z

Successful stations: 9

```

```

st_hucu = earthscope.get_waveforms(
    network="GI",
    station="HUCU",
    location="--",
    channel="HHZ",
    starttime=event_time - 60,
    endtime=event_time + 600
)

print(st_hucu)

```

```

1 Trace(s) in Stream:
GI.HUCU..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880000Z

```

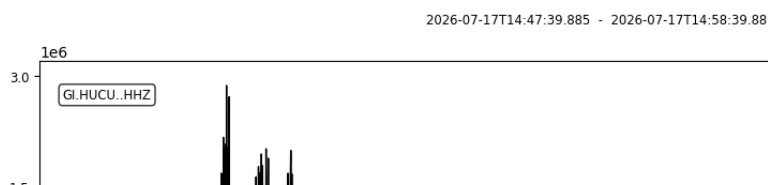
```

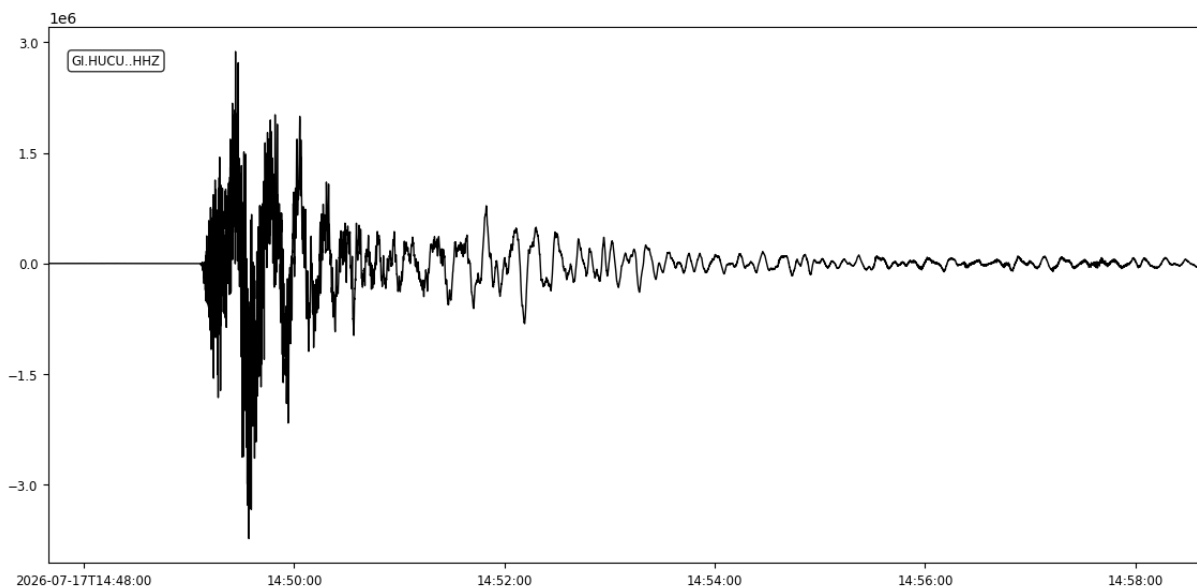
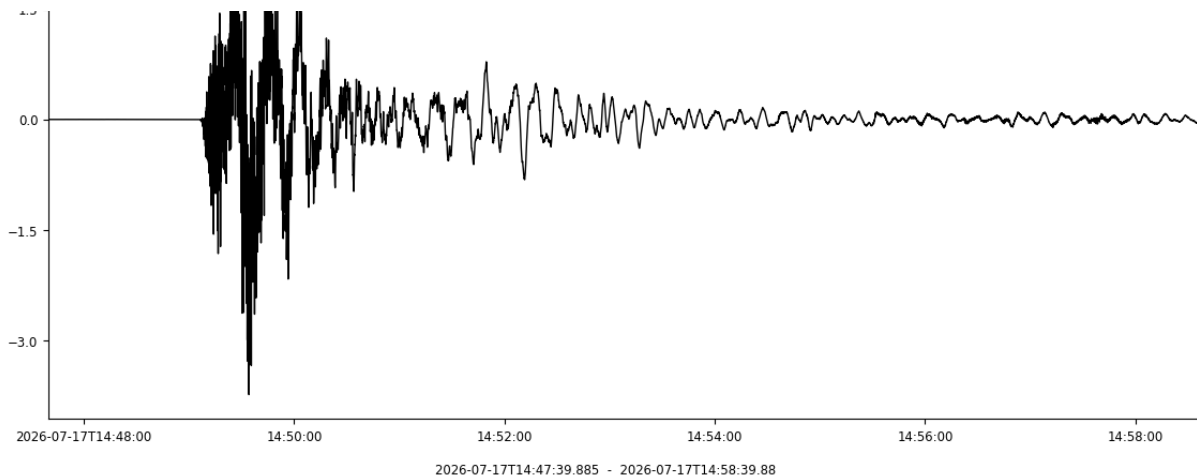
st_hucu.plot(

    size=(1200,600),
    equal_scale=False

)

```





```
st_processed = st_hucu.copy()
print(st_processed)
```

```
1 Trace(s) in Stream:
GI.HUCU..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880000Z | 200.0 Hz, 132000 samples
```

```
"""dtrending: Real seismic recordings can contain a slow baseline drift.
detrend("linear") removes a best-fit straight-line trend from the data.

st_processed.detrend("linear")
```

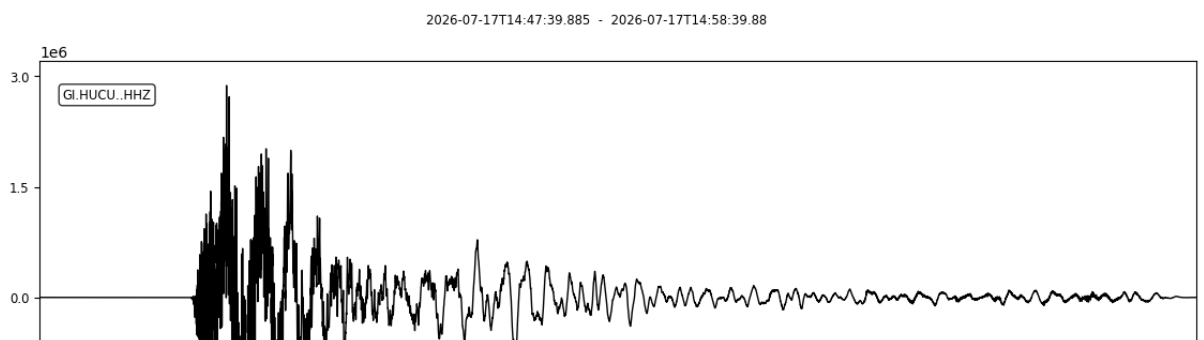
```
1 Trace(s) in Stream:
GI.HUCU..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880000Z | 200.0 Hz, 132000 samples
```

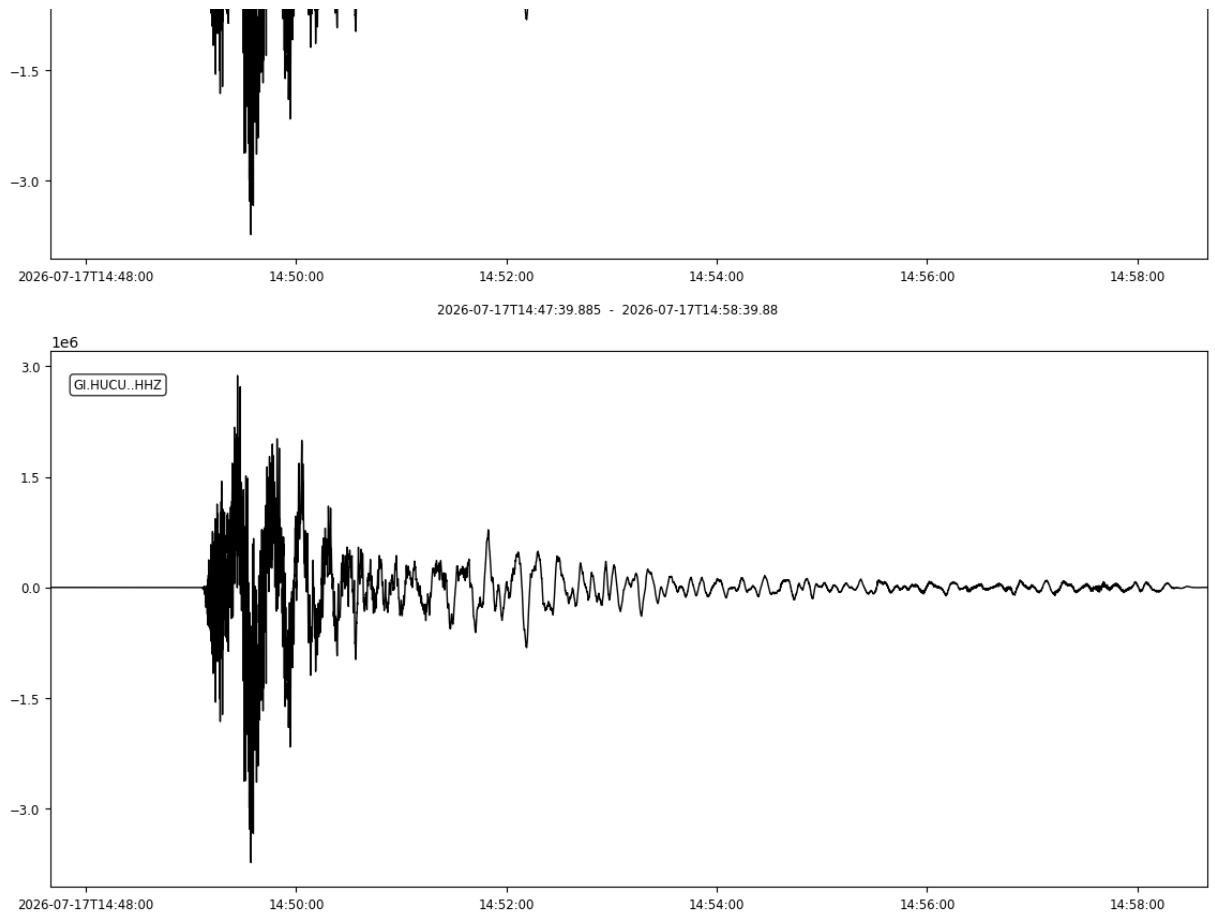
```
"""This gradually reduces the first and last 5% of the trace toward zero.

st_processed.taper(max_percentage=0.05)
```

```
1 Trace(s) in Stream:
GI.HUCU..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880000Z | 200.0 Hz, 132000 samples
```

```
st_processed.plot(
    size=(1200, 600),
    equal_scale=False
)
```





```
print("Raw mean:", st_hucu[0].data.mean())  
print("Processed mean:", st_processed[0].data.mean())  
  
print("Raw max:", st_hucu[0].data.max())  
print("Processed max:", st_processed[0].data.max())
```

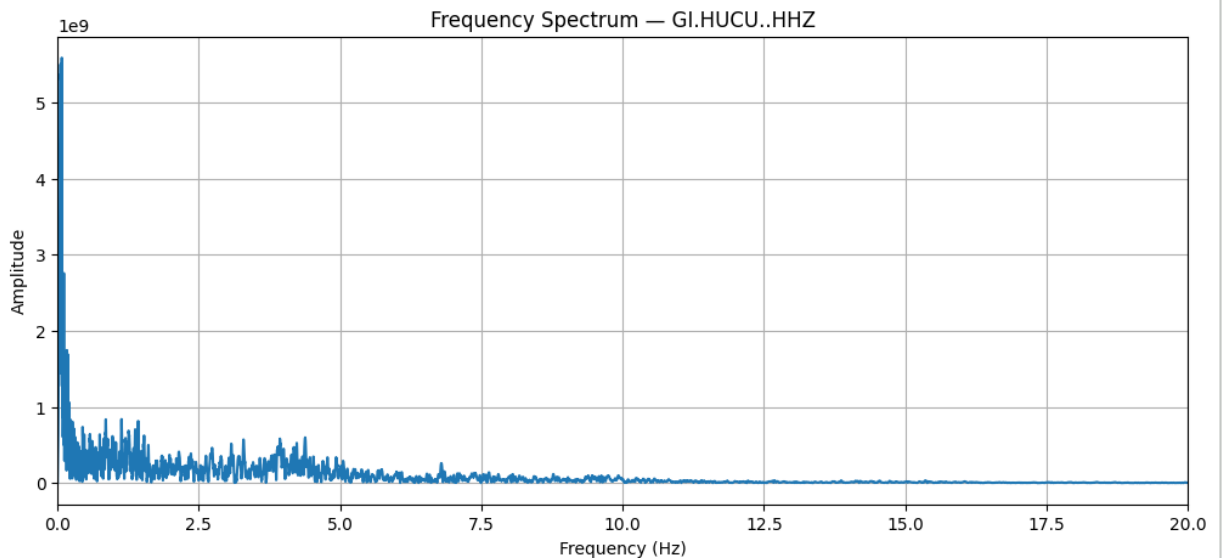
```
Raw mean: 106.9764696969697  
Processed mean: 48.13911269888381  
Raw max: 2872991  
Processed max: 2873067.206905954
```

```
tr = st_processed[0]

data = tr.data
n = len(data)
dt = tr.stats.delta

freq = np.fft.rfftfreq(n, d=dt)
spectrum = np.abs(np.fft.rfft(data))

plt.figure(figsize=(12, 5))
plt.plot(freq, spectrum)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.title("Frequency Spectrum — GI.HUCU..HHZ")
plt.xlim(0, 20)
plt.grid()
plt.show()
```

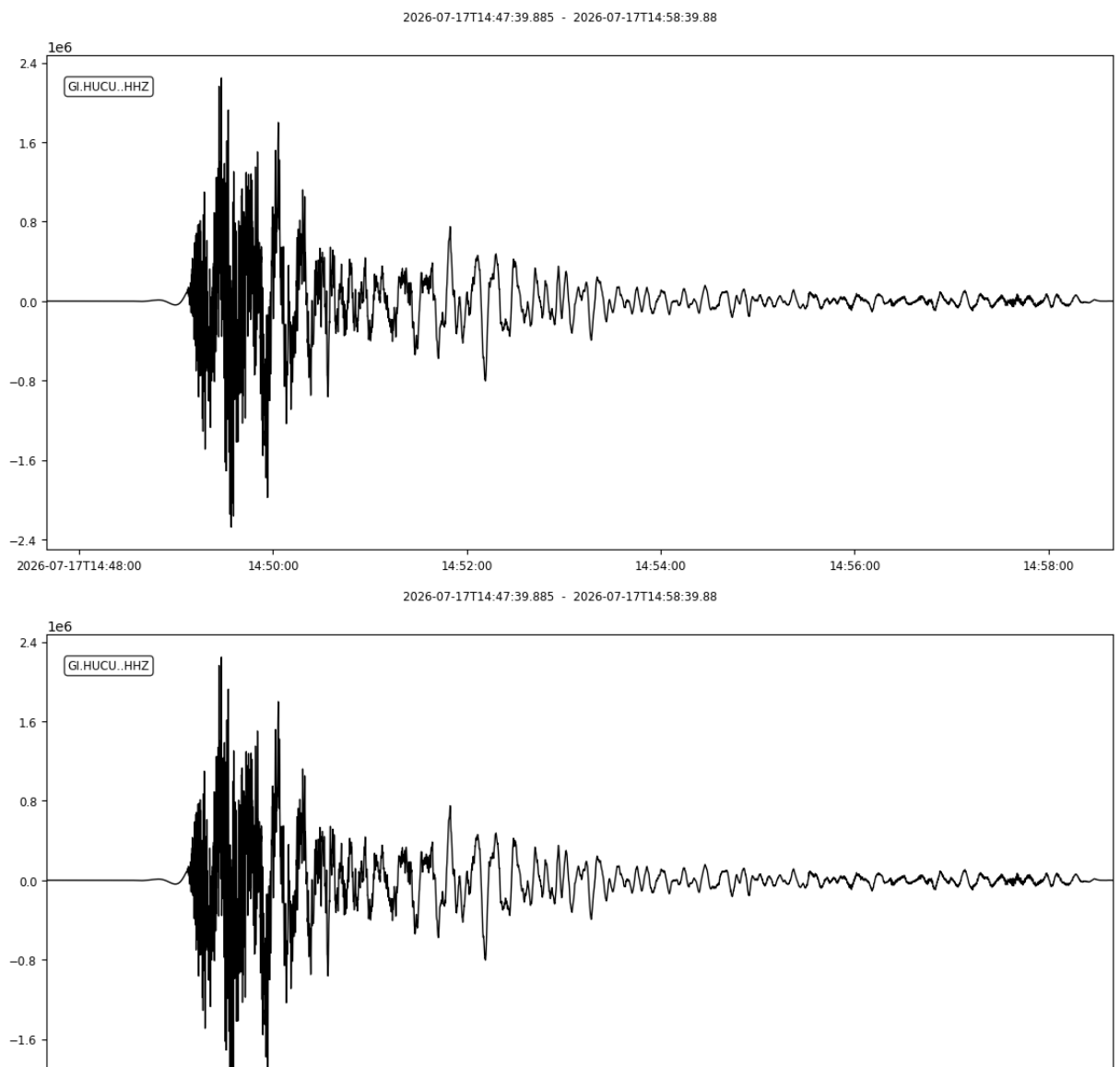


```
st_filtered = st_processed.copy()

st_filtered.filter(
    "bandpass",
    freqmin=0.05,
    freqmax=5.0,
    corners=4,
    zerophase=True
)
```

1 Trace(s) in Stream:
GI.HUCU..HHZ | 2026-07-17T14:47:39.885000Z - 2026-07-17T14:58:39.880000Z | 200.0 Hz, 132000 samples

```
st_filtered.plot(
    size=(1200, 600),
    equal_scale=False
)
```





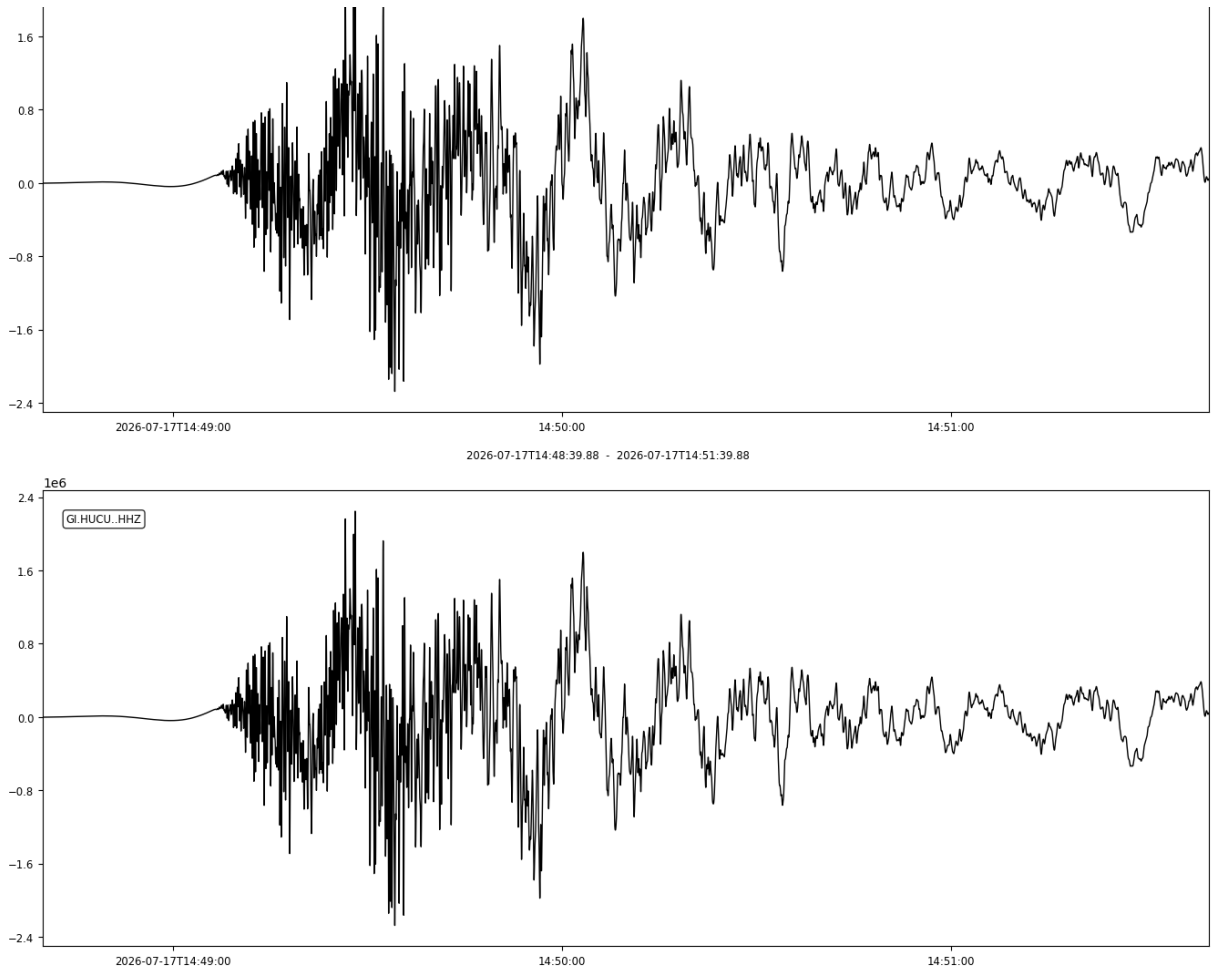
```
t1 = event_time
t2 = event_time + 180

st_zoom = st_filtered.copy()
st_zoom.trim(t1, t2)

st_zoom.plot(
    size=(1400, 600),
    equal_scale=False
)
```

2026-07-17T14:48:39.88 - 2026-07-17T14:51:39.88

2.4
1e6
GI.HUCU.HHZ



```
from obspy.signal.trigger import classic_sta_lta

tr = st_filtered[0]

df = tr.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft = classic_sta_lta(
    tr.data,
    sta_samples,
    lta_samples
)

print("Sampling rate:", df)
print("STA window:", sta_seconds, "seconds")
print("LTA window:", lta_seconds, "seconds")
```

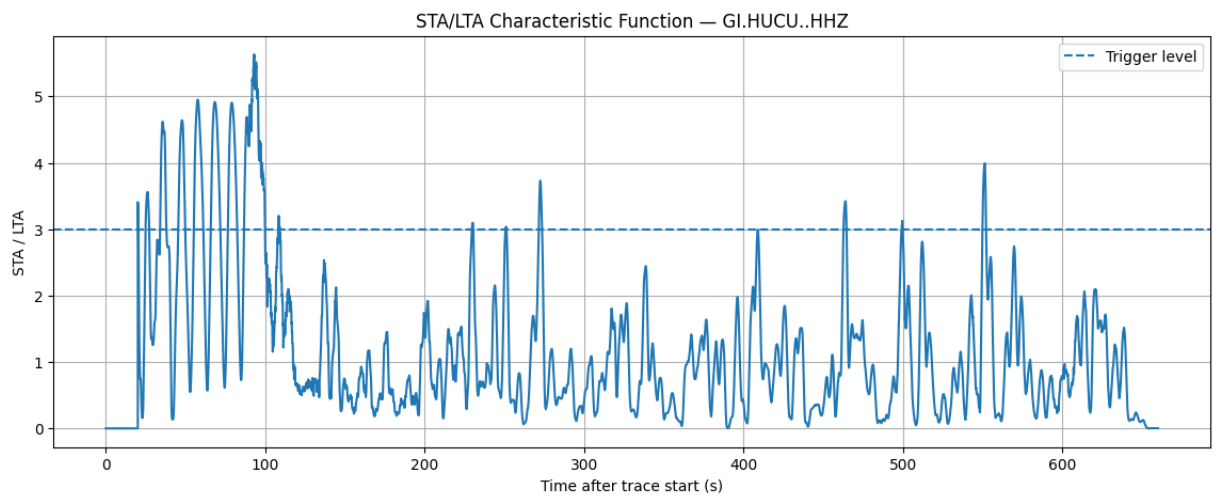
```
Sampling rate: 200.0
STA window: 3 seconds
LTA window: 20 seconds
```

```
time = np.arange(len(cft)) / df

plt.figure(figsize=(14, 5))
plt.plot(time, cft)

plt.axhline(3, linestyle="--", label="Trigger level")
plt.xlabel("Time after trace start (s)")
plt.ylabel("STA / LTA")
plt.title("STA/LTA Characteristic Function – GI.HUCU..HHZ")
plt.legend()
plt.grid()

plt.show()
```



```
tr_pick = st_processed[0].copy()

# Keeping 30 s before origin to 180 s after origin
tr_pick.trim(
    starttime=event_time - 30,
    endtime=event_time + 180
)

print(tr_pick)
```

GI.HUCU..HHZ | 2026-07-17T14:48:09.880000Z - 2026-07-17T14:51:39.880000Z

```
from obspy.signal.trigger import classic_sta_lta

df = tr_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft = classic_sta_lta(
    tr_pick.data,
    sta_samples,
    lta_samples
)
```

```

import numpy as np
import matplotlib.pyplot as plt

time = np.arange(len(cft)) / df - 30

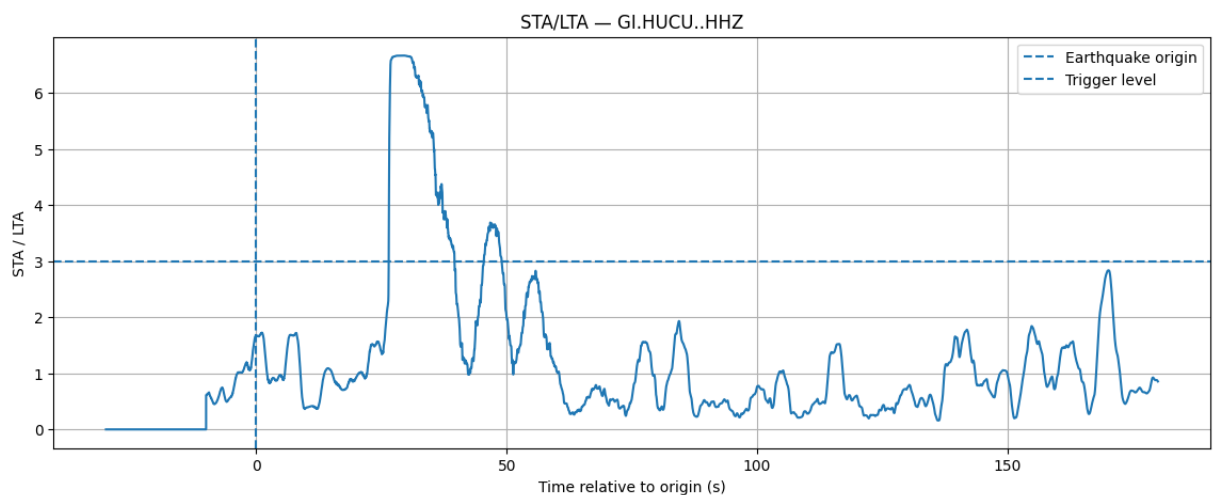
plt.figure(figsize=(14, 5))
plt.plot(time, cft)

plt.axvline(0, linestyle="--", label="Earthquake origin")

plt.axhline(3, linestyle="--", label="Trigger level")

plt.xlabel("Time relative to origin (s)")
plt.ylabel("STA / LTA")
plt.title("STA/LTA - GI.HUCU..HHZ")
plt.legend()
plt.grid()
plt.show()

```



```
tr_plot = tr_pick.copy()
```

```
time = tr_plot.times(rettime=event_time)

plt.figure(figsize=(14, 6))

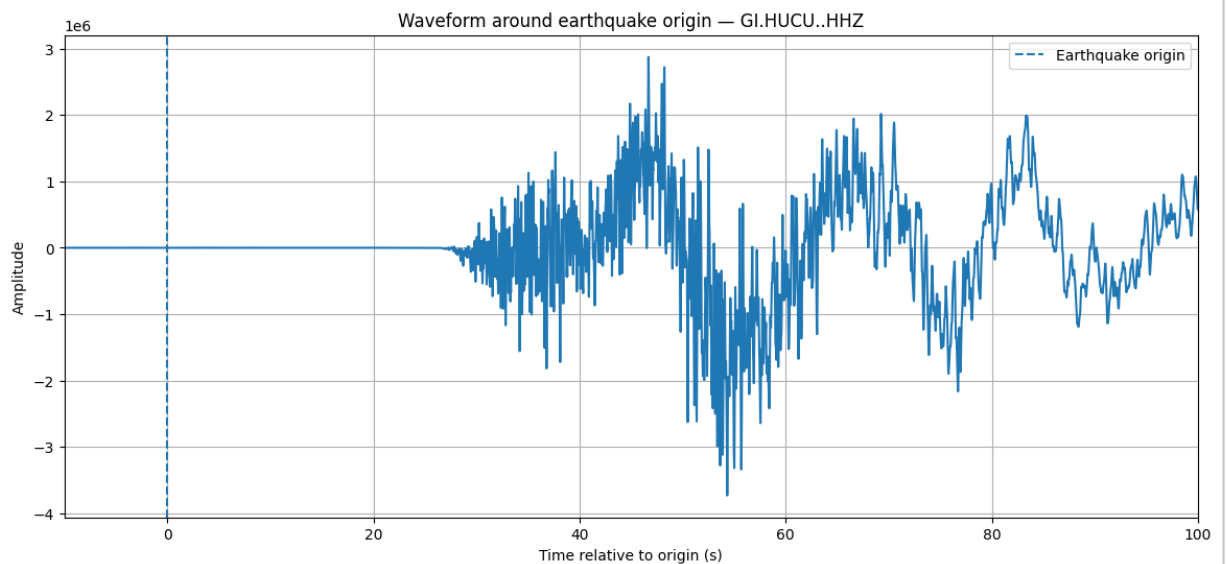
plt.plot(time, tr_plot.data)

plt.axvline(0, linestyle="--", label="Earthquake origin")

plt.xlim(-10, 100)

plt.xlabel("Time relative to origin (s)")
plt.ylabel("Amplitude")
plt.title("Waveform around earthquake origin – GI.HUCU..HHZ")
plt.legend()
plt.grid()

plt.show()
```



```
from obspy.taup import TauPyModel
from obspy.geodetics import locations2degrees

# Earthquake coordinates
event_lat = 14.57
event_lon = -92.96
event_depth_km = 22.0

# HUCU station coordinates
station_lat = 15.407847
station_lon = -91.947188

# Epicentral angular distance
distance_deg = locations2degrees(
    event_lat,
    event_lon,
    station_lat,
    station_lon
)

print("Epicentral distance:", distance_deg, "degrees")
```

Epicentral distance: 1.2880751896687836 degrees

```
model = TauPyModel(model="iasp91")

arrivals = model.get_travel_times(
    source_depth_in_km=event_depth_km,
    distance_in_degree=distance_deg,
    phase_list=["P", "S"]
)

for arrival in arrivals:
    print(
        arrival.name,
        "travel time =",
        round(arrival.time, 2),
        "seconds"
    )
```

P travel time = 22.66 seconds
P travel time = 24.07 seconds
S travel time = 39.89 seconds
S travel time = 41.67 seconds

```

for arrival in arrivals:
    print(
        f"{arrival.name:5s}  "
        f"time = {arrival.time:7.2f} s  "
        f"ray parameter = {arrival.ray_param_sec_degree:.3f}"
    )

```

```

P      time =    22.66 s  ray parameter = 13.753
P      time =    24.07 s  ray parameter = 16.494
S      time =    39.89 s  ray parameter = 24.737
S      time =    41.67 s  ray parameter = 28.580

```

```

plt.figure(figsize=(14, 6))

time = tr_plot.times(reftime=event_time)

plt.plot(time, tr_plot.data)

# Theoretical arrivals
plt.axvline(22.66, linestyle="--", label="Theoretical P 22.66 s")
plt.axvline(24.07, linestyle="--", label="Theoretical P 24.07 s")

plt.axvline(39.89, linestyle="--", label="Theoretical S 39.89 s")
plt.axvline(41.67, linestyle="--", label="Theoretical S 41.67 s")

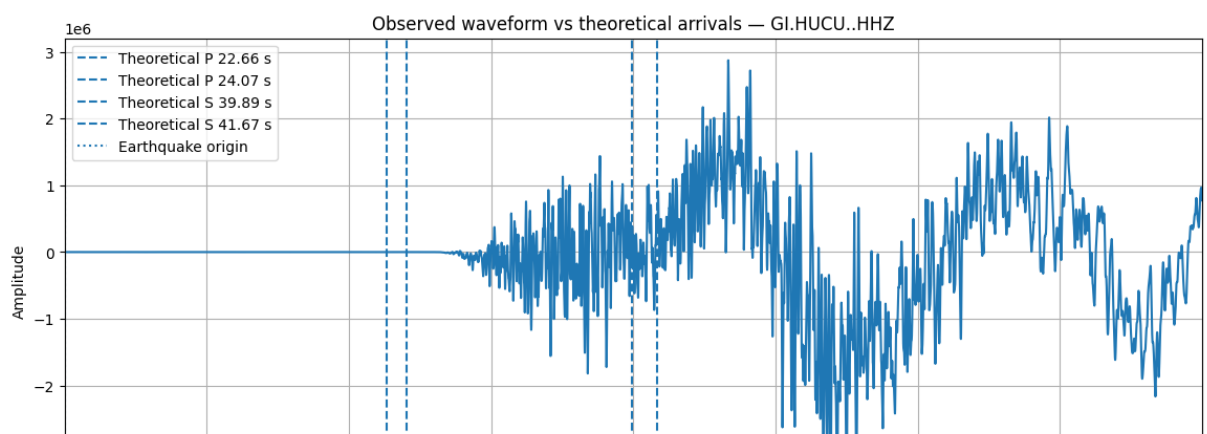
plt.axvline(0, linestyle=":", label="Earthquake origin")

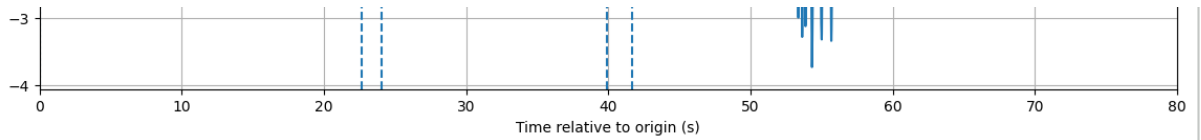
plt.xlim(0, 80)

plt.xlabel("Time relative to origin (s)")
plt.ylabel("Amplitude")
plt.title("Observed waveform vs theoretical arrivals – GI.HUCU..HHZ")
plt.legend()
plt.grid()

plt.show()

```





```
# Looking for the maximum STA/LTA value between 20 and 35 seconds

mask = (time >= 20) & (time <= 35)

peak_index = np.argmax(cft[mask])

candidate_times = time[mask]

p_candidate = candidate_times[peak_index]

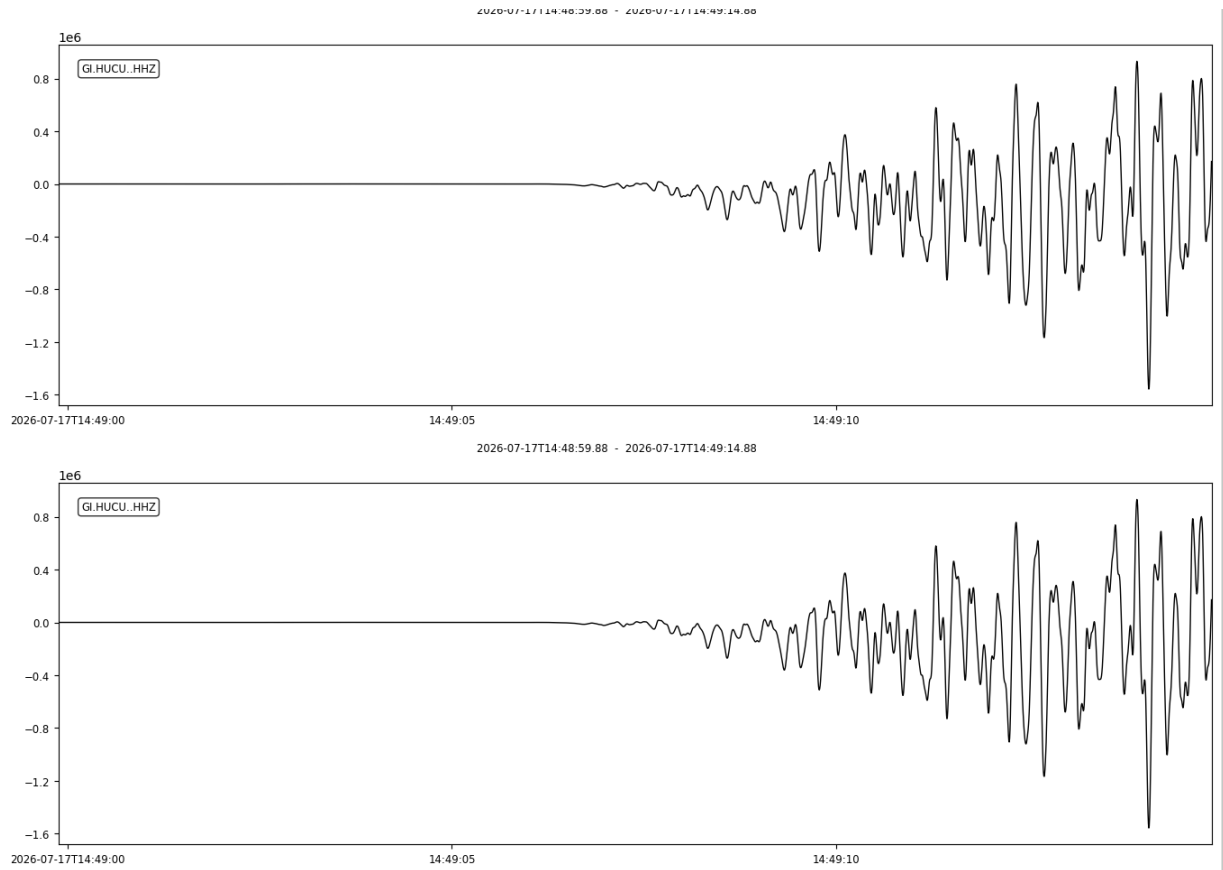
print("Candidate P-related time:", p_candidate, "seconds")
print("STA/LTA value:", cft[mask][peak_index])
```

```
Candidate P-related time: 29.453999999999997 seconds
STA/LTA value: 6.666455508306895
```

```
tr_p = tr_pick.copy()

tr_p.trim(
    starttime=event_time + 20,
    endtime=event_time + 35
)

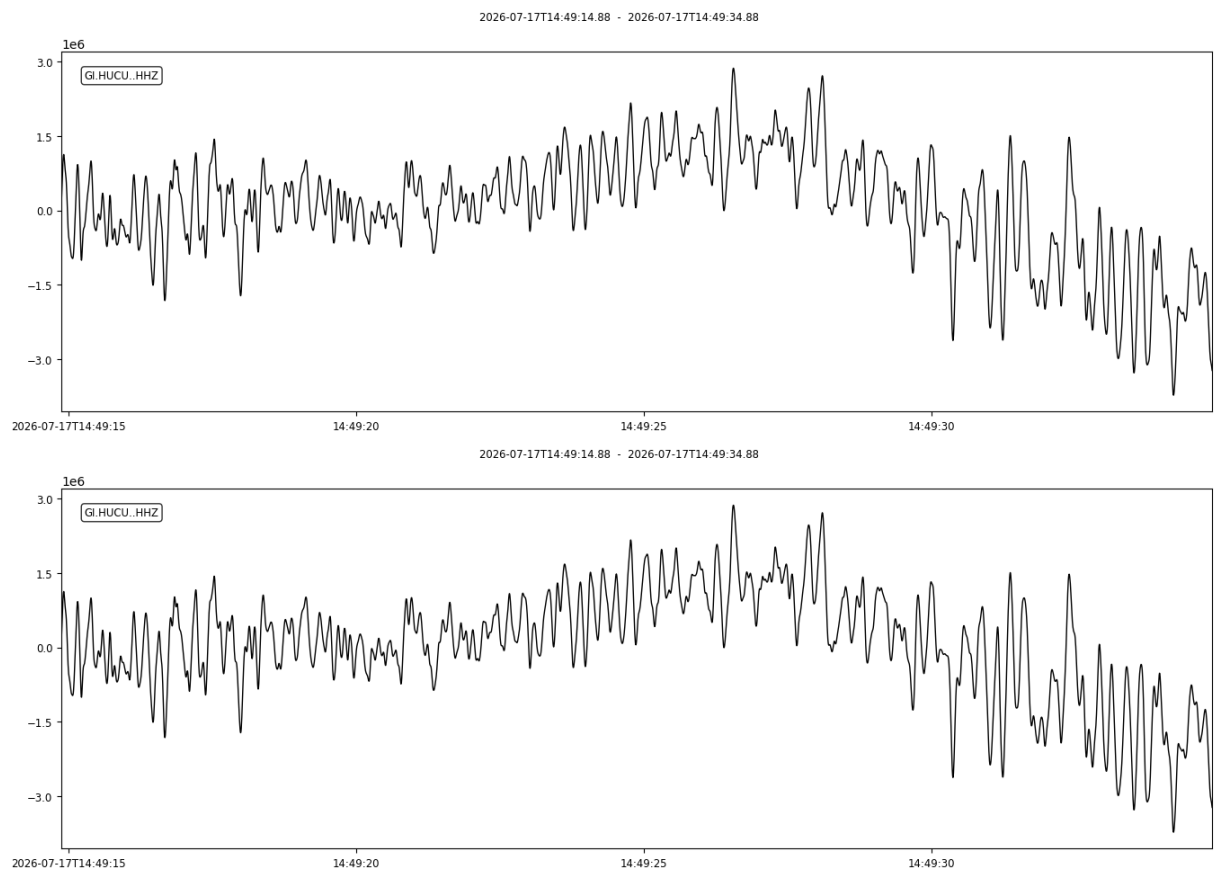
tr_p.plot(
    size=(1400, 500),
    equal_scale=False
)
```



```
tr_s = tr_pick.copy()

tr_s.trim(
    starttime=event_time + 35,
    endtime=event_time + 55
)

tr_s.plot(
    size=(1400, 500),
    equal_scale=False
)
```



```
p_theory = 22.66
s_theory = 39.89
```

```
p_obs = 24.5
s_obs = 41.0
```

```
plt.figure(figsize=(14, 6))

time = tr_plot.times(reftime=event_time)

plt.plot(time, tr_plot.data, label="Observed waveform")

plt.axvline(0, linestyle=":", label="Earthquake origin")

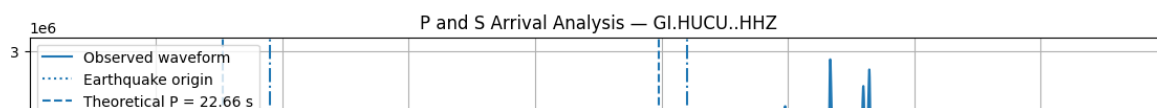
plt.axvline(p_theory, linestyle="--", label="Theoretical P = 22.66 s")
plt.axvline(s_theory, linestyle="--", label="Theoretical S = 39.89 s")

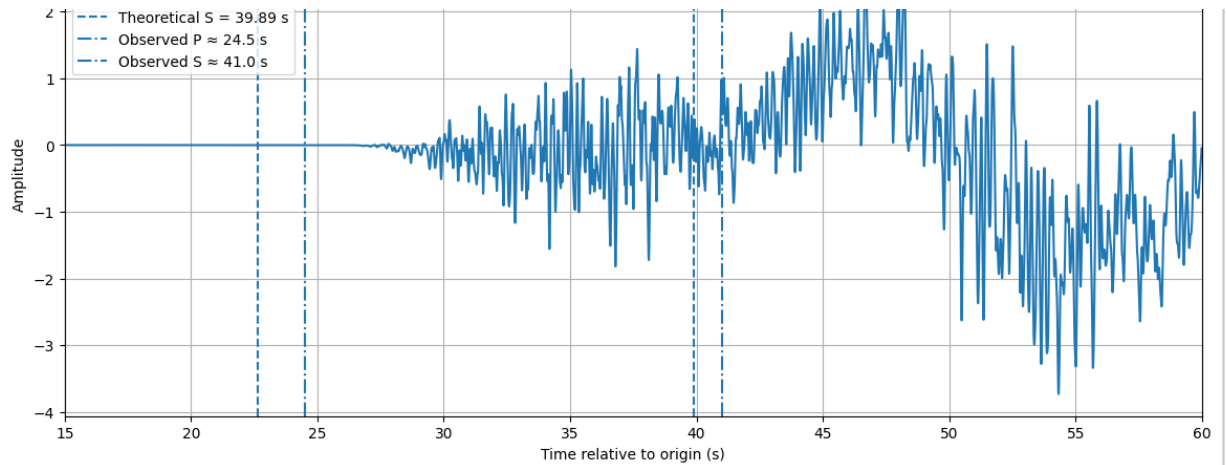
plt.axvline(p_obs, linestyle="-.", label="Observed P ≈ 24.5 s")
plt.axvline(s_obs, linestyle="-.", label="Observed S ≈ 41.0 s")

plt.xlim(15, 60)

plt.xlabel("Time relative to origin (s)")
plt.ylabel("Amplitude")
plt.title("P and S Arrival Analysis – GI.HUCU..HHZ")
plt.legend()
plt.grid()

plt.show()
```





```
sp_time = s_obs - p_obs

print("Observed P:", p_obs, "s")
print("Observed S:", s_obs, "s")
print("Observed S-P:", sp_time, "s")
```

```
Observed P: 24.5 s
Observed S: 41.0 s
Observed S-P: 16.5 s
```

```
from obspy.taup import TauPyModel
import numpy as np

model = TauPyModel(model="iasp91")

source_depth = 22.0
observed_sp = 16.5

distances = np.linspace(0.1, 5.0, 490)
```

```
best_distance = None
best_difference = np.inf
best_p = None
best_s = None

for distance in distances:

    arrivals = model.get_travel_times(
        source_depth_in_km=source_depth,
        distance_in_degree=distance,
        phase_list=["P", "S"]
    )

    p_arrivals = [a.time for a in arrivals if a.name == "P"]
    s_arrivals = [a.time for a in arrivals if a.name == "S"]

    if p_arrivals and s_arrivals:

        sp = s_arrivals[0] - p_arrivals[0]

        difference = abs(sp - observed_sp)

        if difference < best_difference:
            best_difference = difference
            best_distance = distance
            best_p = p_arrivals[0]
            best_s = s_arrivals[0]

print("Estimated angular distance:", best_distance, "degrees")
print("Theoretical P time at this distance:", best_p, "s")
print("Theoretical S time at this distance:", best_s, "s")
print("Theoretical S-P:", best_s - best_p, "s")
print("Difference from observed S-P:", best_difference, "s")
```

```
Estimated angular distance: 1.2222903885480574 degrees
Theoretical P time at this distance: 21.750568338685042 s
Theoretical S time at this distance: 38.258427439045754 s
Theoretical S-P: 16.50785910036071 s
Difference from observed S-P: 0.00785910036071158 s
```

```
from obspy.geodetics import locations2degrees

event_lat = 14.57
event_lon = -92.96

station_lat = 15.407847
station_lon = -91.947188

actual_distance_deg = locations2degrees(
    event_lat,
    event_lon,
    station_lat,
    station_lon
)

actual_distance_km = actual_distance_deg * 111.195

print("Actual angular distance:", actual_distance_deg, "degrees")
print("Approximate distance:", actual_distance_km, "km")
```

```
Actual angular distance: 1.2880751896687836 degrees
Approximate distance: 143.22752071522038 km
```

```
successful_stations = [
    ("GI", "HUCU"),
    ("GI", "SUCU"),
    ("GI", "GUVC"),
    ("GI", "QCIX"),
    ("GI", "GUSP"),
    ("GI", "SROR"),
    ("GI", "AVCS"),
    ("SV", "NUBE"),
    ("SV", "RBDL")
]

print("Number of successful stations:", len(successful_stations))
```

```
Number of successful stations: 9
```

```
all_waveforms = {}

for network, station in successful_stations:

    try:
        st = earthscope.get_waveforms(
            network=network,
            station=station,
            location="--",
            channel="HHZ",
            starttime=event_time - 60,
            endtime=event_time + 600
        )

        all_waveforms[f"{network}.{station}"] = st

        print(
            f"Downloaded: {network}.{station} | "
            f"{len(st)} trace(s)"
        )

    except Exception as e:
        print(
            f"Failed: {network}.{station} | {type(e).__name__}"
        )

print("\nTotal successful stations:", len(all_waveforms))
```

```
Downloaded: GI.HUCU | 1 trace(s)
Downloaded: GI.SUCU | 1 trace(s)
Downloaded: GI.GUVC | 1 trace(s)
Downloaded: GI.QCIX | 1 trace(s)
Downloaded: GI.GUSP | 1 trace(s)
Downloaded: GI.SROR | 1 trace(s)
Downloaded: GI.AVCS | 1 trace(s)
Downloaded: SV.NUBE | 1 trace(s)
Downloaded: SV.RBDL | 1 trace(s)
```

```
Total successful stations: 9
```

```

processed_waveforms = {}

for station_name, st in all_waveforms.items():

    st_proc = st.copy()

    # Remove linear trend
    st_proc.detrend("linear")

    # Taper 5% at both ends
    st_proc.taper(max_percentage=0.05)

    # Band-pass filter
    st_proc.filter(
        "bandpass",
        freqmin=0.05,
        freqmax=5.0,
        corners=4,
        zerophase=True
    )

    processed_waveforms[station_name] = st_proc

    print(f"Processed: {station_name}")

```

```

Processed: GI.HUCU
Processed: GI.SUCU
Processed: GI.GUVC
Processed: GI.QCIX
Processed: GI.GUSP
Processed: GI.SROR
Processed: GI.AVCS
Processed: SV.NUBE
Processed: SV.RBDL

```

```

fig, axes = plt.subplots(9, 1, figsize=(14, 20), sharex=True)

for ax, (station_name, st) in zip(axes, processed_waveforms.items()):

    tr = st[0]

    time = tr.times(reftime=event_time)

    ax.plot(time, tr.data)
    ax.axvline(0, linestyle="--", label="Origin time")

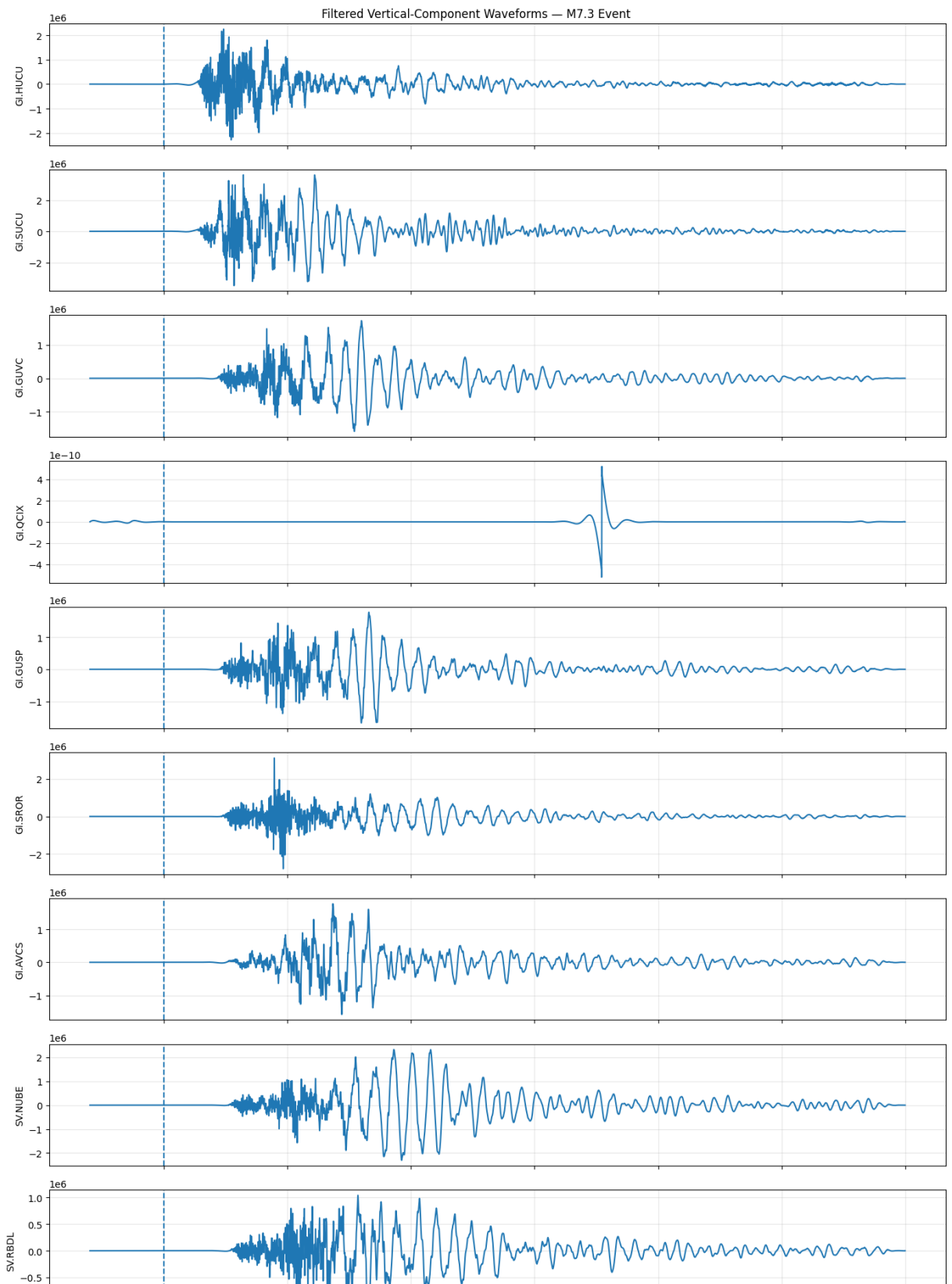
    ax.set_ylabel(station_name)

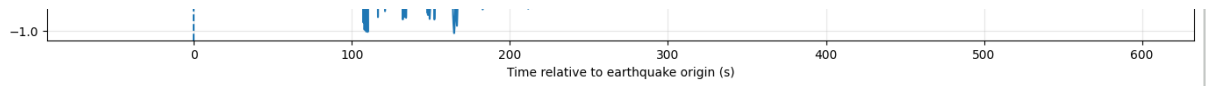
```

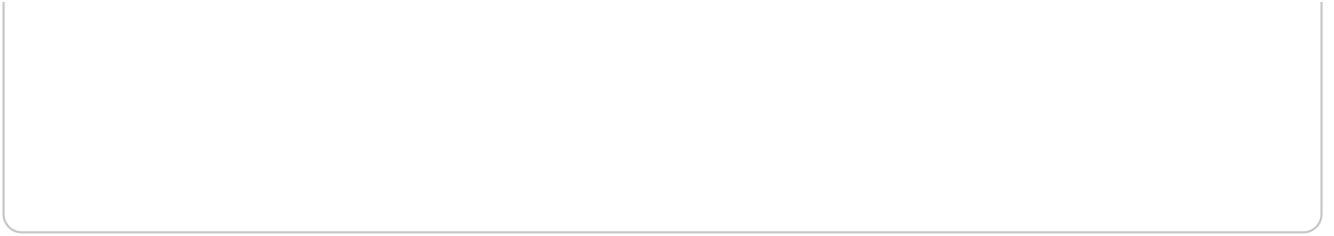
```
ax.grid(True, alpha=0.3)

axes[-1].set_xlabel("Time relative to earthquake origin (s)")

plt.suptitle("Filtered Vertical-Component Waveforms — M7.3 Event")
plt.tight_layout()
plt.show()
```







```
station_name = "GI.SUCU"

tr = processed_waveforms[station_name][0]

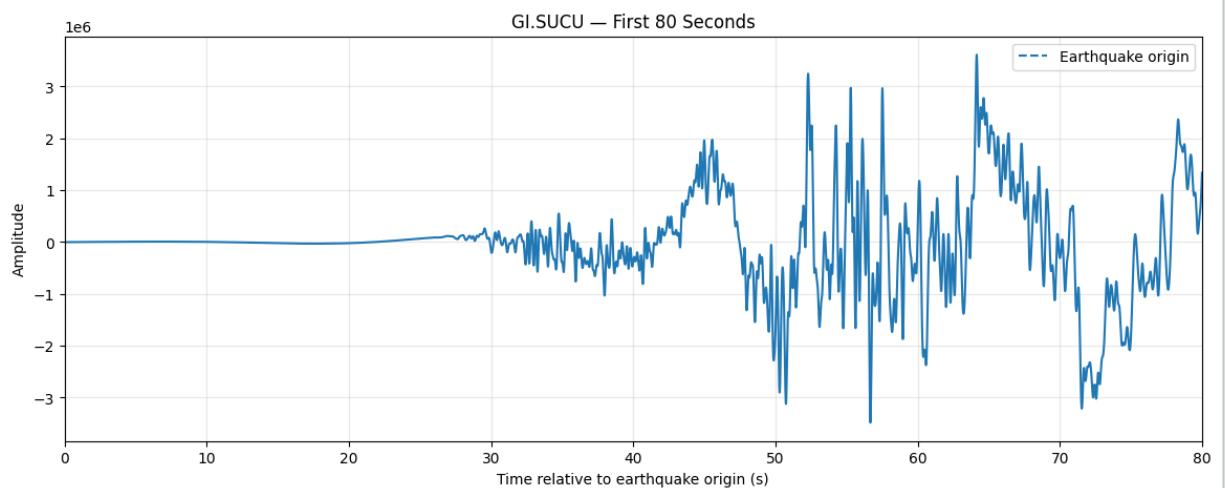
time = tr.times(reftime=event_time)

plt.figure(figsize=(14, 5))
plt.plot(time, tr.data)

plt.axvline(0, linestyle="--", label="Earthquake origin")

plt.xlim(0, 80)
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title(f"{station_name} - First 80 Seconds")
plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
tr = processed_waveforms["GI.SUCU"][0]

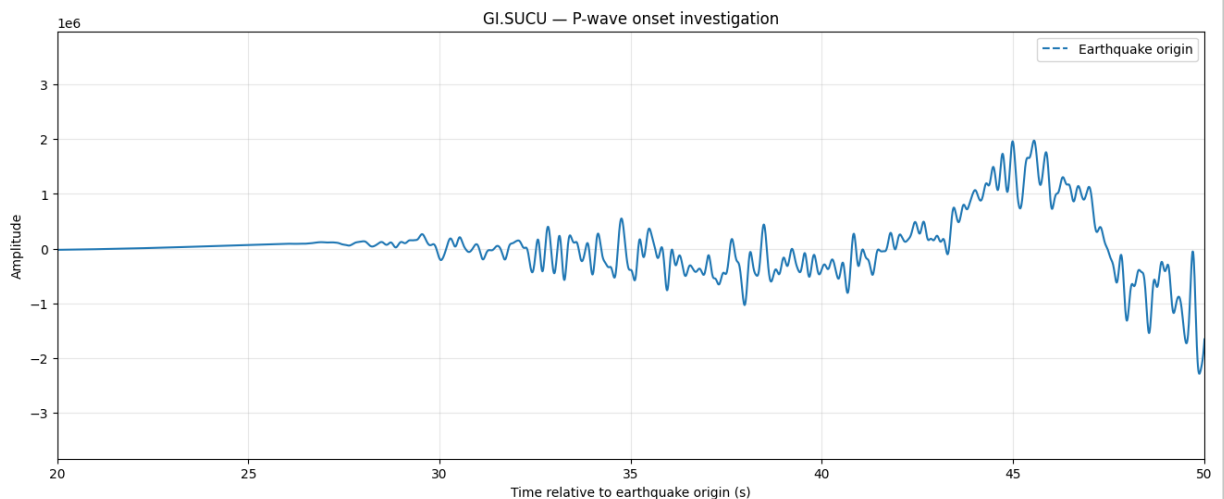
time = tr.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time, tr.data)

plt.axvline(0, linestyle="--", label="Earthquake origin")

plt.xlim(20, 50)
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.SUCU - P-wave onset investigation")
plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```

from obspy.signal.trigger import classic_sta_lta

tr_sucu = processed_waveforms["GI.SUCU"][0].copy()

# Focus on the useful time window
tr_sucu.trim(
    starttime=event_time - 10,
    endtime=event_time + 80
)

df = tr_sucu.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_sucu = classic_sta_lta(
    tr_sucu.data,
    sta_samples,
    lta_samples
)

time_sucu = np.arange(len(cft_sucu)) / df - 10

plt.figure(figsize=(14, 5))

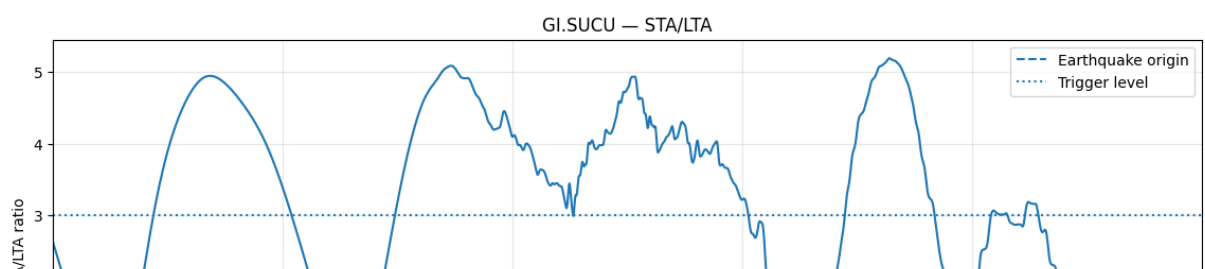
plt.plot(time_sucu, cft_sucu)
plt.axvline(0, linestyle="--", label="Earthquake origin")

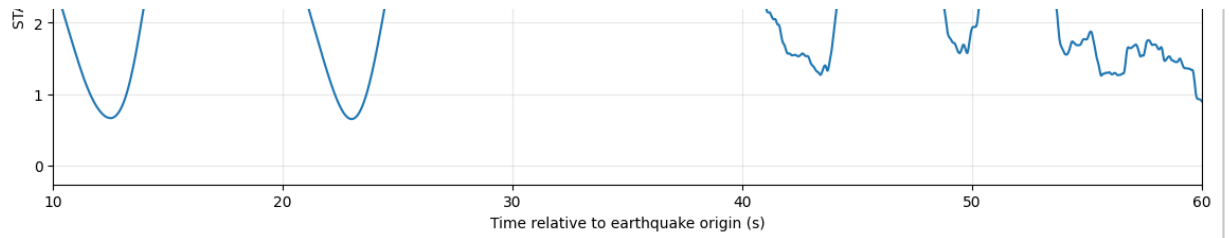
plt.axhline(3, linestyle=":", label="Trigger level")

plt.xlim(10, 60)
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.SUCU - STA/LTA")
plt.grid(True, alpha=0.3)
plt.legend()

plt.show()

```





```
from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

# SUCU coordinates
sucu_lat = 15.529
sucu_lon = -92.103

distance_sucu = locations2degrees(
    14.57, -92.96,
    sucu_lat, sucu_lon
)

model = TauPyModel(model="iasp91")

arrivals_sucu = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_sucu,
    phase_list=["P", "S"]
)

print("Distance:", distance_sucu, "degrees")

for arrival in arrivals_sucu:
    print(
        arrival.name,
        ":",
        round(arrival.time, 2),
        "s"
    )
```

```
Distance: 1.266725381653031 degrees
P : 22.36 s
P : 23.72 s
S : 39.36 s
S : 41.06 s
```

```

from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

# SUCU coordinates
sucu_lat = 15.529
sucu_lon = -92.103

# Source-station angular distance
distance_sucu = locations2degrees(
    14.57, -92.96,
    sucu_lat, sucu_lon
)

# IASP91 Earth model
model = TauPyModel(model="iasp91")

arrivals_sucu = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_sucu,
    phase_list=["P", "S"]
)

print("Distance:", distance_sucu, "degrees")

for arrival in arrivals_sucu:
    print(arrival.name, ":", round(arrival.time, 2), "s")

```

```

Distance: 1.266725381653031 degrees
P : 22.36 s
P : 23.72 s
S : 39.36 s
S : 41.06 s

```

```

tr_guvc = processed_waveforms["GI.GUVC"][0]

time_guvc = tr_guvc.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time_guvc, tr_guvc.data)

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

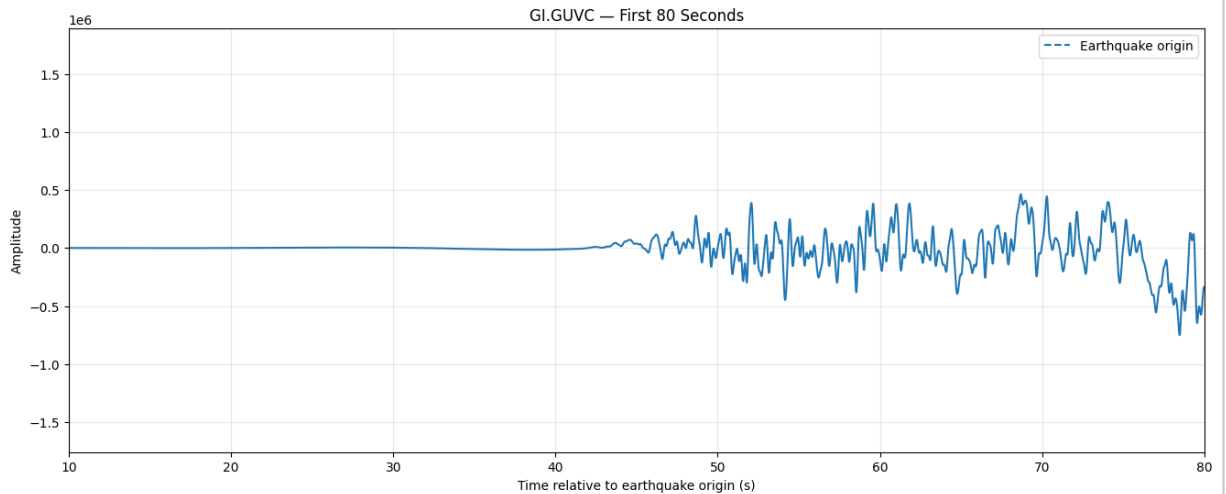
plt.xlim(10, 80)

```

```
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUVC - First 80 Seconds")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



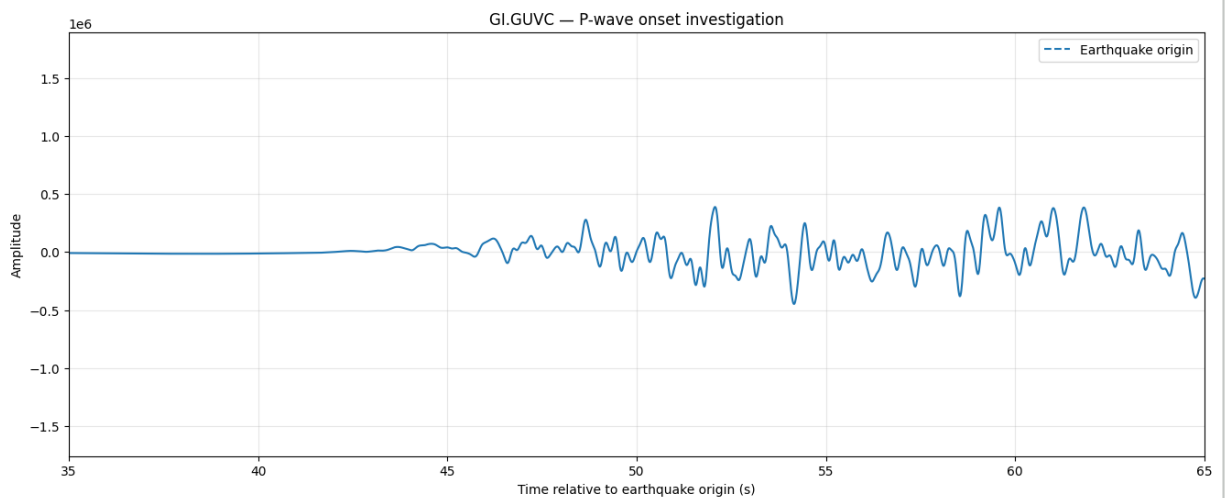
```
tr_guvc = processed_waveforms["GI.GUVC"][0]

time_guvc = tr_guvc.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time_guvc, tr_guvc.data)

plt.axvline(
    0,
    linestyle="--",
```

```
        label="Earthquake origin"  
    )  
  
plt.xlim(35, 65)  
  
plt.xlabel("Time relative to earthquake origin (s)")  
plt.ylabel("Amplitude")  
plt.title("GI.GUVC – P-wave onset investigation")  
  
plt.grid(True, alpha=0.3)  
plt.legend()  
  
plt.show()
```



```
from obspy.signal.trigger import classic_sta_lta  
  
tr_guvc_pick = processed_waveforms["GI.GUVC"][0].copy()  
  
tr_guvc_pick.trim(  

```

```

        starttime=event_time - 10,
        endtime=event_time + 120
    )

df = tr_guvc_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_guvc = classic_sta_lta(
    tr_guvc_pick.data,
    sta_samples,
    lta_samples
)

time_guvc_sta = np.arange(len(cft_guvc)) / df - 10

plt.figure(figsize=(14, 5))

plt.plot(time_guvc_sta, cft_guvc)

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

plt.xlim(10, 100)

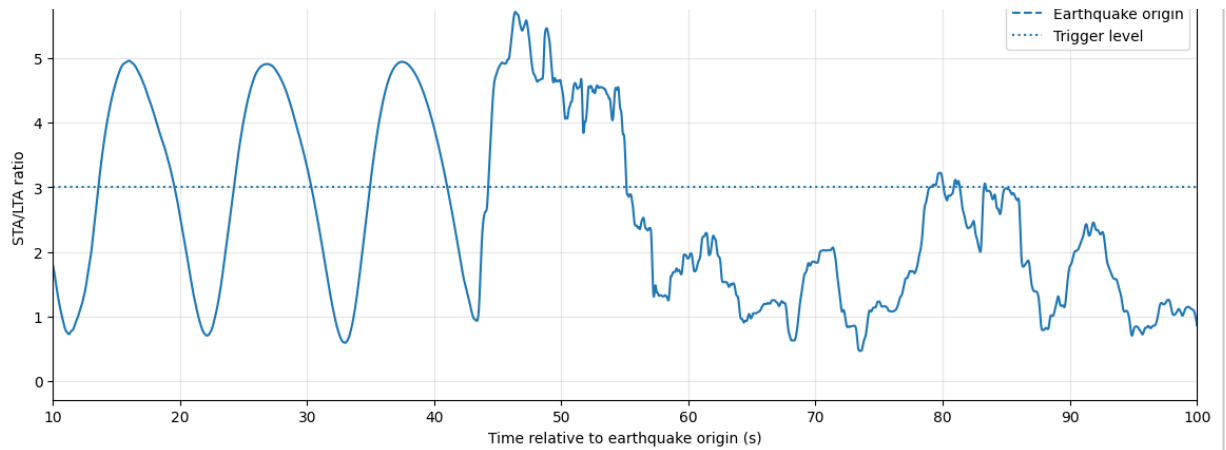
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.GUVC - STA/LTA")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()

```

GI.GUVC — STA/LTA



```
# Find GUVG coordinates from the station inventory
```

```
for network in stations:
    for station in network:
        if network.code == "GI" and station.code == "GUVG":
            print("Network :", network.code)
            print("Station :", station.code)
            print("Latitude:", station.latitude)
            print("Longitude:", station.longitude)
            print("Elevation:", station.elevation)
```

```
Network : GI
Station : GUVG
Latitude: 14.532
Longitude: -90.498
Elevation: 1800.0
```

```

from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

guvc_lat = 14.532
guvc_lon = -90.498

distance_guvc = locations2degrees(
    14.57, -92.96,
    guvc_lat, guvc_lon
)

model = TauPyModel(model="iasp91")

arrivals_guvc = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_guvc,
    phase_list=["P", "S"]
)

print("Distance:", distance_guvc, "degrees")

for arrival in arrivals_guvc:
    print(
        arrival.name,
        ":",
        round(arrival.time, 2),
        "s"
    )

```

```

Distance: 2.3833211768029425 degrees
P : 37.72 s
P : 42.2 s
P : 42.43 s
S : 66.98 s
S : 73.1 s
S : 73.5 s

```

```

tr_guvc = processed_waveforms["GI.GUVC"][0]

time_guvc = tr_guvc.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time_guvc, tr_guvc.data)

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

```

```
)

# Theoretical S arrivals
plt.axvline(
    66.98,
    linestyle=":",
    label="IASP91 S = 66.98 s"
)

plt.axvline(
    73.1,
    linestyle=":",
    label="IASP91 S = 73.10 s"
)

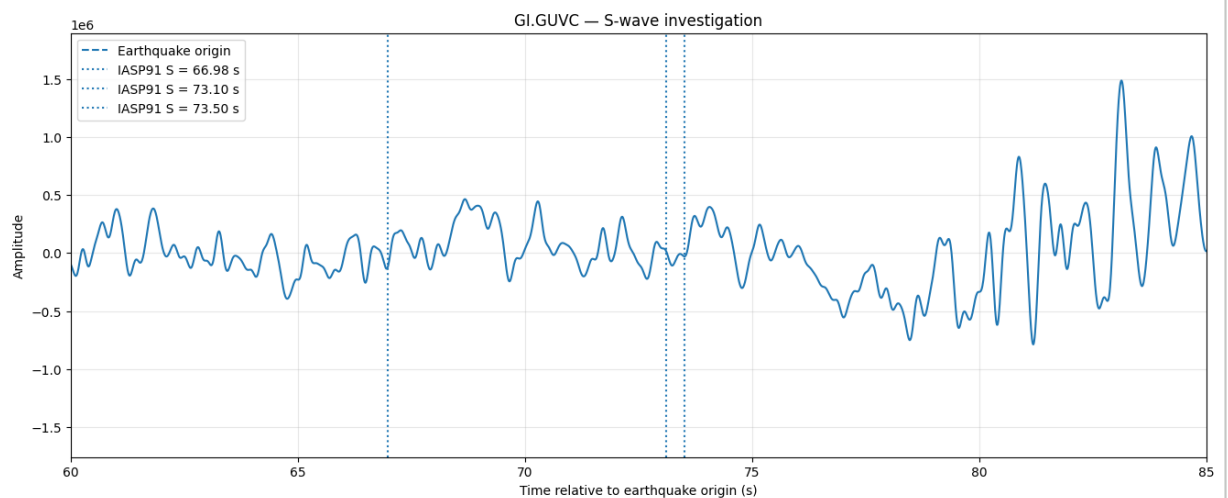
plt.axvline(
    73.5,
    linestyle=":",
    label="IASP91 S = 73.50 s"
)

plt.xlim(60, 85)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUVC – S-wave investigation")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
tr_qcix = processed_waveforms["GI.QCIX"][0]

print(tr_qcix)
print("Sampling rate:", tr_qcix.stats.sampling_rate)
print("Number of samples:", tr_qcix.stats.npts)
print("Min amplitude:", tr_qcix.data.min())
print("Max amplitude:", tr_qcix.data.max())
```

```
GI.QCIX..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z
Sampling rate: 100.0
Number of samples: 66000
Min amplitude: -5.209658810937244e-10
Max amplitude: 5.209658810944075e-10
```

```
tr_qcix_raw = all_waveforms["GI.QCIX"][0]

print(tr_qcix_raw)
print("Raw min:", tr_qcix_raw.data.min())
print("Raw max:", tr_qcix_raw.data.max())
print("Raw mean:", tr_qcix_raw.data.mean())
```

```
GI.QCIX..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880000Z
Raw min: 8388607
Raw max: 8388607
Raw mean: 8388607.0
```

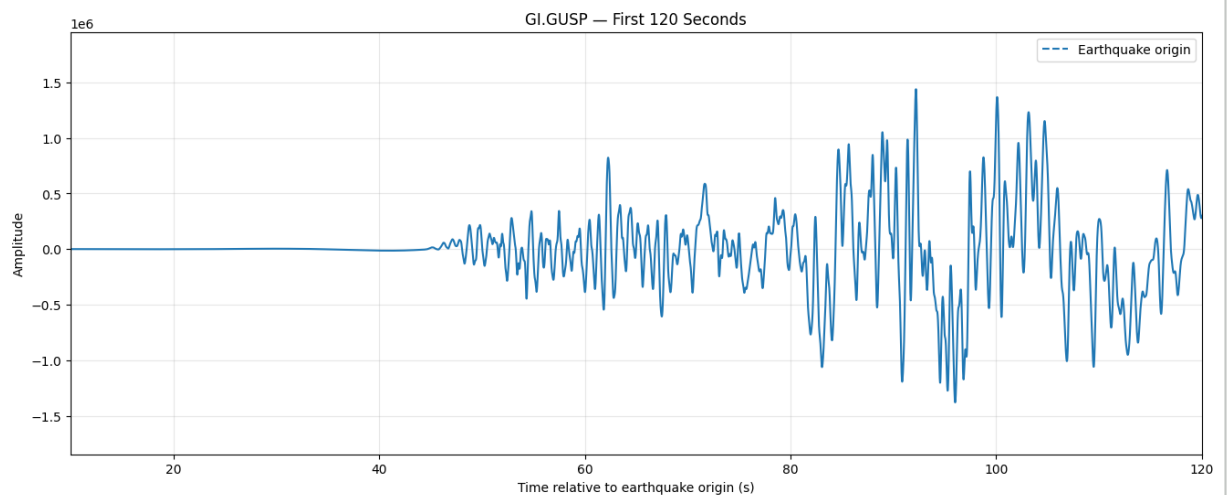
```
tr_gusp = processed_waveforms["GI.GUSP"][0]

time_gusp = tr_gusp.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time_gusp, tr_gusp.data)

plt.axvline(
```

```
0,  
linestyle="--",  
label="Earthquake origin"  
)  
  
plt.xlim(10, 120)  
  
plt.xlabel("Time relative to earthquake origin (s)")  
plt.ylabel("Amplitude")  
plt.title("GI.GUSP - First 120 Seconds")  
  
plt.grid(True, alpha=0.3)  
plt.legend()  
  
plt.show()
```



```
tr_gusp = processed_waveforms["GI.GUSP"][0]  
  
time_gusp = tr_gusp.times(reftime=event_time)
```

```
plt.figure(figsize=(16, 6))
plt.plot(time_gusp, tr_gusp.data)

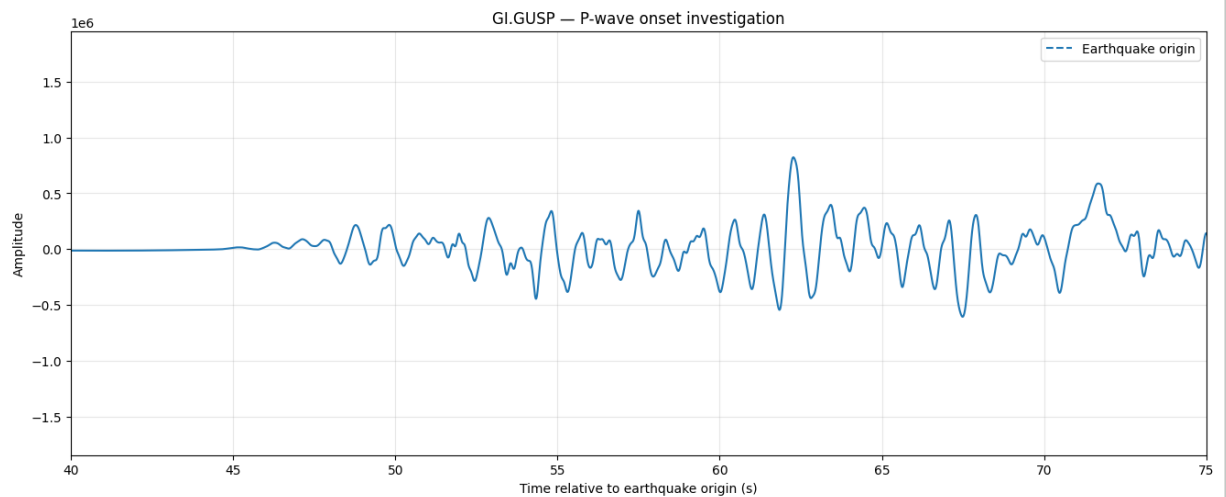
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlim(40, 75)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUSP - P-wave onset investigation")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
from obspy.signal.trigger import classic_sta_lta

tr_gusp_pick = processed_waveforms["GI.GUSP"][0].copy()

tr_gusp_pick.trim(
    starttime=event_time - 10,
    endtime=event_time + 150
)

df = tr_gusp_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_gusp = classic_sta_lta(
    tr_gusp_pick.data,
    sta_samples,
    lta_samples
)

time_gusp_sta = np.arange(len(cft_gusp)) / df - 10

plt.figure(figsize=(14, 5))

plt.plot(time_gusp_sta, cft_gusp)

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

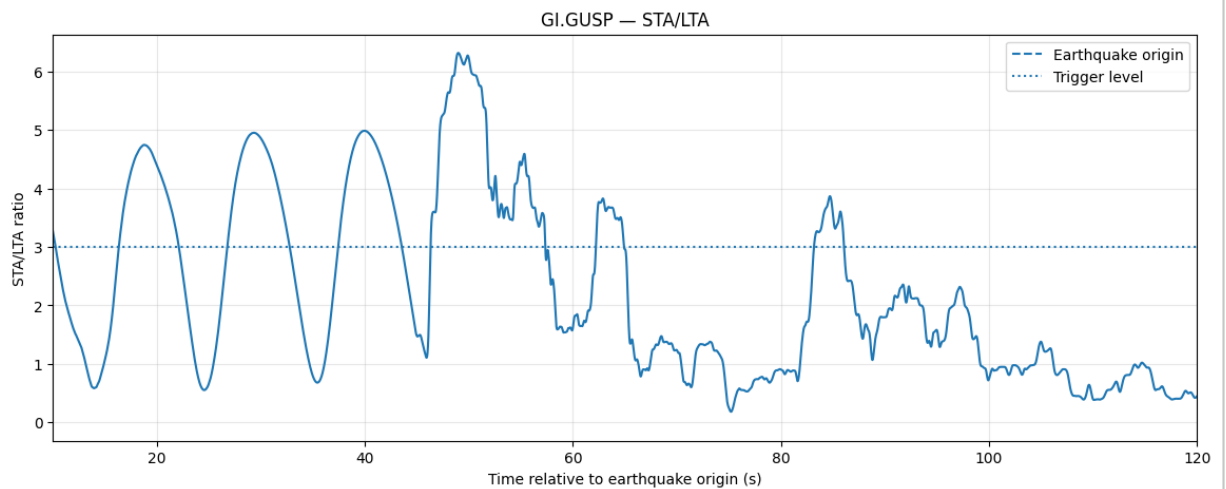
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

plt.xlim(10, 120)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.GUSP - STA/LTA")
```

```
plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
# Find GUSP coordinates from the station inventory

for network in stations:
    for station in network:
        if network.code == "GI" and station.code == "GUSP":
            print("Network :", network.code)
            print("Station :", station.code)
            print("Latitude:", station.latitude)
            print("Longitude:", station.longitude)
            print("Elevation:", station.elevation)
```

```
Network : GI
Station : GUSP
Latitude: 14.552
Longitude: -90.3168
Elevation: 2450.0
```

```

from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

gusp_lat = 14.552
gusp_lon = -90.3168

# Source-station angular distance
distance_gusp = locations2degrees(
    14.57, -92.96,
    gusp_lat, gusp_lon
)

# IASP91 Earth model
model = TauPyModel(model="iasp91")

arrivals_gusp = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_gusp,
    phase_list=["P", "S"]
)

print("Distance:", distance_gusp, "degrees")

for arrival in arrivals_gusp:
    print(
        arrival.name,
        ":",
        round(arrival.time, 2),
        "s"
    )

```

```

Distance: 2.558350770467267 degrees
P : 40.13 s
P : 45.19 s
P : 45.39 s
S : 71.31 s
S : 78.27 s
S : 78.63 s

```

```

tr_gusp = processed_waveforms["GI.GUSP"][0]

time_gusp = tr_gusp.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time_gusp, tr_gusp.data)

plt.axvline(
    71.31,

```

```
        linestyle=":",
        label="IASP91 S = 71.31 s"
    )

    plt.axvline(
        78.27,
        linestyle=":",
        label="IASP91 S = 78.27 s"
    )

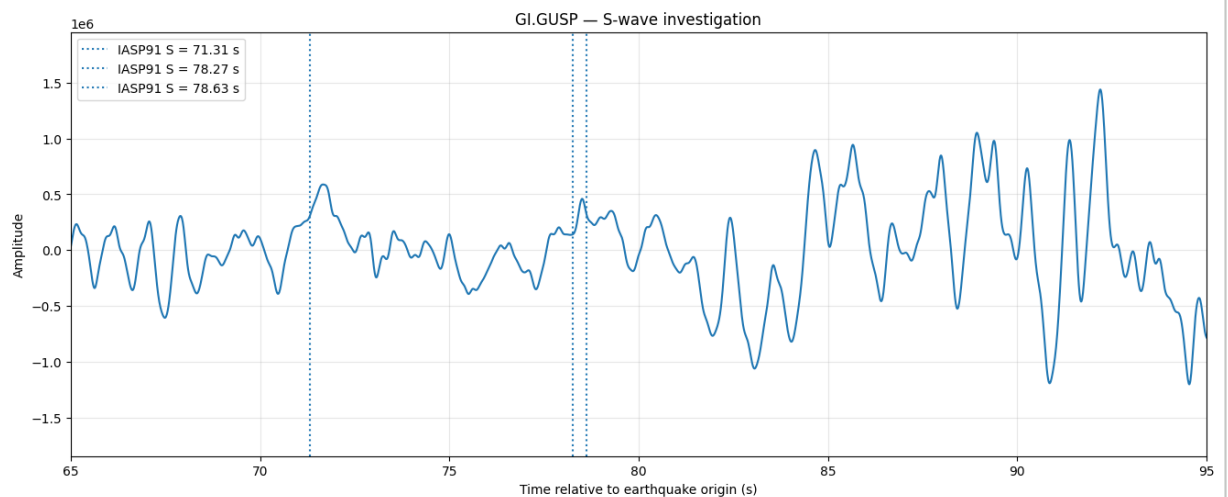
    plt.axvline(
        78.63,
        linestyle=":",
        label="IASP91 S = 78.63 s"
    )

    plt.xlim(65, 95)

    plt.xlabel("Time relative to earthquake origin (s)")
    plt.ylabel("Amplitude")
    plt.title("GI.GUSP - S-wave investigation")

    plt.grid(True, alpha=0.3)
    plt.legend()

    plt.show()
```



```
tr_gusp_s = processed_waveforms["GI.GUSP"][0].copy()

tr_gusp_s.trim(
    starttime=event_time - 10,
    endtime=event_time + 120
)

df = tr_gusp_s.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_gusp_s = classic_sta_lta(
    tr_gusp_s.data,
    sta_samples,
    lta_samples
)

time_gusp_s = np.arange(len(cft_gusp_s)) / df - 10

plt.figure(figsize=(14, 5))

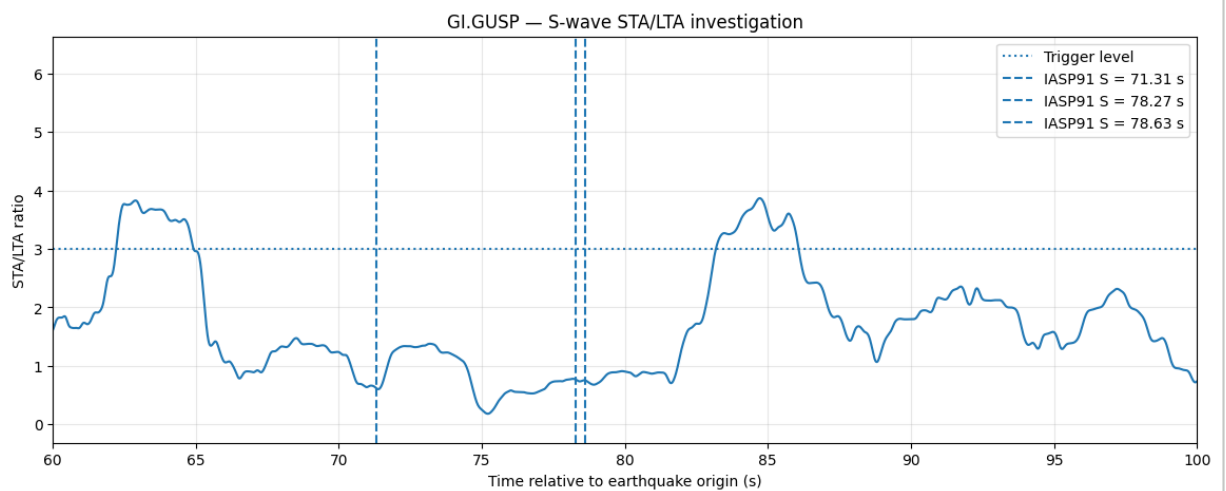
plt.plot(time_gusp_s, cft_gusp_s)

plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

plt.axvline(
    71.31,
    linestyle="--",
    label="IASP91 S = 71.31 s"
)

plt.axvline(
```

```
    /8.2/,  
    linestyle="--",  
    label="IASP91 S = 78.27 s"  
)  
  
plt.axvline(  
    78.63,  
    linestyle="--",  
    label="IASP91 S = 78.63 s"  
)  
  
plt.xlim(60, 100)  
  
plt.xlabel("Time relative to earthquake origin (s)")  
plt.ylabel("STA/LTA ratio")  
plt.title("GI.GUSP - S-wave STA/LTA investigation")  
  
plt.grid(True, alpha=0.3)  
plt.legend()  
  
plt.show()
```



```
tr_sror = processed_waveforms["GI.SROR"][0]

time_sror = tr_sror.times(reftime=event_time)

plt.figure(figsize=(16, 6))
plt.plot(time_sror, tr_sror.data)

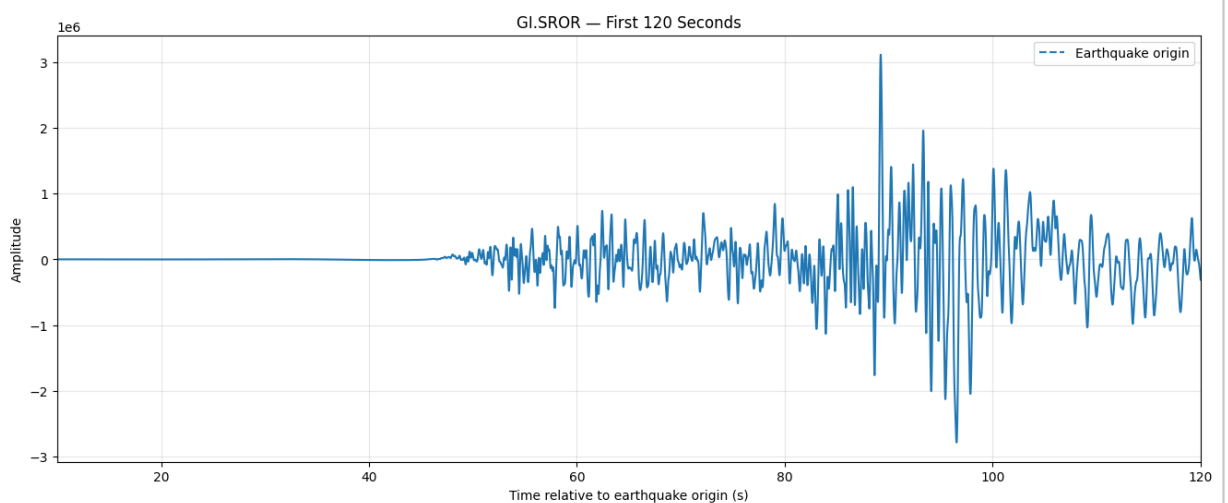
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlim(10, 120)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.SROR – First 120 Seconds")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
from obspy.signal.trigger import classic_sta_lta

tr_sror_pick = processed_waveforms["GI.SR0R"][0].copy()

tr_sror_pick.trim(
    starttime=event_time - 10,
    endtime=event_time + 150
)

df = tr_sror_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_sror = classic_sta_lta(
    tr_sror_pick.data,
    sta_samples,
    lta_samples
)

time_sror_sta = np.arange(len(cft_sror)) / df - 10

plt.figure(figsize=(14, 5))

plt.plot(time_sror_sta, cft_sror)

plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

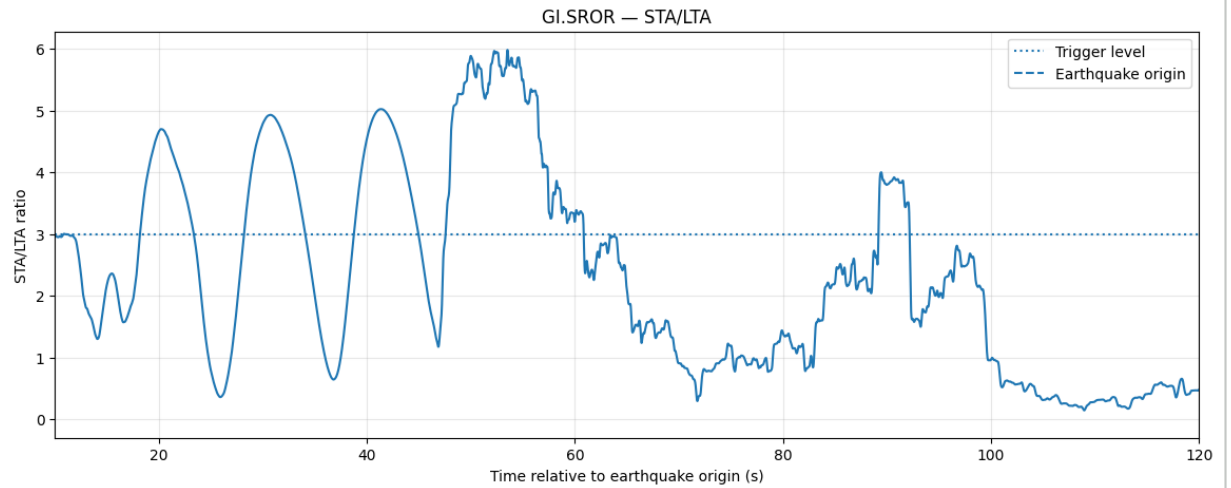
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlim(10, 120)
```

```
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.SROR - STA/LTA")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
for network in stations:
    for station in network:
        if network.code == "GI" and station.code == "SR0R":
            print("Network :", network.code)
            print("Station :", station.code)
            print("Latitude:", station.latitude)
            print("Longitude:", station.longitude)
            print("Elevation:", station.elevation)
```

```
Network : GI
Station : SR0R
Latitude: 14.231059
Longitude: -90.198398
Elevation: 1180.0
```

```

from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

srór_lat = 14.231059
srór_lon = -90.198398

# Source-station angular distance
distance_srór = locations2degrees(
    14.57, -92.96,
    srór_lat, srór_lon
)

# IASP91 Earth model
model = TauPyModel(model="iasp91")

arrivals_srór = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_srór,
    phase_list=["P", "S"]
)

print("Distance:", distance_srór, "degrees")

for arrival in arrivals_srór:
    print(
        arrival.name,
        ":",
        round(arrival.time, 2),
        "s"
    )

```

```

Distance: 2.696203216057268 degrees
P : 42.02 s
P : 47.54 s
P : 47.73 s
S : 74.72 s
S : 82.34 s
S : 82.67 s

```

```

tr_srór = processed_waveforms["GI.SROR"][0]

time_srór = tr_srór.times(reftime=event_time)

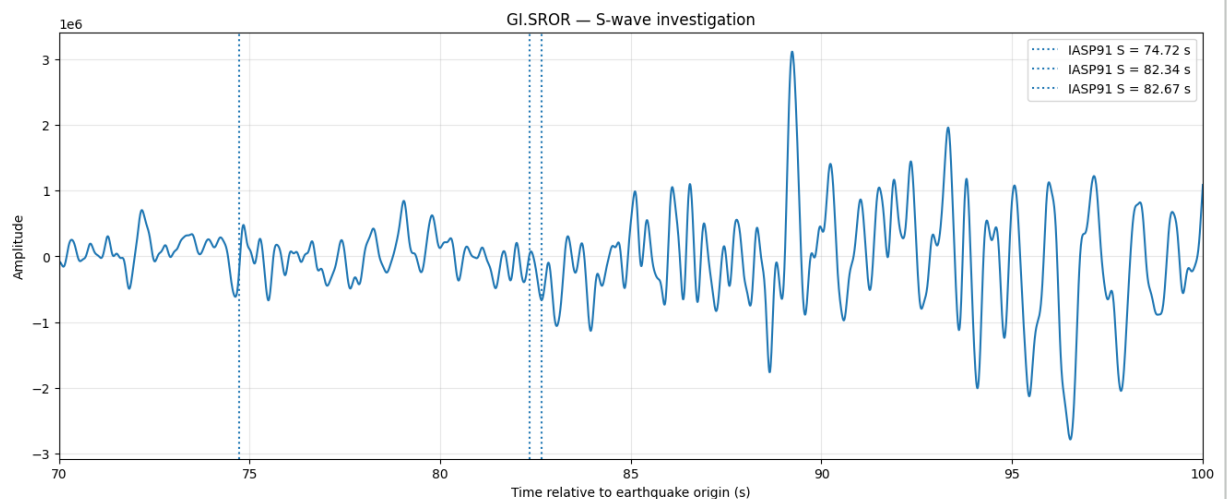
plt.figure(figsize=(16, 6))

plt.plot(time_srór, tr_srór.data)

plt.axvline(

```

```
74.72,  
linestyle=":",  
label="IASP91 S = 74.72 s"  
)  
  
plt.axvline(  
    82.34,  
    linestyle=":",  
    label="IASP91 S = 82.34 s"  
)  
  
plt.axvline(  
    82.67,  
    linestyle=":",  
    label="IASP91 S = 82.67 s"  
)  
  
plt.xlim(70, 100)  
  
plt.xlabel("Time relative to earthquake origin (s)")  
plt.ylabel("Amplitude")  
plt.title("GI.SROR - S-wave investigation")  
  
plt.grid(True, alpha=0.3)  
plt.legend()  
  
plt.show()
```



```
tr_avcs = processed_waveforms["GI.AVCS"][0]

time_avcs = tr_avcs.times(reftime=event_time)

plt.figure(figsize=(16, 6))

plt.plot(time_avcs, tr_avcs.data)

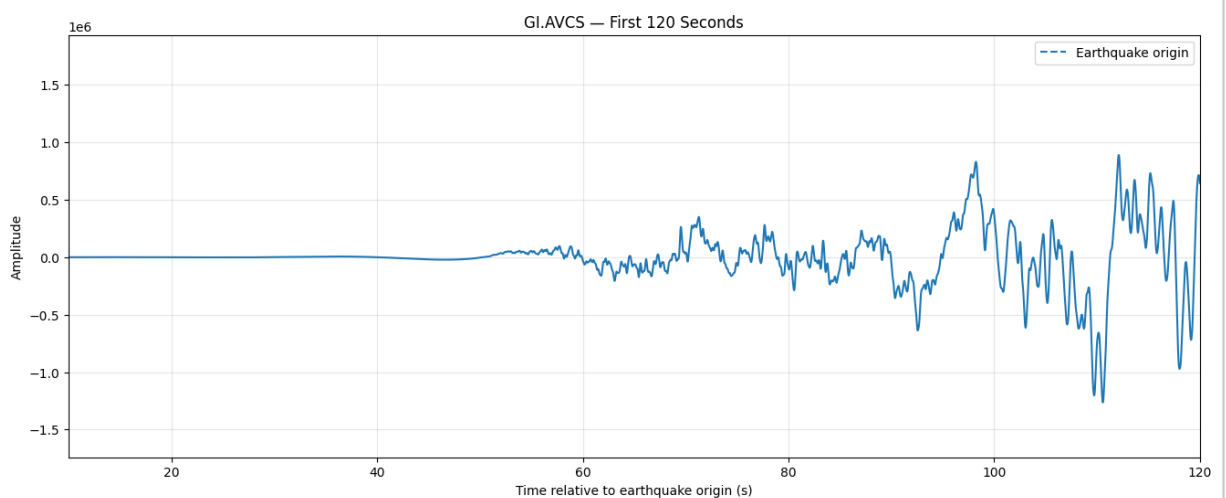
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlim(10, 120)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.AVCS - First 120 Seconds")

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()
```



```
from obspy.signal.trigger import classic_sta_lta

tr_avcs_pick = processed_waveforms["GI.AVCS"][0].copy()

tr_avcs_pick.trim(
    starttime=event_time - 10,
    endtime=event_time + 150
)

df = tr_avcs_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_avcs = classic_sta_lta(
    tr_avcs_pick.data,
    sta_samples,
    lta_samples
)

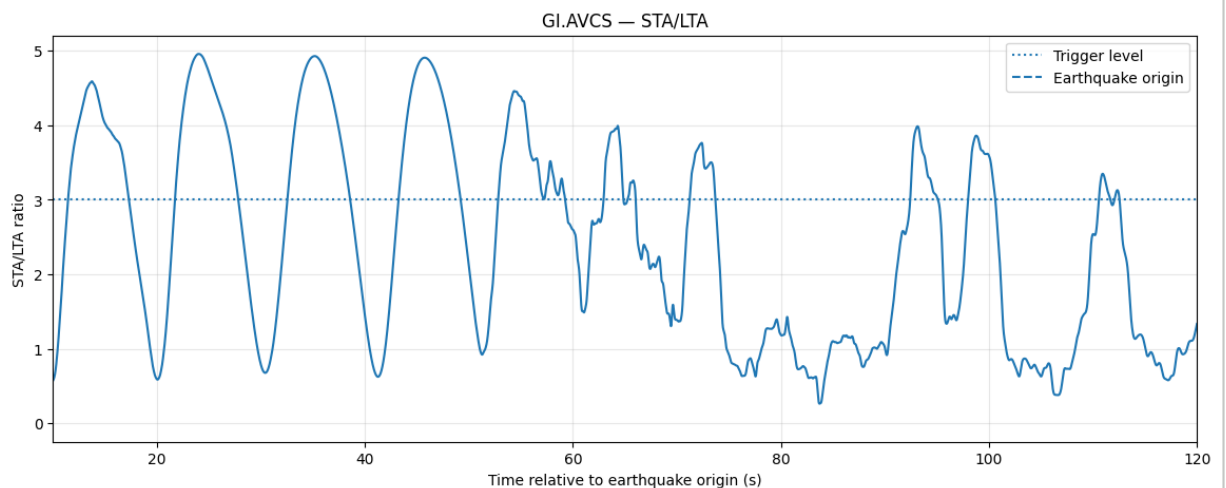
time_avcs_sta = np.arange(len(cft_avcs)) / df - 10

plt.figure(figsize=(14, 5))

plt.plot(time_avcs_sta, cft_avcs)

plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
```

```
)  
  
plt.axvline(  
    0,  
    linestyle="--",  
    label="Earthquake origin"  
)  
  
plt.xlim(10, 120)  
  
plt.xlabel("Time relative to earthquake origin (s)")  
plt.ylabel("STA/LTA ratio")  
plt.title("GI.AVCS - STA/LTA")  
  
plt.grid(True, alpha=0.3)  
plt.legend()  
  
plt.show()
```



```
for network in stations:
    for station in network:
        if network.code == "GI" and station.code == "AVCS":
            print("Network :", network.code)
            print("Station :", station.code)
            print("Latitude:", station.latitude)
            print("Longitude:", station.longitude)
            print("Elevation:", station.elevation)
```

```
Network : GI
Station : AVCS
Latitude: 15.815492
Longitude: -90.276573
Elevation: 157.0
```

```

from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

avcs_lat = 15.815492
avcs_lon = -90.276573

# Source-station angular distance
distance_avcs = locations2degrees(
    14.57, -92.96,
    avcs_lat, avcs_lon
)

# IASP91 Earth model
model = TauPyModel(model="iasp91")

arrivals_avcs = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_avcs,
    phase_list=["P", "S"]
)

print("Distance:", distance_avcs, "degrees")

for arrival in arrivals_avcs:
    print(
        arrival.name,
        ":",
        round(arrival.time, 2),
        "s"
    )

```

```

Distance: 2.8735136766104596 degrees
P : 44.46 s
P : 50.56 s
P : 50.73 s
S : 79.1 s
S : 87.58 s
S : 87.88 s

```

```

from obspy import UTCDateTime
import matplotlib.pyplot as plt
import numpy as np

tr = all_waveforms["GI.AVCS"][0].copy()

# Time relative to earthquake origin
time = np.arange(len(tr.data)) / tr.stats.sampling_rate
time = time + (tr.stats.starttime - event_time)

```

```

plt.figure(figsize=(16, 6))

plt.plot(time, tr.data)

# Earthquake origin
plt.axvline(
    0,
    color="blue",
    linestyle="--",
    label="Earthquake origin"
)

# IASP91 P-wave arrivals
p_times_avcs = [44.46, 50.56, 50.73]

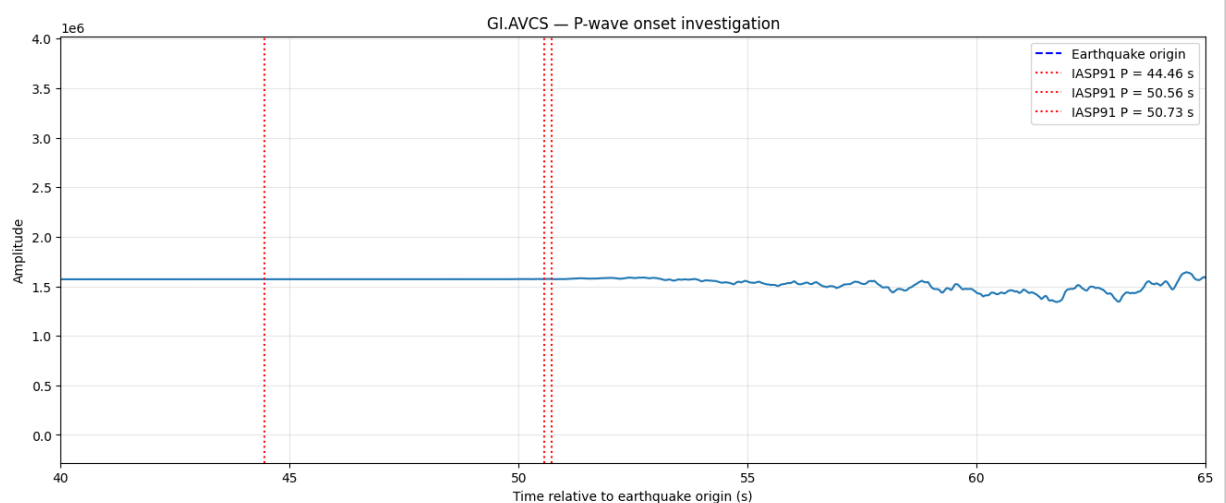
for p in p_times_avcs:
    plt.axvline(
        p,
        color="red",
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

plt.xlim(40, 65)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.AVCS - P-wave onset investigation")

plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```



```
from obspy.signal.trigger import classic_sta_lta

# AVCS trace
tr_avcs = all_waveforms["GI.AVCS"][0].copy()

# Trim around the expected P-wave arrivals
tr_avcs.trim(
    starttime=event_time + 35,
    endtime=event_time + 65
)

# Sampling rate
df = tr_avcs.stats.sampling_rate

# STA/LTA window lengths
sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

# Calculate STA/LTA characteristic function
cft_avcs = classic_sta_lta(
    tr_avcs.data,
    sta_samples,
    lta_samples
)

# Time relative to earthquake origin
time_avcs = (
    tr_avcs.times()
    + (tr_avcs.stats.starttime - event_time)
)
```

```

# Plot
plt.figure(figsize=(14, 5))

plt.plot(
    time_avcs,
    cft_avcs,
    label="STA/LTA"
)

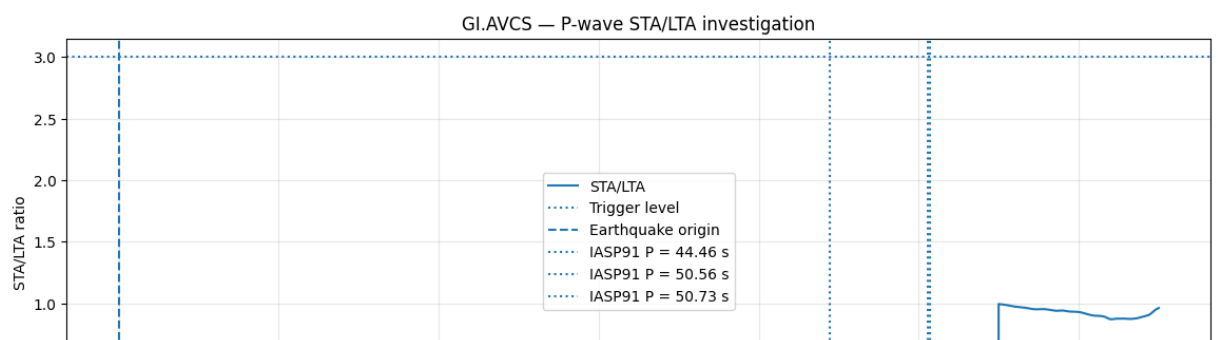
# Trigger level
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

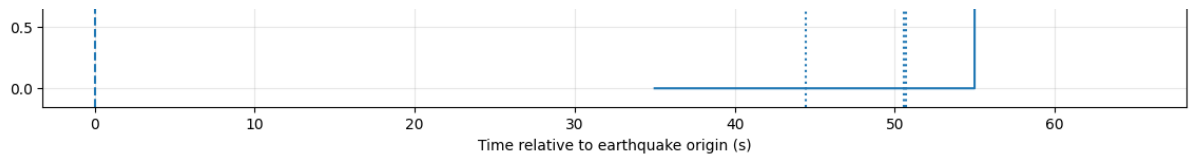
# Earthquake origin
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

# Theoretical P arrivals
for arrival in arrivals_avcs:
    if arrival.name == "P":
        plt.axvline(
            arrival.time,
            linestyle=":",
            label=f"IASP91 P = {arrival.time:.2f} s"
        )

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.AVCS - P-wave STA/LTA investigation")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```





```

from obspy.signal.trigger import classic_sta_lta

# AVCS trace
tr_avcs = all_waveforms["GI.AVCS"][0].copy()

# Start well before the expected P arrivals
tr_avcs.trim(
    starttime=event_time + 20,
    endtime=event_time + 65
)

df = tr_avcs.stats.sampling_rate

# STA and LTA windows
sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

# STA/LTA
cft_avcs = classic_sta_lta(
    tr_avcs.data,
    sta_samples,
    lta_samples
)

# Time relative to earthquake origin
time_avcs = (
    tr_avcs.times()

```

```

    + (tr_avcs.stats.starttime - event_time)
)

# Plot
plt.figure(figsize=(14, 5))

plt.plot(
    time_avcs,
    cft_avcs,
    label="STA/LTA"
)

# Trigger level
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

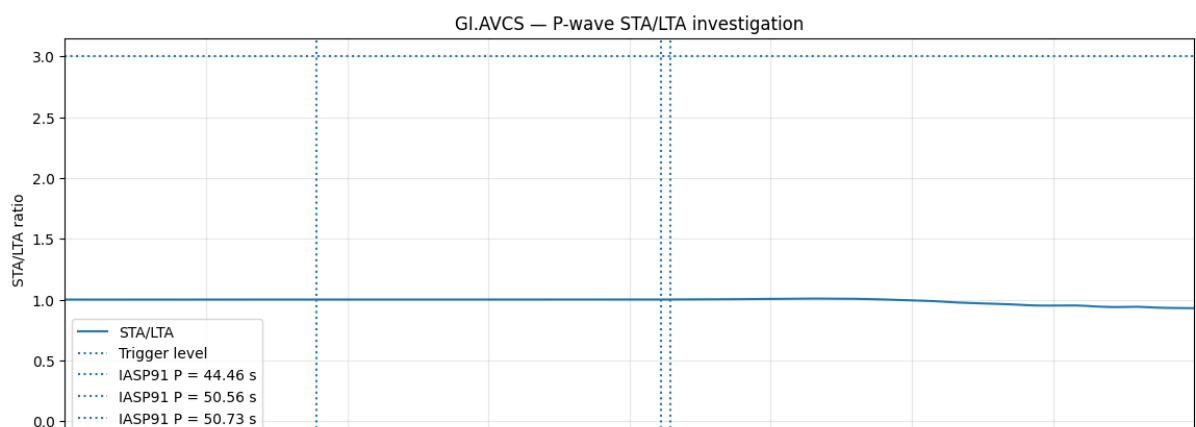
# Theoretical P arrivals
for arrival in arrivals_avcs:
    if arrival.name == "P":
        plt.axvline(
            arrival.time,
            linestyle=":",
            label=f"IASP91 P = {arrival.time:.2f} s"
        )

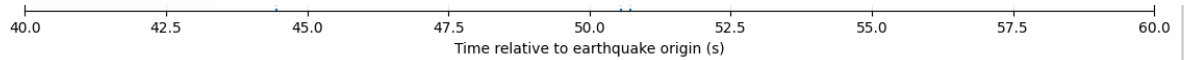
plt.xlim(40, 60)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.AVCS - P-wave STA/LTA investigation")
plt.legend()
plt.grid(alpha=0.3)

plt.show()

```





```
# AVCS trace
tr_avcs = all_waveforms["GI.AVCS"][0].copy()

# Focus around theoretical S arrivals
tr_avcs.trim(
    starttime=event_time + 70,
    endtime=event_time + 100
)

# Time relative to earthquake origin
time_avcs = (
    tr_avcs.times()
    + (tr_avcs.stats.starttime - event_time)
)

plt.figure(figsize=(14, 5))

plt.plot(
    time_avcs,
    tr_avcs.data,
    label="GI.AVCS waveform"
)

# Theoretical S arrivals
for arrival in arrivals_avcs:
    if arrival.name == "S":
        plt.axvline(
            arrival.time,
            linestyle=":",
            label=f"IASP91 S = {arrival.time:.2f} s"
        )

plt.savefig('')
```

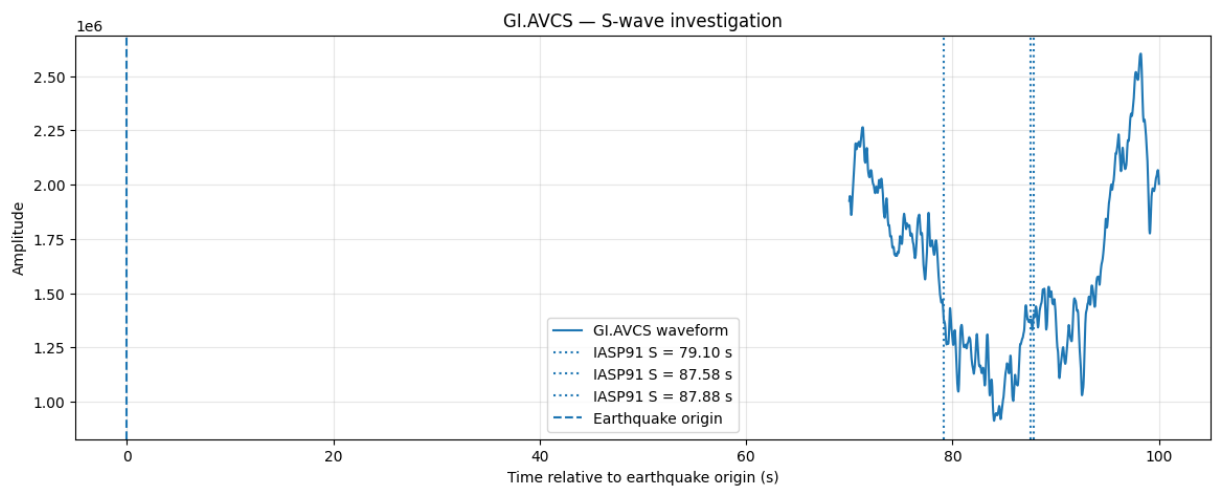
```

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.AVCS - S-wave investigation")

plt.legend()
plt.grid(alpha=0.3)
plt.show()

```



```

from obspy.signal.trigger import classic_sta_lta

# AVCS trace
tr_avcs = all_waveforms["GI.AVCS"][0].copy()

# Start early enough for the 20 s LTA window
tr_avcs.trim(
    starttime=event_time + 55,
    ...
)

```

```

        endtime=event_time + 100
    )

df = tr_avcs.stats.sampling_rate

# STA/LTA windows
sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

# Calculate STA/LTA
cft_avcs = classic_sta_lta(
    tr_avcs.data,
    sta_samples,
    lta_samples
)

# Time relative to earthquake origin
time_avcs = (
    tr_avcs.times()
    + (tr_avcs.stats.starttime - event_time)
)

# Plot
plt.figure(figsize=(14, 5))

plt.plot(
    time_avcs,
    cft_avcs,
    label="STA/LTA"
)

# Trigger level
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

# Theoretical S arrivals
for arrival in arrivals_avcs:
    if arrival.name == "S":
        plt.axvline(
            arrival.time,
            linestyle="--",
            label=f"IASP91 S = {arrival.time:.2f} s"
        )

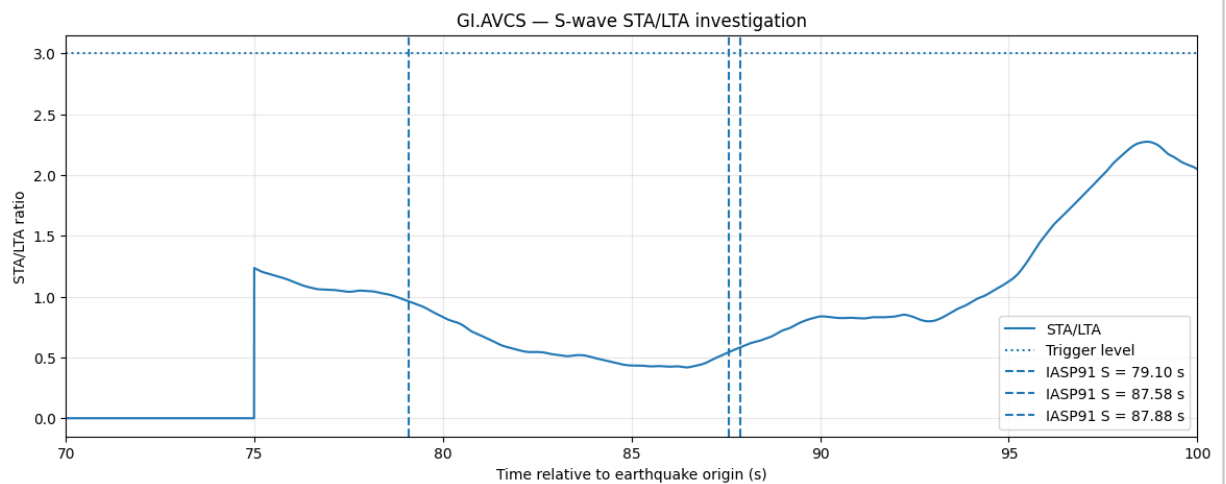
```

```
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.AVCS - S-wave STA/LTA investigation")

plt.xlim(70, 100)

plt.legend()
plt.grid(alpha=0.3)

plt.show()
```



```
st_nube = earthscope.get_waveforms(  
    network="SV",  
    station="NUBE",  
    location="--",  
    channel="HHZ",  
    starttime=event_time - 60,  
    endtime=event_time + 600  
)  
  
print(st_nube)
```

```
1 Trace(s) in Stream:  
SV.NUBE..HHZ | 2026-07-17T14:47:39.890000Z - 2026-07-17T14:58:39.880
```

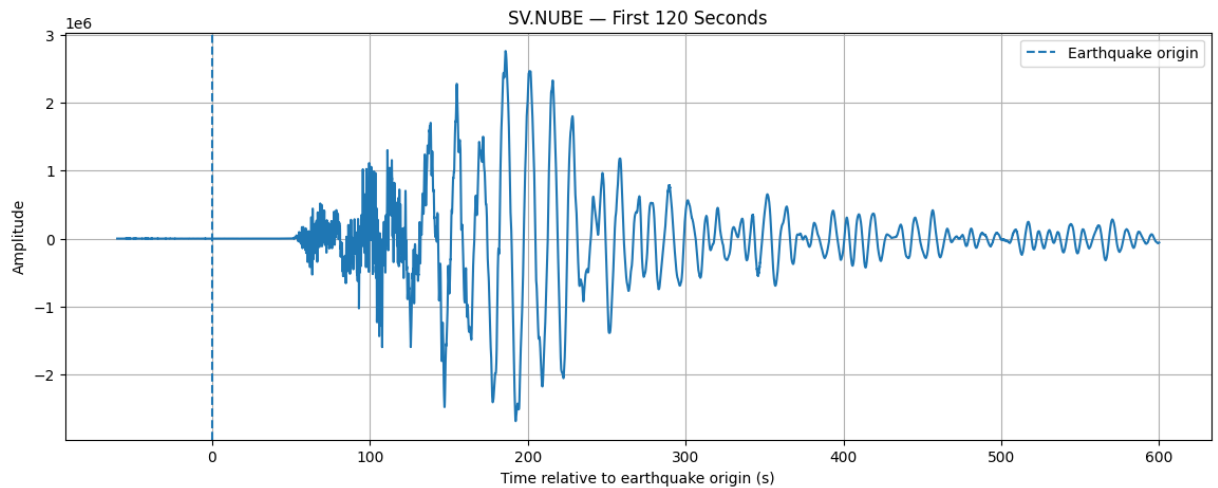
```
tr = st_nube[0]

time = tr.times()
time_relative = time - 60

plt.figure(figsize=(14, 5))
plt.plot(time_relative, tr.data)

plt.axvline(0, linestyle="--", label="Earthquake origin")

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("SV.NUBE - First 120 Seconds")
plt.legend()
plt.grid(True)
plt.show()
```



```
nube_inventory = earthscope.get_stations(
    network="SV",
    station="NUBE",
    level="station"
)

for net in nube_inventory:
    for sta in net:
        print(
            "Network:", net.code,
            "Station:", sta.code,
            "Latitude:", sta.latitude,
            "Longitude:", sta.longitude,
            "Elevation:", sta.elevation
        )
```

Network: SV Station: NUBE Latitude: 13.90216 Longitude: -89.779907 E

```

from obspy.geodetics import locations2degrees
from obspy.taup import TauPyModel

# NUBE coordinates
nube_lat = 13.90216
nube_lon = -89.779907

# Source-station angular distance
distance_nube = locations2degrees(
    14.57, -92.96,          # earthquake
    nube_lat, nube_lon      # NUBE
)

# IASP91 Earth model
model = TauPyModel(model="iasp91")

arrivals_nube = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_nube,
    phase_list=["P", "S"]
)

print("Distance:", distance_nube, "degrees")

for arrival in arrivals_nube:
    print(arrival.name, ":", round(arrival.time, 2), "s")

```

```

Distance: 3.1539081983444124 degrees
P : 48.31 s
P : 55.34 s
P : 55.48 s
S : 86.03 s
S : 95.87 s
S : 96.12 s

```

```

tr = st_nube[0]

# Time relative to earthquake origin
time = tr.times() - 60

# Plot P-wave window
plt.figure(figsize=(14, 5))

plt.plot(time, tr.data, label="SV.NUBE waveform")

# Earthquake origin
plt.axvline(
    0,

```

```

        linestyle="--",
        label="Earthquake origin"
    )

# IASP91 theoretical P arrivals
p_times_nube = [48.31, 55.34, 55.48]

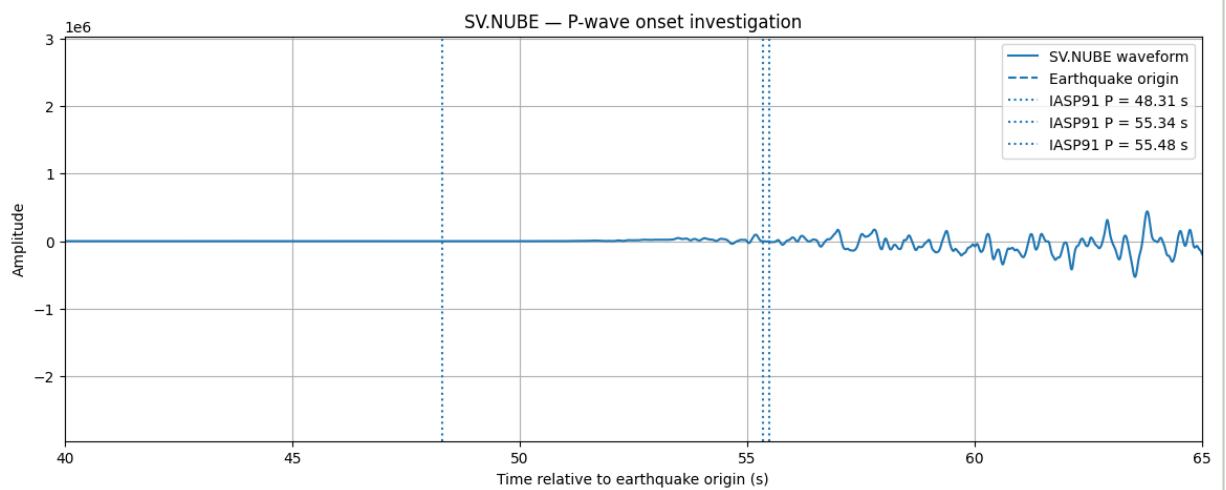
for i, p_time in enumerate(p_times_nube):
    plt.axvline(
        p_time,
        linestyle=":",
        label=f"IASP91 P = {p_time:.2f} s"
    )

plt.xlim(40, 65)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("SV.NUBE - P-wave onset investigation")
plt.legend()
plt.grid(True)

plt.show()

```



```
from obspy.signal.trigger import classic_sta_lta

tr_pick = st_nube[0].copy()

# Focus on the P-wave region
tr_pick.trim(
    starttime=event_time + 35,
    endtime=event_time + 100
)

df = tr_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft = classic_sta_lta(
    tr_pick.data,
    sta_samples,
    lta_samples
)

time_sta = tr_pick.times() + 35

plt.figure(figsize=(14, 5))

plt.plot(time_sta, cft, label="STA/LTA")

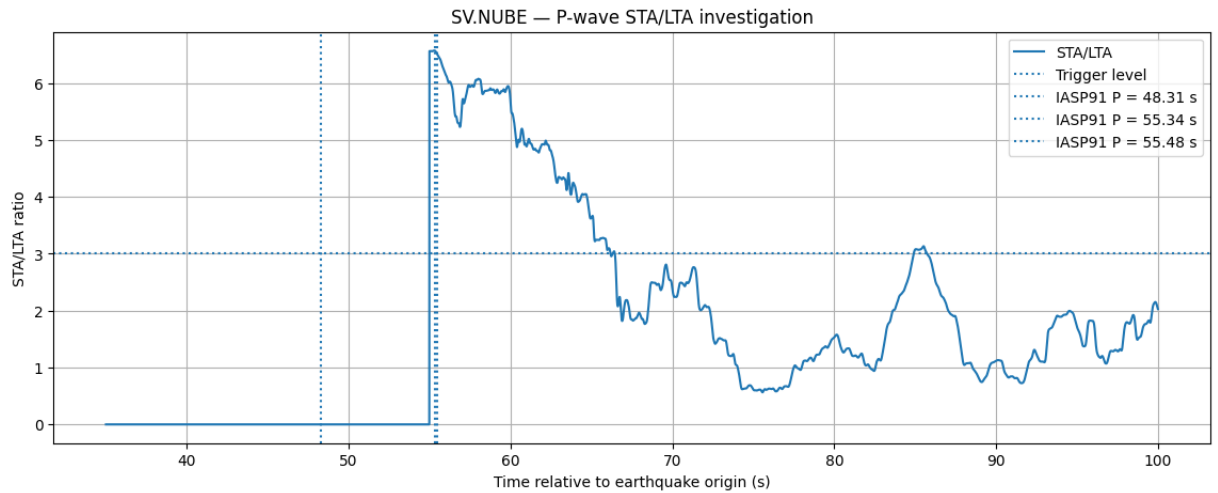
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

for p_time in [48.31, 55.34, 55.48]:
    plt.axvline(
        p_time,
        linestyle=":",
        label=f"IASP91 P = {p_time:.2f} s"
    )

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("SV.NUBE – P-wave STA/LTA investigation")
```

```
plt.legend()
plt.grid(True)

plt.show()
```



```
# SV.NUBE – S-wave investigation
```

```
tr_s = st_nube[0].copy()

tr_s.trim(
    starttime=event_time + 80,
    endtime=event_time + 105
)

time_s = tr_s.times() + 80

plt.figure(figsize=(14, 5))

plt.plot(
    time_s,
    tr_s.data,
    label="SV.NUBE waveform"
```

```

)

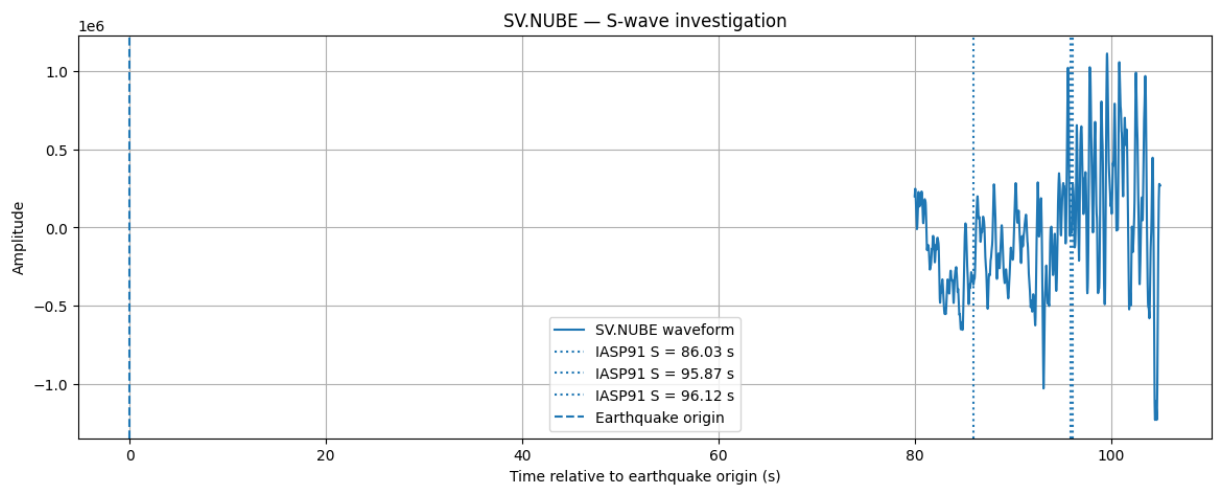
for s_time in [86.03, 95.87, 96.12]:
    plt.axvline(
        s_time,
        linestyle=":",
        label=f"IASP91 S = {s_time:.2f} s"
    )

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("SV.NUBE - S-wave investigation")
plt.legend()
plt.grid(True)

plt.show()

```



```
# SV.NUBE – S-wave STA/LTA investigation

from obspy.signal.trigger import classic_sta_lta

tr_s_pick = st_nube[0].copy()

# Focus around the expected S-wave arrivals
tr_s_pick.trim(
    starttime=event_time + 80,
    endtime=event_time + 105
)

df = tr_s_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_s = classic_sta_lta(
    tr_s_pick.data,
    sta_samples,
    lta_samples
)

time_s_sta = np.arange(len(cft_s)) / df + 80

plt.figure(figsize=(14, 5))

plt.plot(
    time_s_sta,
    cft_s,
    label="STA/LTA"
)

plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

for s_time in [86.03, 95.87, 96.12]:
    plt.axvline(
        s_time,
        linestyle="--",
        label=f"IASP91 S = {s_time:.2f} s"
```

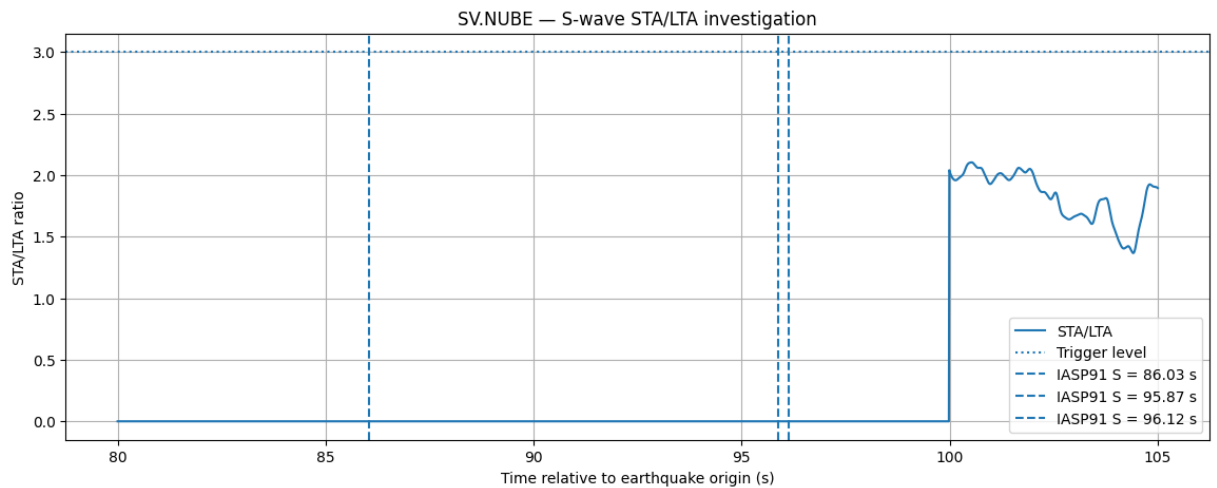
```

)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("SV.NUBE - S-wave STA/LTA investigation")
plt.legend()
plt.grid(True)

plt.show()

```



```

# SV.RBDL - First 120 seconds

st_rbdL = all_waveforms["SV.RBDL"].copy()

tr_rbdL = st_rbdL[0]

start = event_time - 60
time_rbdL = np.arange(tr_rbdL.stats.npts) / tr_rbdL.stats.sampling_r
time_rbdL = time_rbdL - 60

plt.figure(figsize=(14, 5))

```

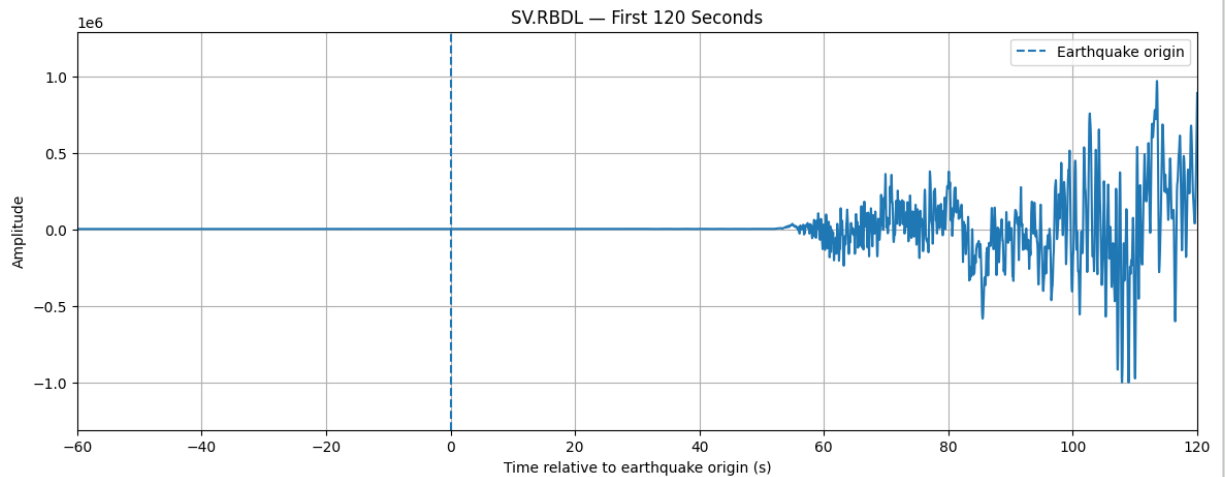
```
plt.plot(time_rbd1, tr_rbd1.data)

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.xlim(-60, 120)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("SV.RBDL - First 120 Seconds")

plt.legend()
plt.grid(True)
plt.show()
```



```
# Get exact SV.RBDL station coordinates
```

```
rbdl_inv = earthscope.get_stations(  
    network="SV",  
    station="RBDL",  
    level="station"  
)
```

```
rbdl_sta = rbdl_inv[0][0]
```

```
rbdl_lat = rbdl_sta.latitude  
rbdl_lon = rbdl_sta.longitude  
rbdl_elev = rbdl_sta.elevation
```

```
print("Network :", rbdl_sta.code)  
print("Station :", rbdl_sta.code)  
print("Latitude:", rbdl_lat)  
print("Longitude:", rbdl_lon)  
print("Elevation:", rbdl_elev)
```

```
Network : RBDL  
Station : RBDL  
Latitude: 14.113  
Longitude: -89.6827  
Elevation: 1431.0
```

```
# Distance and IASP91 theoretical arrivals

distance_deg_rbd1 = locations2degrees(
    14.57, -92.96,      # earthquake
    rbd1_lat, rbd1_lon  # RBDL
)

print("Distance:", distance_deg_rbd1, "degrees")

model = TauPyModel(model="iasp91")

arrivals_rbd1 = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_deg_rbd1,
    phase_list=["P", "S"]
)

for arr in arrivals_rbd1:
    print(arr.name, ":", round(arr.time, 2), "s")
```

```
Distance: 3.207851123216647 degrees
P : 49.06 s
P : 56.26 s
P : 56.4 s
S : 87.37 s
S : 97.46 s
S : 97.7 s
```

```
# SV.RBDL - P-wave onset investigation

tr_rbd1_p = st_rbd1[0].copy()

plt.figure(figsize=(14, 5))

time_rbd1 = (
    np.arange(tr_rbd1_p.stats.npts)
    / tr_rbd1_p.stats.sampling_rate
    - 60
)

plt.plot(time_rbd1, tr_rbd1_p.data, label="SV.RBDL waveform")

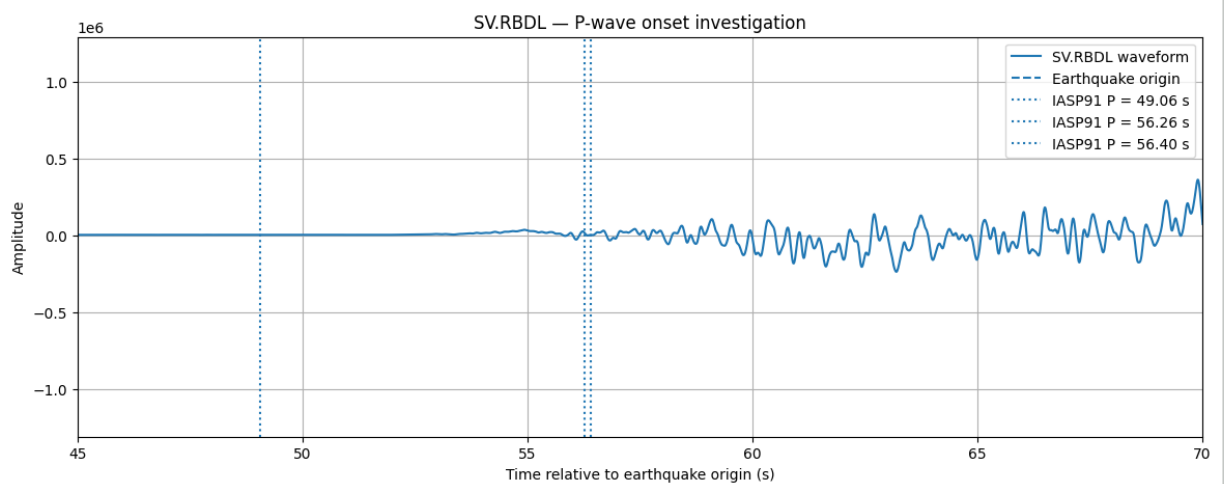
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)
```

```
# IASP91 theoretical P arrivals
for p_time in [49.06, 56.26, 56.40]:
    plt.axvline(
        p_time,
        linestyle=":",
        label=f"IASP91 P = {p_time:.2f} s"
    )

plt.xlim(45, 70)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("SV.RBDL – P-wave onset investigation")

plt.legend()
plt.grid(True)
plt.show()
```



```
# SV.RBDL – P-wave STA/LTA investigation

tr_rbdL_pick = st_rbdL[0].copy()
```

```
tr_rbdL_pick.trim(
    starttime=event_time + 35,
    endtime=event_time + 90
)

df = tr_rbdL_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_rbdL = classic_sta_lta(
    tr_rbdL_pick.data,
    sta_samples,
    lta_samples
)

time_rbdL_sta = np.arange(len(cft_rbdL)) / df + 35

plt.figure(figsize=(14, 5))

plt.plot(
    time_rbdL_sta,
    cft_rbdL,
    label="STA/LTA"
)

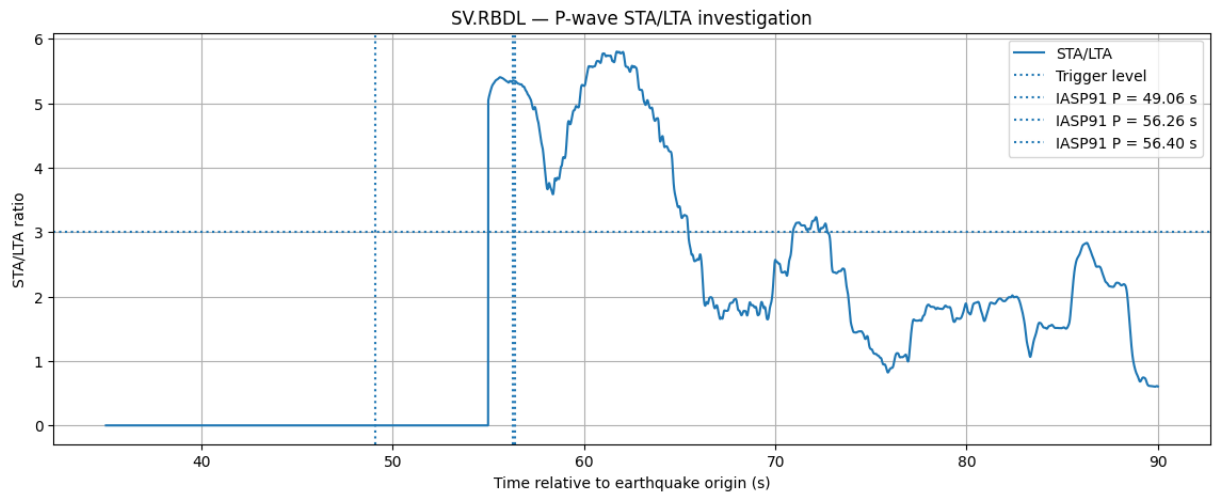
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

for p_time in [49.06, 56.26, 56.40]:
    plt.axvline(
        p_time,
        linestyle=":",
        label=f"IASP91 P = {p_time:.2f} s"
    )

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("SV.RBDL - P-wave STA/LTA investigation")

plt.legend()
plt.grid(True)
```

```
plt.show()
```



```
# SV.RBDL — S-wave investigation

tr_rbdL_s = st_rbdL[0].copy()

plt.figure(figsize=(14, 5))

time_rbdL_s = (
    np.arange(tr_rbdL_s.stats.npts)
    / tr_rbdL_s.stats.sampling_rate
    - 60
)

plt.plot(
    time_rbdL_s,
    tr_rbdL_s.data,
    label="SV.RBDL waveform"
)

plt.axvline(
```

```

    0,
    linestyle="--",
    label="Earthquake origin"
)

# IASP91 theoretical S arrivals
for s_time in [87.37, 97.46, 97.70]:
    plt.axvline(
        s_time,
        linestyle=":",
        label=f"IASP91 S = {s_time:.2f} s"
    )

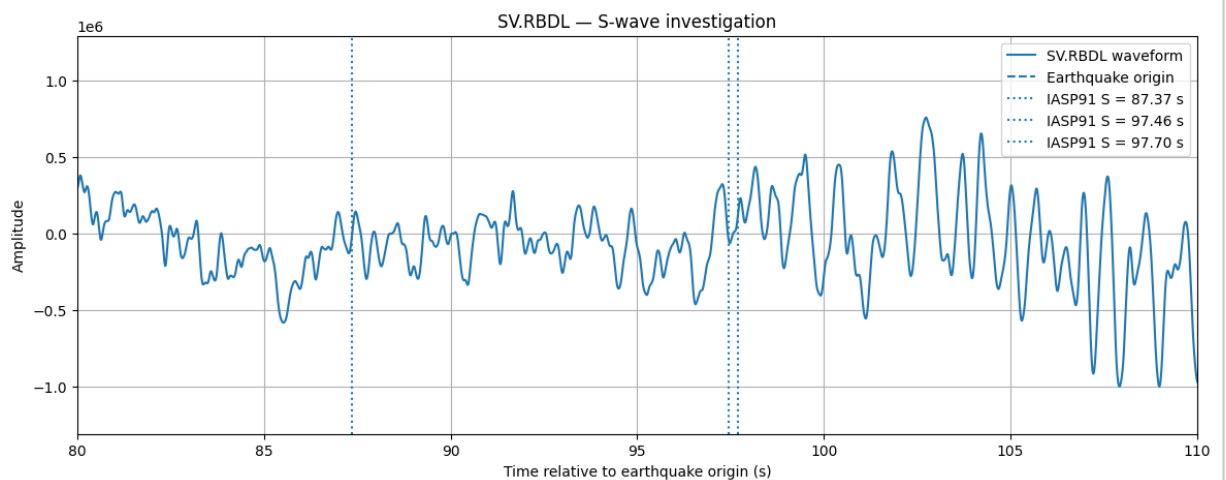
plt.xlim(80, 110)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("SV.RBDL - S-wave investigation")

plt.legend()
plt.grid(True)

plt.show()

```



```
# SV.RBDL - S-wave STA/LTA investigation

tr_rbdL_s_pick = st_rbdL[0].copy()

tr_rbdL_s_pick.trim(
    starttime=event_time + 80,
    endtime=event_time + 110
)

df = tr_rbdL_s_pick.stats.sampling_rate

sta_seconds = 3
lta_seconds = 20

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft_rbdL_s = classic_sta_lta(
    tr_rbdL_s_pick.data,
    sta_samples,
    lta_samples
)

time_rbdL_s_sta = np.arange(len(cft_rbdL_s)) / df + 80

plt.figure(figsize=(14, 5))

plt.plot(
    time_rbdL_s_sta,
    cft_rbdL_s,
    label="STA/LTA"
)

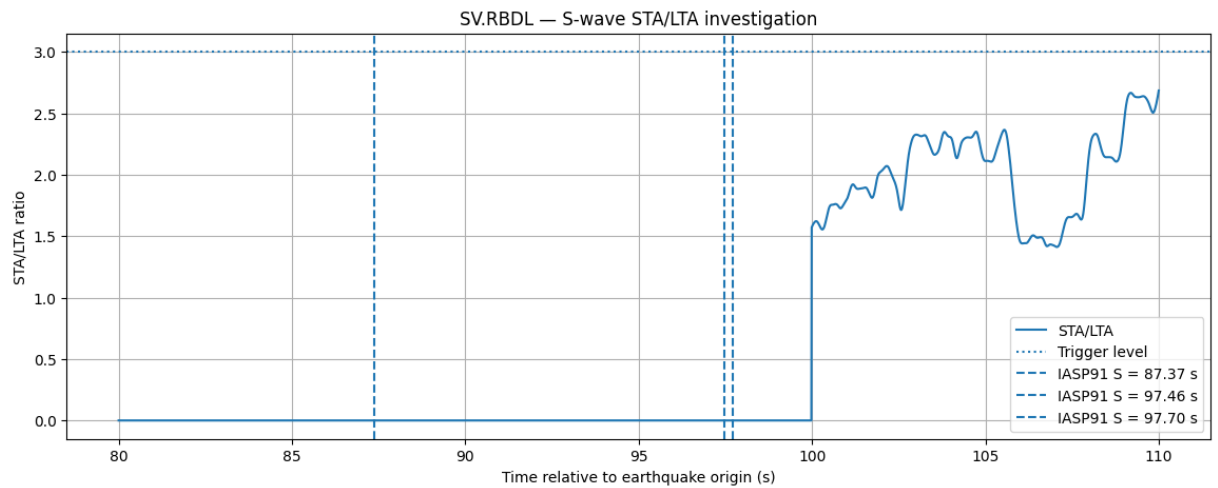
plt.axhline(
    3,
    linestyle=":",
    label="Trigger level"
)

for s_time in [87.37, 97.46, 97.70]:
    plt.axvline(
        s_time,
        linestyle="--",
        label=f"IASP91 S = {s_time:.2f} s"
    )
```

```
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("SV.RBDL - S-wave STA/LTA investigation")

plt.legend()
plt.grid(True)

plt.show()
```



```
# Final station results table

import pandas as pd

results = pd.DataFrame([
    ["GI.HUCU", 1.2881, 24.5, 41.0],
    ["GI.SUCU", None, None, None],
    ["GI.GUVC", None, None, None],
    ["GI.QCIX", None, None, None],
    ["GI.GUSP", None, None, None],
    ["GI.SROR", 2.6962, None, None],
    ["GI.AVCS", 2.8735, None, None],
    ["SV.NUBE", 3.1539, 55.0, 97.6],
    ["SV.RBDL", 3.2079, 56.3, 97.6]
], columns=[
    "Station",
    "Distance_deg",
    "P_obs_s",
    "S_obs_s"
])

results["S_minus_P_s"] = (
    results["S_obs_s"] - results["P_obs_s"]
)

results
```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	
0	GI.HUCU	1.2881	24.5	41.0	16.5	
1	GI.SUCU	NaN	NaN	NaN	NaN	
2	GI.GUVC	NaN	NaN	NaN	NaN	
3	GI.QCIX	NaN	NaN	NaN	NaN	
4	GI.GUSP	NaN	NaN	NaN	NaN	
5	GI.SROR	2.6962	NaN	NaN	NaN	
6	GI.AVCS	2.8735	NaN	NaN	NaN	
7	SV.NUBE	3.1539	55.0	97.6	42.6	
8	SV.RBDL	3.2079	56.3	97.6	41.3	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

```

# SROR - P-wave onset investigation

station = "GI.SROR"
tr = all_waveforms[station][0].copy()

# Removing mean/trend
tr.detrend("linear")

# Time relative to earthquake origin
times = tr.times(reftime=event_time)

# Theoretical P arrivals
p_arrivals = [42.02, 47.54, 47.73]

plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.SROR waveform")

# Earthquake origin
plt.axvline(0, linestyle="--", label="Earthquake origin")

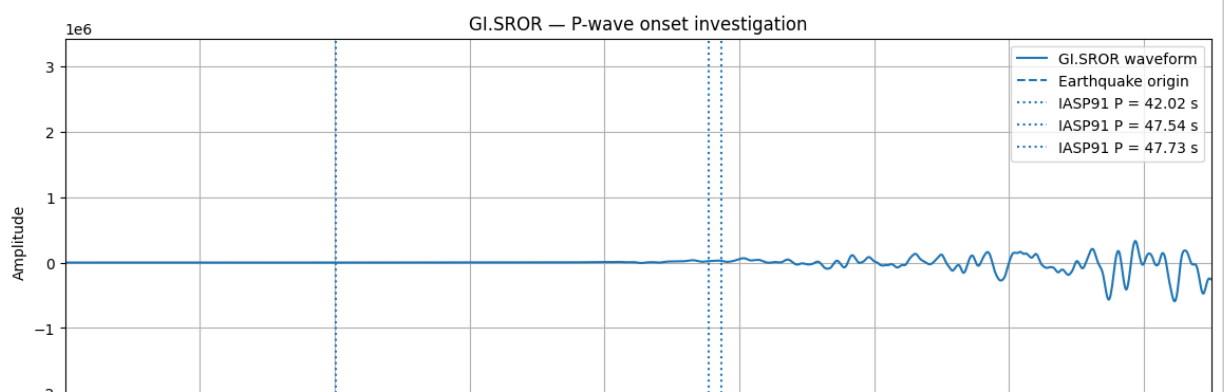
# Theoretical P arrivals
for p in p_arrivals:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

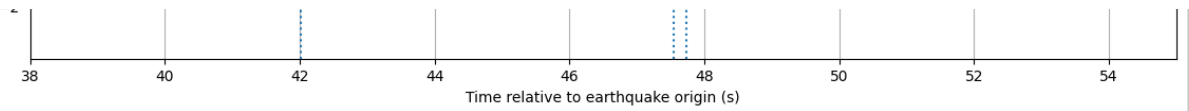
plt.xlim(38, 55)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.SROR - P-wave onset investigation")
plt.legend()
plt.grid(True)

plt.show()

```





```
results.loc[results["Station"] == "GI.SR0R", "P_obs_s"] = 47.5
```

```
# SR0R – S-wave onset investigation

station = "GI.SR0R"
tr = all_waveforms[station][0].copy()

tr.detrend("linear")

times = tr.times(reftime=event_time)

# Theoretical S arrivals
s_arrivals = [74.72, 82.34, 82.67]

plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.SR0R waveform")

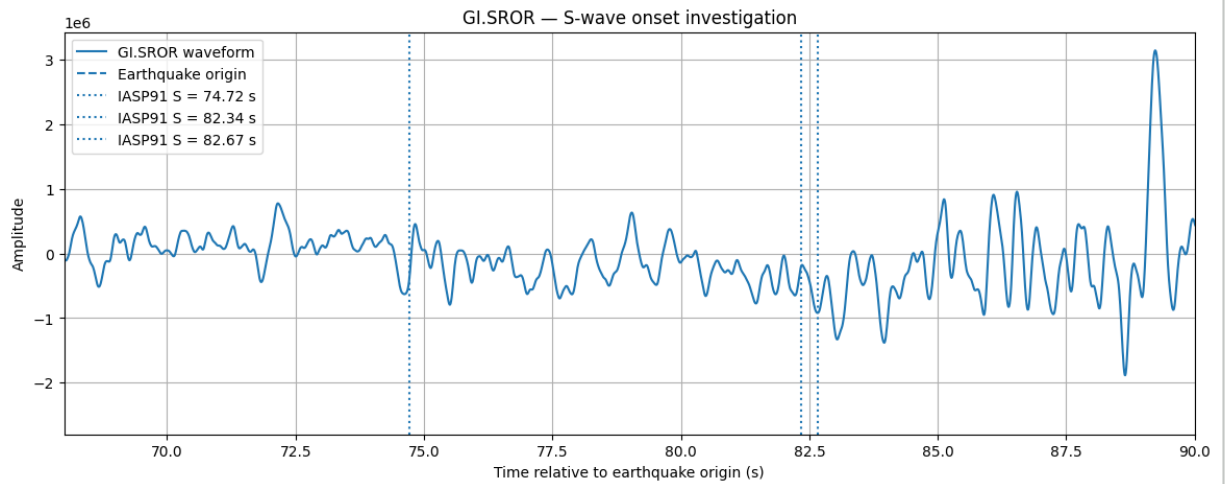
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

for s in s_arrivals:
    plt.axvline(
        s,
        linestyle=":",
        label=f"IASP91 S = {s:.2f} s"
    )
```

```
plt.xlim(68, 90)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.SROR – S-wave onset investigation")
plt.legend()
plt.grid(True)

plt.show()
```



```

results.loc[results["Station"] == "GI.SROR", "S_obs_s"] = 84.8

results.loc[
    results["Station"] == "GI.SROR",
    "S_minus_P_s"
] = (
    results.loc[results["Station"] == "GI.SROR", "S_obs_s"]
    - results.loc[results["Station"] == "GI.SROR", "P_obs_s"]
)

results

```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	
0	GI.HUCU	1.2881	24.5	41.0	16.5	
1	GI.SUCU	NaN	NaN	NaN	NaN	
2	GI.GUVC	NaN	NaN	NaN	NaN	
3	GI.QCIX	NaN	NaN	NaN	NaN	
4	GI.GUSP	NaN	NaN	NaN	NaN	
5	GI.SROR	2.6962	47.5	84.8	37.3	
6	GI.AVCS	2.8735	NaN	NaN	NaN	
7	SV.NUBE	3.1539	55.0	97.6	42.6	
8	SV.RBDL	3.2079	56.3	97.6	41.3	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

```

# GI.AVCS – P-wave onset investigation

station = "GI.AVCS"
tr = all_waveforms[station][0].copy()

tr.detrend("linear")

times = tr.times(reftime=event_time)

p_arrivals = [44.46, 50.56, 50.73]

plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.AVCS waveform")

```

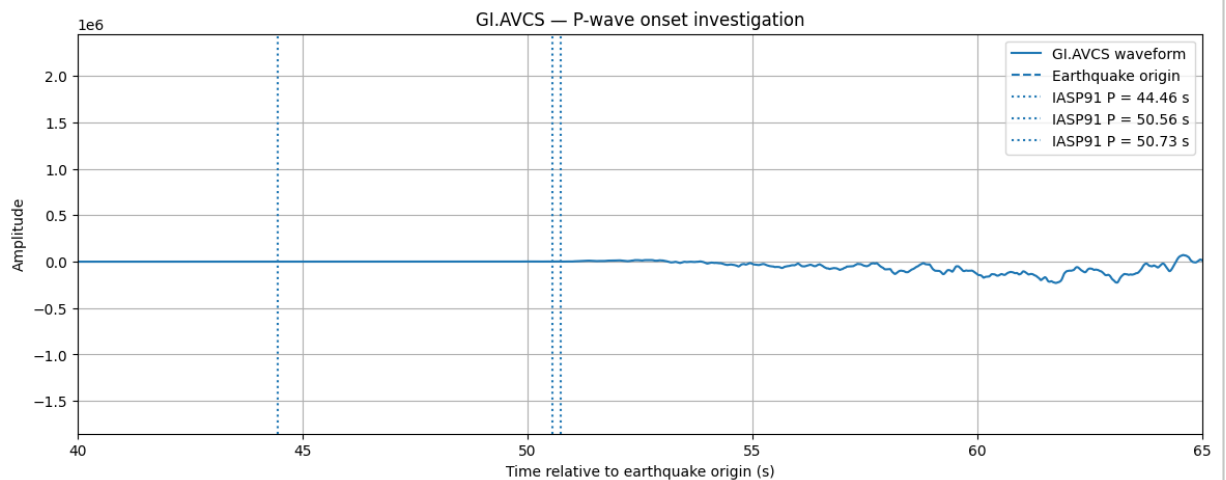
```
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

for p in p_arrivals:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

plt.xlim(40, 65)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.AVCS - P-wave onset investigation")
plt.legend()
plt.grid(True)

plt.show()
```



```
results.loc[results["Station"] == "GI.AVCS", "P_obs_s"] = 50.6
```

```
results
```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	
0	GI.HUCU	1.2881	24.5	41.0	16.5	 
1	GI.SUCU	NaN	NaN	NaN	NaN	
2	GI.GUVC	NaN	NaN	NaN	NaN	
3	GI.QCIX	NaN	NaN	NaN	NaN	
4	GI.GUSP	NaN	NaN	NaN	NaN	
5	GI.SROR	2.6962	47.5	84.8	37.3	
6	GI.AVCS	2.8735	50.6	NaN	NaN	
7	SV.NUBE	3.1539	55.0	97.6	42.6	
8	SV.RBDL	3.2079	56.3	97.6	41.3	

Next steps:

[Generate code with results](#)

[New interactive sheet](#)

```
# GI.AVCS – S-wave onset investigation

station = "GI.AVCS"
tr = all_waveforms[station][0].copy()

tr.detrend("linear")

times = tr.times(reftime=event_time)

s_arrivals = [79.10, 87.58, 87.88]

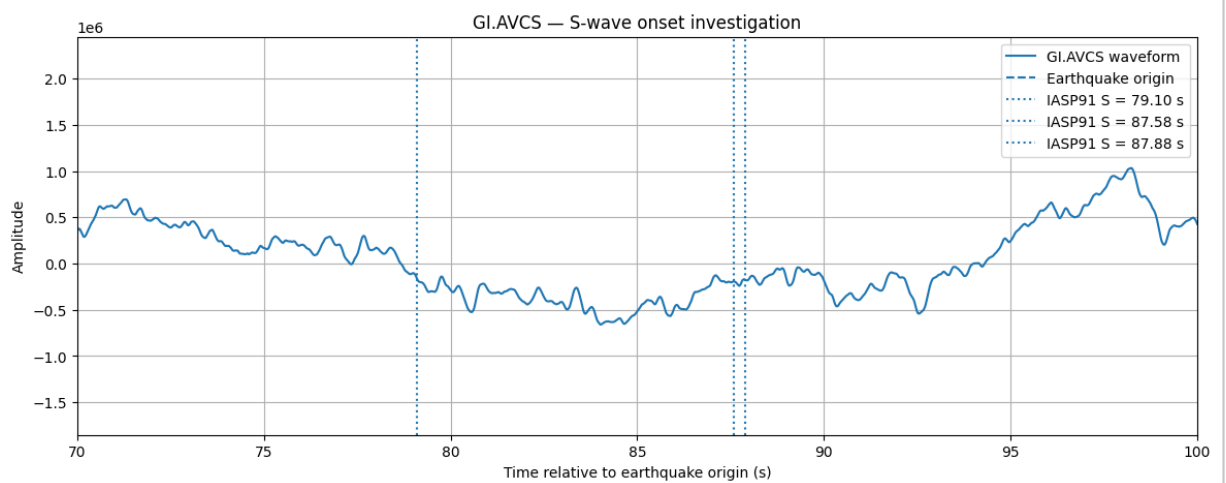
plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.AVCS waveform")

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

for s in s_arrivals:
    plt.axvline(
```

```
s,  
    linestyle=":",  
    label=f"IASP91 S = {s:.2f} s"  
)  
  
plt.xlim(70, 100)  
  
plt.xlabel("Time relative to earthquake origin (s)")  
plt.ylabel("Amplitude")  
plt.title("GI.AVCS – S-wave onset investigation")  
plt.legend()  
plt.grid(True)  
  
plt.show()
```



```
# GI.SUCU – correct station coordinates

station_lat = 14.560000
station_lon = -91.557000

distance_deg = locations2degrees(
    14.57, -92.96,          # earthquake
    station_lat, station_lon
)

print(f"GI.SUCU distance = {distance_deg:.4f} degrees")

arrivals = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_deg,
    phase_list=["P", "S"]
)

print("\nTheoretical arrivals:")
for a in arrivals:
    if a.name.startswith("P") or a.name.startswith("S"):
        print(f"{a.name}: {a.time:.2f} s")
```

GI.SUCU distance = 1.3579 degrees

Theoretical arrivals:
P: 23.62 s
P: 25.22 s
S: 41.61 s
S: 43.67 s

```
# GI.SUCU – corrected P-wave onset investigation
```

```
station = "GI.SUCU"
tr = all_waveforms[station][0].copy()

tr.detrend("linear")

times = tr.times(reftime=event_time)

p_arrivals = [23.62, 25.22]

plt.figure(figsize=(14, 5))

plt.plot(
    times,
    tr.data,
    label="GI.SUCU waveform"
```

```

)

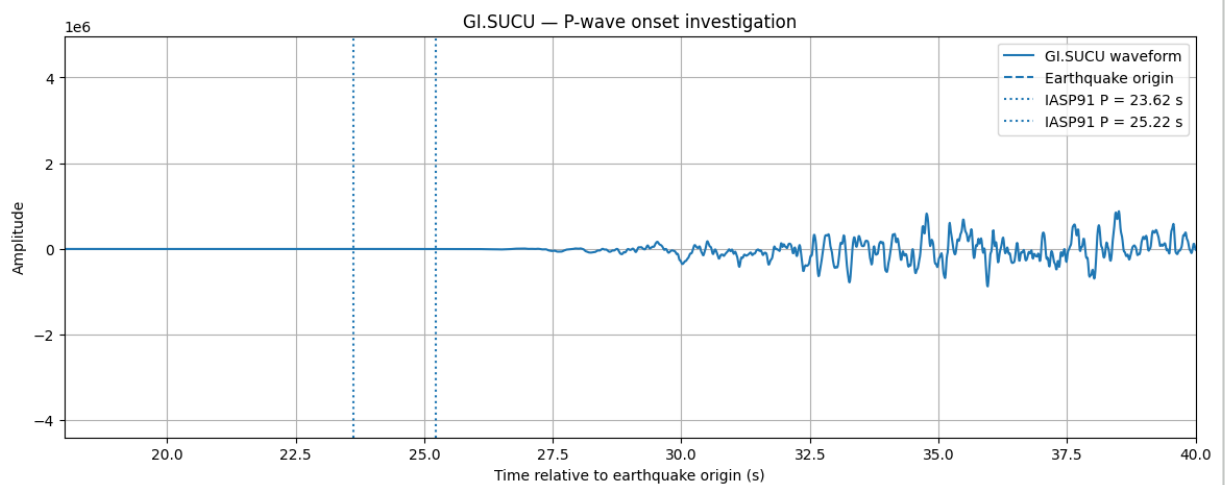
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

for p in p_arrivals:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

plt.xlim(18, 40)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.SUCU – P-wave onset investigation")
plt.legend()
plt.grid()
plt.show()

```



```
from obspy.signal.trigger import classic_sta_lta

tr_pick = all_waveforms["GI.SUCU"][0].copy()
tr_pick.detrend("linear")

# Focus around the suspected P arrival
tr_pick.trim(
    starttime=event_time - 10,
    endtime=event_time + 60
)

df = tr_pick.stats.sampling_rate

sta_seconds = 2
lta_seconds = 10

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft = classic_sta_lta(
    tr_pick.data,
    sta_samples,
    lta_samples
)

time = np.arange(len(cft)) / df - 10

plt.figure(figsize=(14, 5))

plt.plot(time, cft, label="STA/LTA")

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

plt.axvline(
    23.62,
    linestyle=":",
    label="IASP91 P = 23.62 s"
)

plt.axvline(
    25.22,
    linestyle=":",
```

```

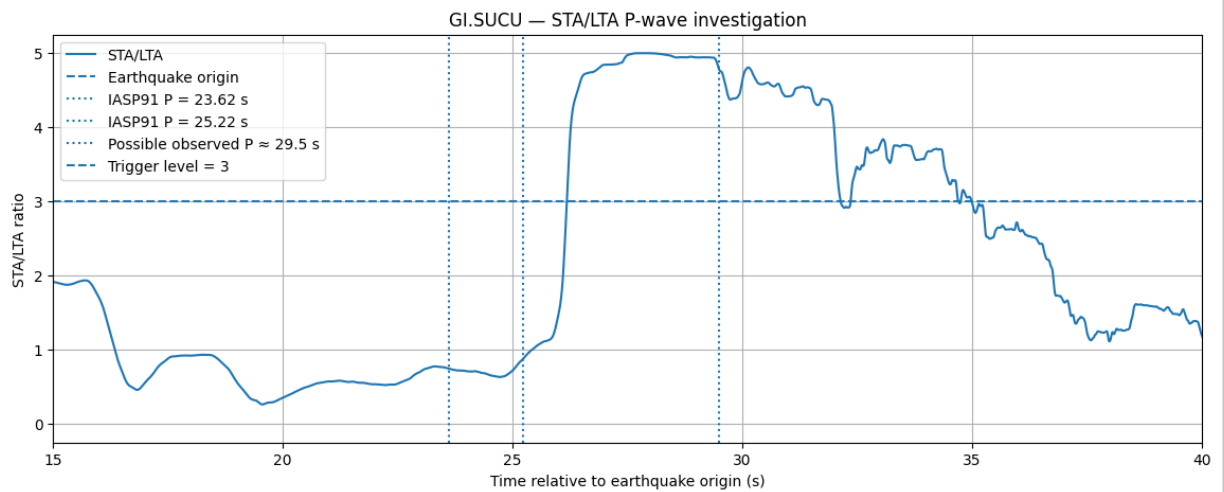
        label="IASP91 P = 25.22 s"
    )

    plt.axvline(
        29.5,
        linestyle=":",
        label="Possible observed P ≈ 29.5 s"
    )

    plt.axhline(
        3,
        linestyle="--",
        label="Trigger level = 3"
    )

    plt.xlim(15, 40)
    plt.xlabel("Time relative to earthquake origin (s)")
    plt.ylabel("STA/LTA ratio")
    plt.title("GI.SUCU – STA/LTA P-wave investigation")
    plt.legend()
    plt.grid()
    plt.show()

```



```
tr = all_waveforms["GI.SUCU"][0].copy()
tr.detrend("linear")

times = tr.times(reftime=event_time)

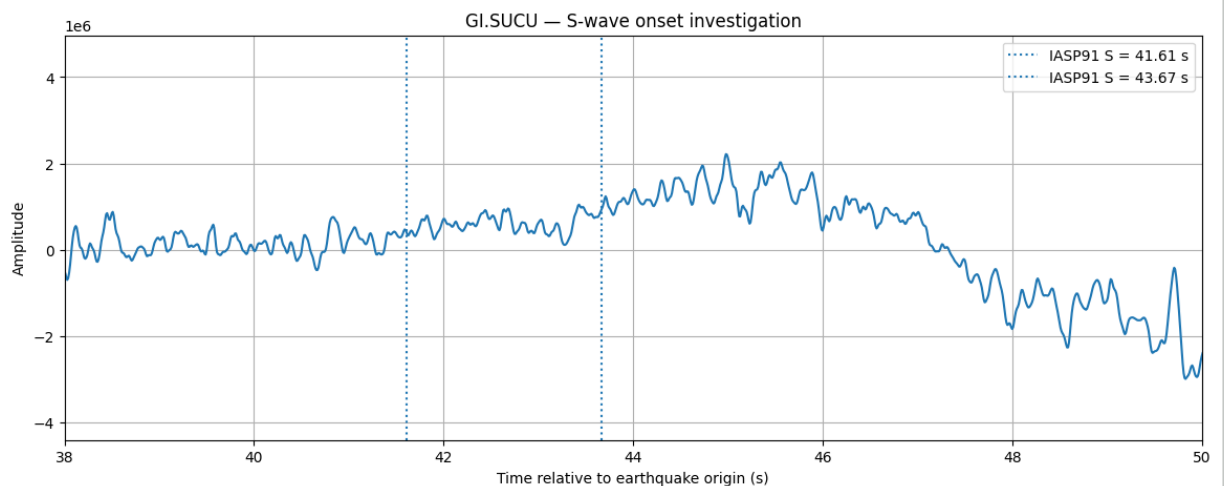
plt.figure(figsize=(14, 5))

plt.plot(times, tr.data)

plt.axvline(41.61, linestyle=":", label="IASP91 S = 41.61 s")
plt.axvline(43.67, linestyle=":", label="IASP91 S = 43.67 s")

plt.xlim(38, 50)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.SUCU – S-wave onset investigation")
plt.legend()
plt.grid()
plt.show()
```



```
# GI.GUVC – station distance and theoretical arrivals

station_lat = 14.532
station_lon = -90.498

distance_deg = locations2degrees(
    14.57, -92.96,          # earthquake
    station_lat, station_lon
)

print(f"GI.GUVC distance = {distance_deg:.4f} degrees")

arrivals = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_deg,
    phase_list=["P", "S"]
)

print("\nTheoretical arrivals:")

for a in arrivals:
    if a.name.startswith("P") or a.name.startswith("S"):
        print(f"{a.name}: {a.time:.2f} s")
```

GI.GUVC distance = 2.3833 degrees

Theoretical arrivals:

P: 37.72 s
P: 42.20 s
P: 42.43 s
S: 66.98 s
S: 73.10 s
S: 73.50 s

```
# GI.GUVC – P-wave onset investigation

tr = all_waveforms["GI.GUVC"][0].copy()
tr.detrend("linear")

times = tr.times(reftime=event_time)

p_arrivals = [37.72, 42.20, 42.43]

plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.GUVC waveform")

plt.axvline(
```

```

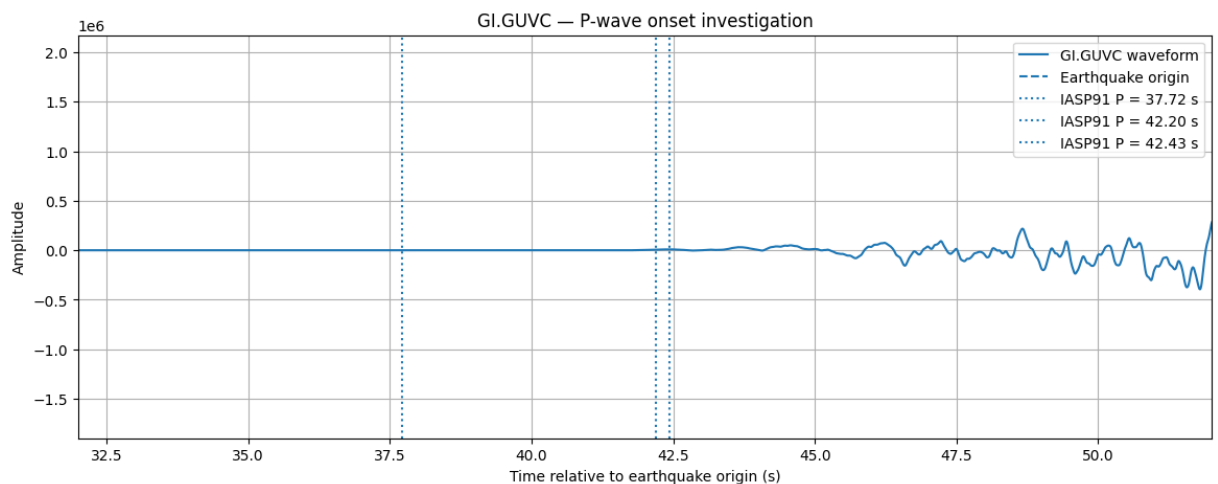
    0,
    linestyle="--",
    label="Earthquake origin"
)

for p in p_arrivals:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

plt.xlim(32, 52)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUVC – P-wave onset investigation")
plt.legend()
plt.grid()
plt.show()

```



```
# GI.GUVC – S-wave onset investigation
```

```

tr = all_waveforms["GI.GUVC"][0].copy()
tr.detrend("linear")

times = tr.times(reftime=event_time)

s_arrivals = [66.98, 73.10, 73.50]

plt.figure(figsize=(14, 5))

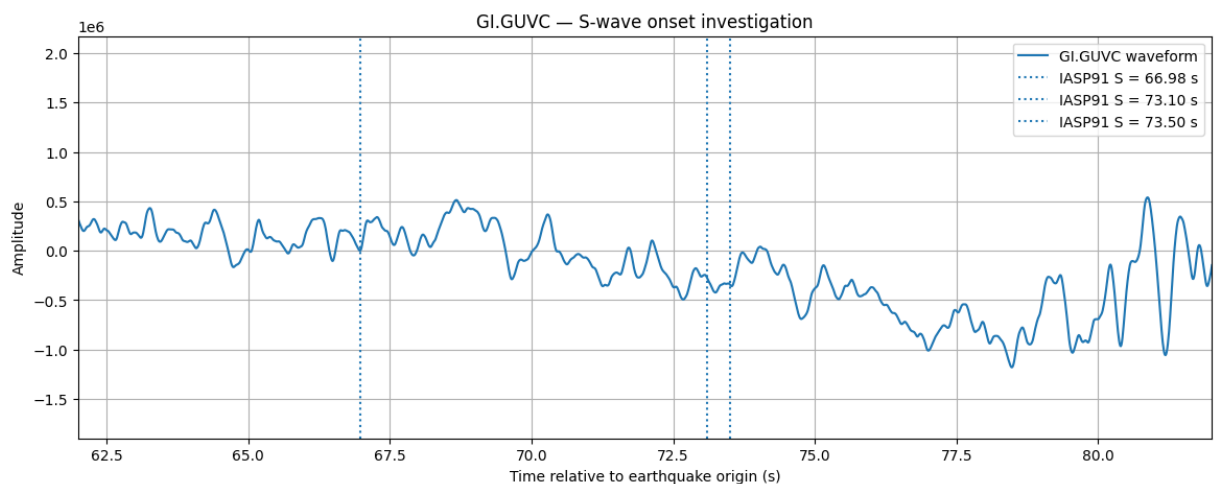
plt.plot(times, tr.data, label="GI.GUVC waveform")

for s in s_arrivals:
    plt.axvline(
        s,
        linestyle=":",
        label=f"IASP91 S = {s:.2f} s"
    )

plt.xlim(62, 82)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUVC – S-wave onset investigation")
plt.legend()
plt.grid()
plt.show()

```



```
# GI.QCIX – station distance and theoretical arrivals

station_lat = 16.003414
station_lon = -90.777245

distance_deg = locations2degrees(
    14.57, -92.96,
    station_lat, station_lon
)

print(f"GI.QCIX distance = {distance_deg:.4f} degrees")

arrivals = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_deg,
    phase_list=["P", "S"]
)

print("\nTheoretical arrivals:")

for a in arrivals:
    if a.name.startswith("P") or a.name.startswith("S"):
        print(f"{a.name}: {a.time:.2f} s")
```

GI.QCIX distance = 2.5471 degrees

Theoretical arrivals:

P: 39.97 s

P: 45.00 s

P: 45.20 s

S: 71.03 s

S: 77.94 s

S: 78.30 s

```
# GI.QCIX – P-wave onset investigation
```

```
tr = all_waveforms["GI.QCIX"][0].copy()
tr.detrend("linear")
```

```
times = tr.times(reftime=event_time)
```

```
p_arrivals = [39.97, 45.00, 45.20]
```

```

plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.QCIX waveform")

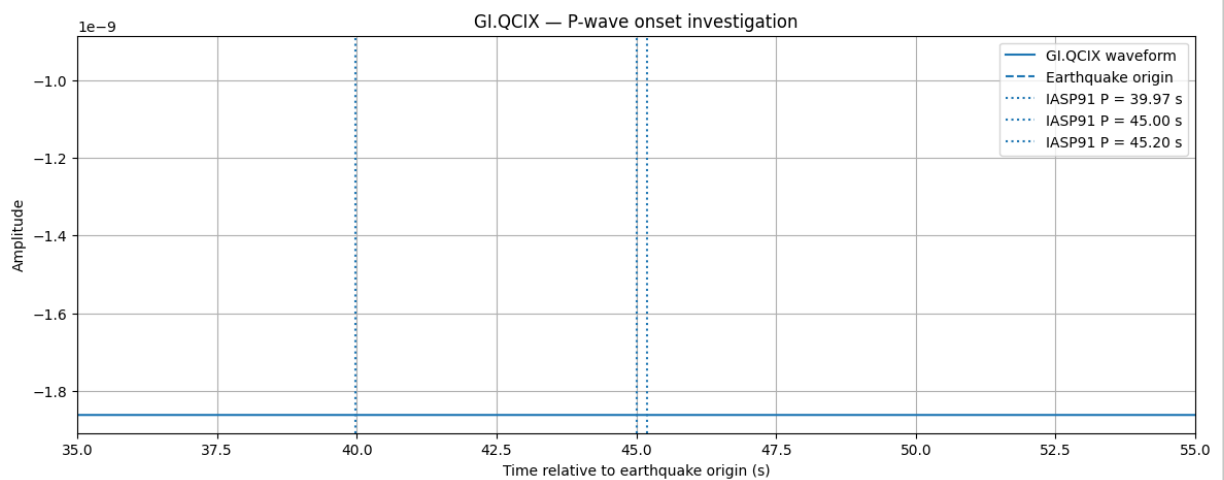
plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)

for p in p_arrivals:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

plt.xlim(35, 55)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.QCIX – P-wave onset investigation")
plt.legend()
plt.grid()
plt.show()

```



```
tr = all_waveforms["GI.QCIX"][0]

print("Minimum:", tr.data.min())
print("Maximum:", tr.data.max())
print("Mean:", tr.data.mean())
print("Standard deviation:", tr.data.std())
print("Sampling rate:", tr.stats.sampling_rate)
print("Number of samples:", len(tr.data))
```

```
Minimum: 8388607
Maximum: 8388607
Mean: 8388607.0
Standard deviation: 0.0
Sampling rate: 100.0
Number of samples: 66000
```

```
# GI.GUSP – station distance and theoretical arrivals

station_lat = 14.552
station_lon = -90.3168

distance_deg = locations2degrees(
    14.57, -92.96,
    station_lat, station_lon
)

print(f"GI.GUSP distance = {distance_deg:.4f} degrees")

arrivals = model.get_travel_times(
    source_depth_in_km=22.0,
    distance_in_degree=distance_deg,
    phase_list=["P", "S"]
)

print("\nTheoretical arrivals:")

for a in arrivals:
    if a.name.startswith("P") or a.name.startswith("S"):
        print(f"{a.name}: {a.time:.2f} s")
```

GI.GUSP distance = 2.5584 degrees

Theoretical arrivals:

P: 40.13 s
 P: 45.19 s
 P: 45.39 s
 S: 71.31 s
 S: 78.27 s
 S: 78.63 s

```
tr = all_waveforms["GI.GUSP"][0].copy()
tr.detrend("linear")

times = tr.times(reftime=event_time)

plt.figure(figsize=(14, 5))

plt.plot(times, tr.data, label="GI.GUSP waveform")

plt.axvline(
    0,
    linestyle="--",
    label="Earthquake origin"
)
```

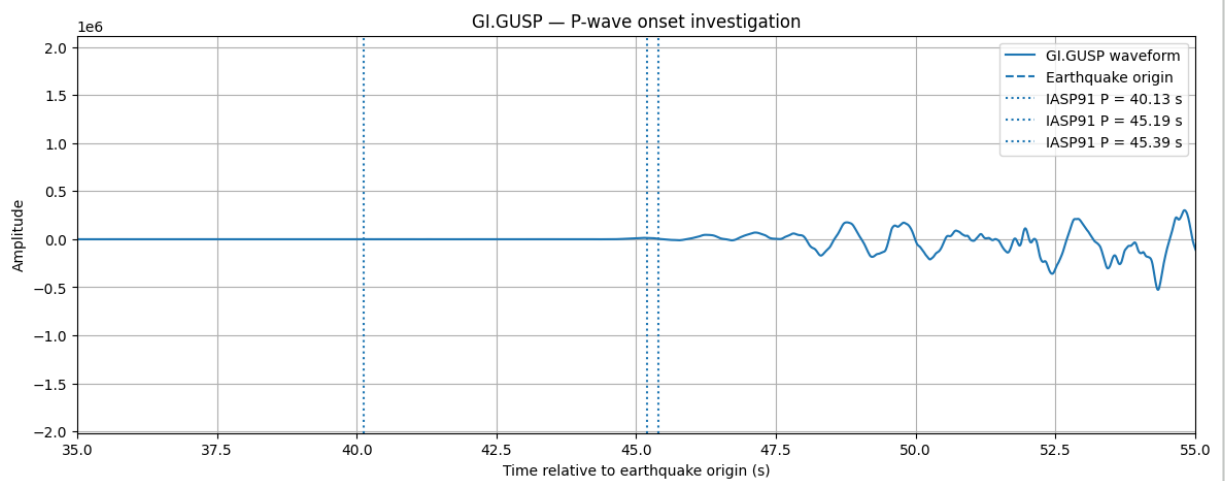
```

for p in [40.13, 45.19, 45.39]:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

plt.xlim(35, 55)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUSP – P-wave onset investigation")
plt.legend()
plt.grid()
plt.show()

```



```

from obspy.signal.trigger import classic_sta_lta

tr_pick = all_waveforms["GI.GUSP"][0].copy()
tr_pick.detrend("linear")

```

```

tr_pick.trim(
    starttime=event_time + 30,
    endtime=event_time + 60
)

df = tr_pick.stats.sampling_rate

sta_seconds = 1
lta_seconds = 10

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft = classic_sta_lta(
    tr_pick.data,
    sta_samples,
    lta_samples
)

time = np.arange(len(cft)) / df + 30

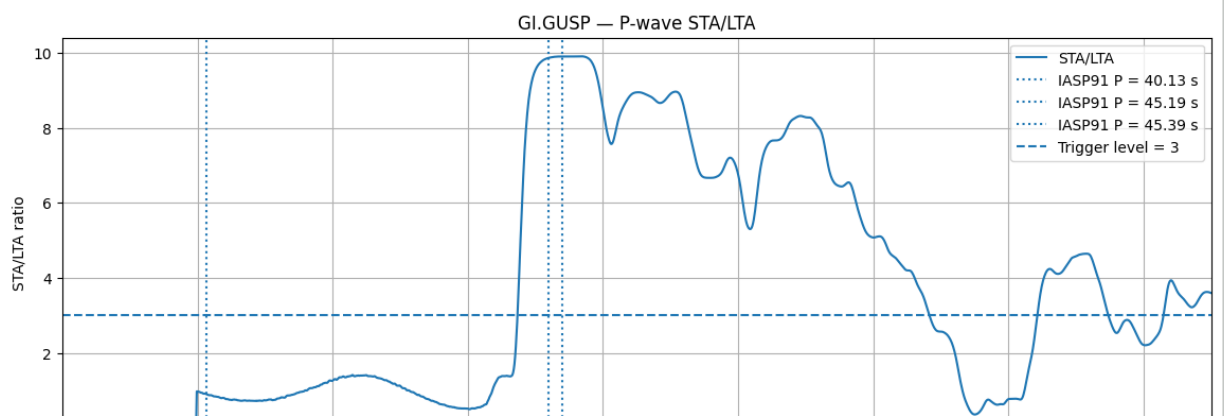
plt.figure(figsize=(14, 5))
plt.plot(time, cft, label="STA/LTA")

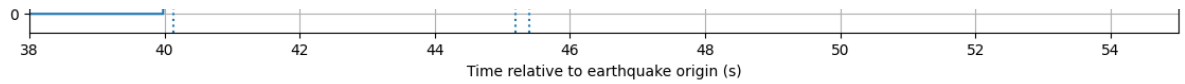
for p in [40.13, 45.19, 45.39]:
    plt.axvline(p, linestyle=":", label=f"IASP91 P = {p:.2f} s")

plt.axhline(3, linestyle="--", label="Trigger level = 3")

plt.xlim(38, 55)
plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.GUSP - P-wave STA/LTA")
plt.legend()
plt.grid()
plt.show()

```





```

from obspy.signal.trigger import classic_sta_lta

tr_pick = all_waveforms["GI.GUSP"][0].copy()

print(tr_pick.id)

tr_pick.detrend("linear")

tr_pick.trim(
    starttime=event_time + 30,
    endtime=event_time + 60
)

df = tr_pick.stats.sampling_rate

sta_seconds = 1
lta_seconds = 10

sta_samples = int(sta_seconds * df)
lta_samples = int(lta_seconds * df)

cft = classic_sta_lta(
    tr_pick.data,
    sta_samples,
    lta_samples
)

time = np.arange(len(cft)) / df + 30

plt.figure(figsize=(14, 5))

plt.plot(time, cft, label="STA/LTA")

```

```

for p in [40.13, 45.19, 45.39]:
    plt.axvline(
        p,
        linestyle=":",
        label=f"IASP91 P = {p:.2f} s"
    )

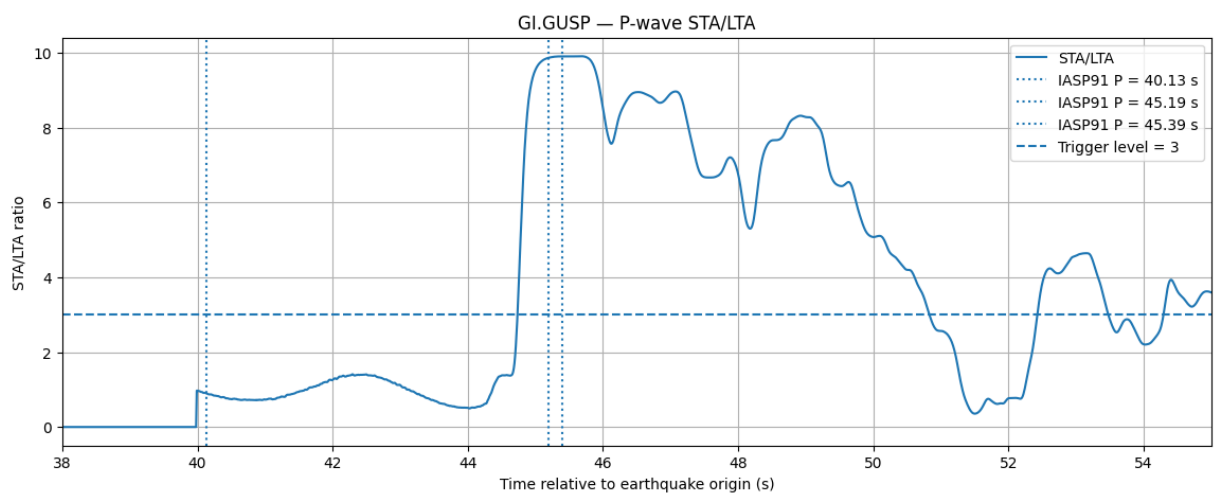
plt.axhline(
    3,
    linestyle="--",
    label="Trigger level = 3"
)

plt.xlim(38, 55)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("STA/LTA ratio")
plt.title("GI.GUSP - P-wave STA/LTA")
plt.legend()
plt.grid()
plt.show()

```

GI.GUSP..HHZ



```

tr = all_waveforms["GI.GUSP"][0].copy()
tr.detrend("linear")

times = tr.times(reftime=event_time)

s_arrivals = [71.31, 78.27, 78.63]

plt.figure(figsize=(14, 5))

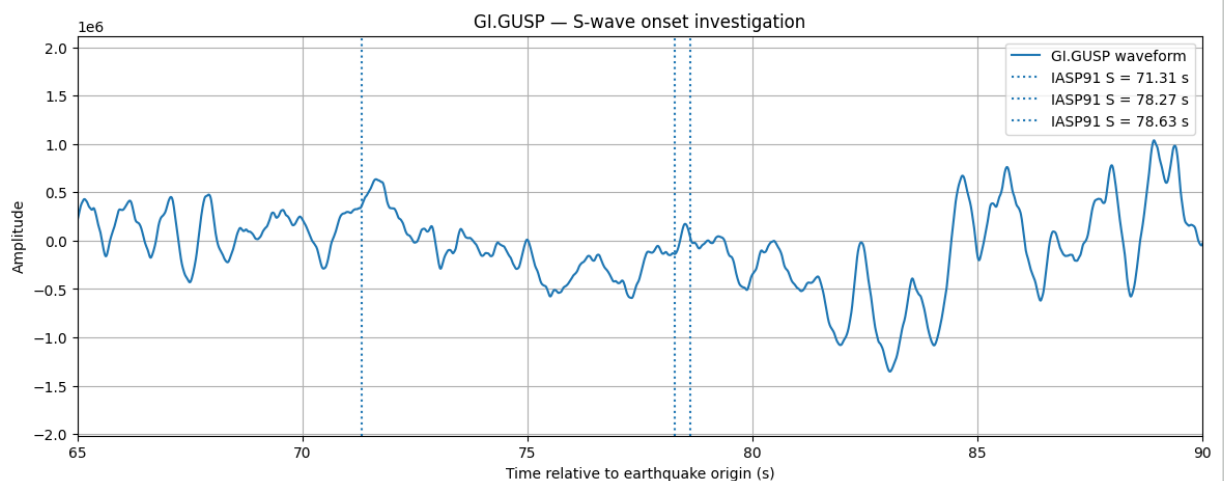
plt.plot(times, tr.data, label="GI.GUSP waveform")

for s in s_arrivals:
    plt.axvline(
        s,
        linestyle=":",
        label=f"IASP91 S = {s:.2f} s"
    )

plt.xlim(65, 90)

plt.xlabel("Time relative to earthquake origin (s)")
plt.ylabel("Amplitude")
plt.title("GI.GUSP - S-wave onset investigation")
plt.legend()
plt.grid()
plt.show()

```



```
import pandas as pd
import numpy as np

results = pd.DataFrame({
    "Station": [
        "GI.HUCU",
        "GI.SUCU",
        "GI.GUVC",
        "GI.QCIX",
        "GI.GUSP",
        "GI.SROR",
        "GI.AVCS",
        "SV.NUBE",
        "SV.RBDL"
    ],
    "Distance_deg": [
        1.2881,
        1.3579,
        2.3833,
        2.5471,
        2.5584,
        2.6962,
        2.8735,
        3.1539,
        3.2079
    ],
    "P_obs_s": [
        24.5,
        25.8,
        np.nan,
        np.nan,
        45.1,
        47.5,
        50.6,
        55.0,
        56.3
    ],
    ],
```

```
"S_obs_s": [  
    41.0,  
    43.7,  
    np.nan,  
    np.nan,  
    np.nan,  
    84.8,  
    np.nan,  
    97.6,  
    97.6  
]  
})  
  
results["S_minus_P_s"] = results["S_obs_s"] - results["P_obs_s"]  
  
results
```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	
0	GI.HUCU	1.2881	24.5	41.0	16.5	
1	GI.SUCU	1.3579	25.8	43.7	17.9	
2	GI.GUVC	2.3833	NaN	NaN	NaN	
3	GI.QCIX	2.5471	NaN	NaN	NaN	
4	GI.GUSP	2.5584	45.1	NaN	NaN	
5	GI.SROR	2.6962	47.5	84.8	37.3	
6	GI.AVCS	2.8735	50.6	NaN	NaN	
7	SV.NUBE	3.1539	55.0	97.6	42.6	
8	SV.RBDL	3.2079	56.3	97.6	41.3	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

```
# Keep only stations having both P and S picks
```

```
sp_data = results.dropna(subset=["P_obs_s", "S_obs_s"]).copy()
```

```
print(sp_data)
```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s
0	GI.HUCU	1.2881	24.5	41.0	16.5
1	GI.SUCU	1.3579	25.8	43.7	17.9
5	GI.SROR	2.6962	47.5	84.8	37.3
7	SV.NUBE	3.1539	55.0	97.6	42.6
8	SV.RBDL	3.2079	56.3	97.6	41.3

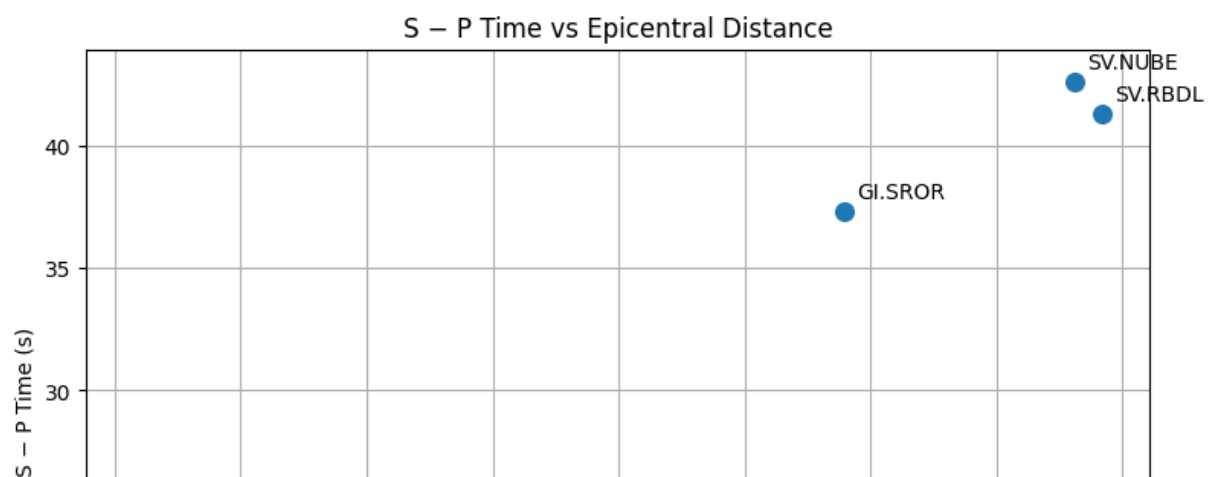
```
plt.figure(figsize=(9, 6))
```

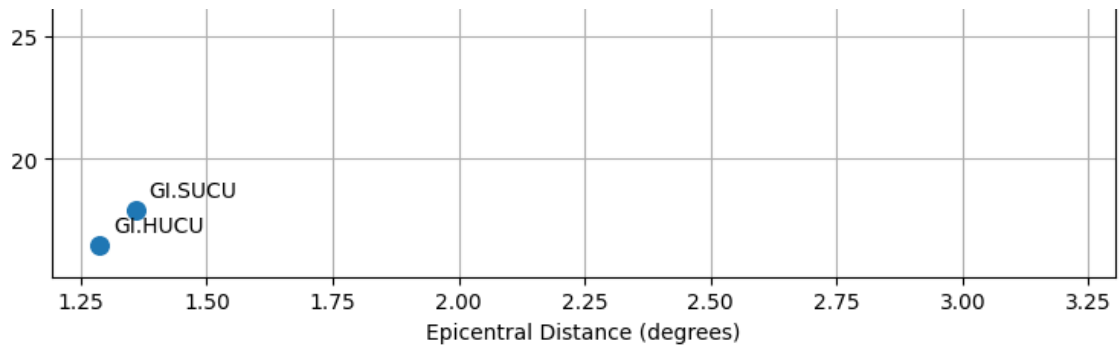
```
plt.scatter(
    sp_data["Distance_deg"],
    sp_data["S_minus_P_s"],
    s=70
)
```

```
for _, row in sp_data.iterrows():
    plt.annotate(
        row["Station"],
        (row["Distance_deg"], row["S_minus_P_s"]),
        xytext=(6, 6),
        textcoords="offset points"
    )
```

```
plt.xlabel("Epicentral Distance (degrees)")
plt.ylabel("S - P Time (s)")
plt.title("S - P Time vs Epicentral Distance")
plt.grid()
```

```
plt.show()
```





```
# Generate IASP91 theoretical S-P times

dist_range = np.linspace(1.0, 3.5, 150)

theoretical_sp = []

for distance in dist_range:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P", "S"]
    )

    p_arrivals = [a.time for a in arrivals if a.name.startswith("P")]
    s_arrivals = [a.time for a in arrivals if a.name.startswith("S")]

    if p_arrivals and s_arrivals:
        # Use the earliest P and earliest S arrivals
        p_time = min(p_arrivals)
        s_time = min(s_arrivals)

        theoretical_sp.append(s_time - p_time)
    else:
        theoretical_sp.append(np.nan)
```

```
plt.figure(figsize=(10, 6))
```

```
# Observed data
```

```

plt.scatter(
    sp_data["Distance_deg"],
    sp_data["S_minus_P_s"],
    s=70,
    label="Observed S-P"
)

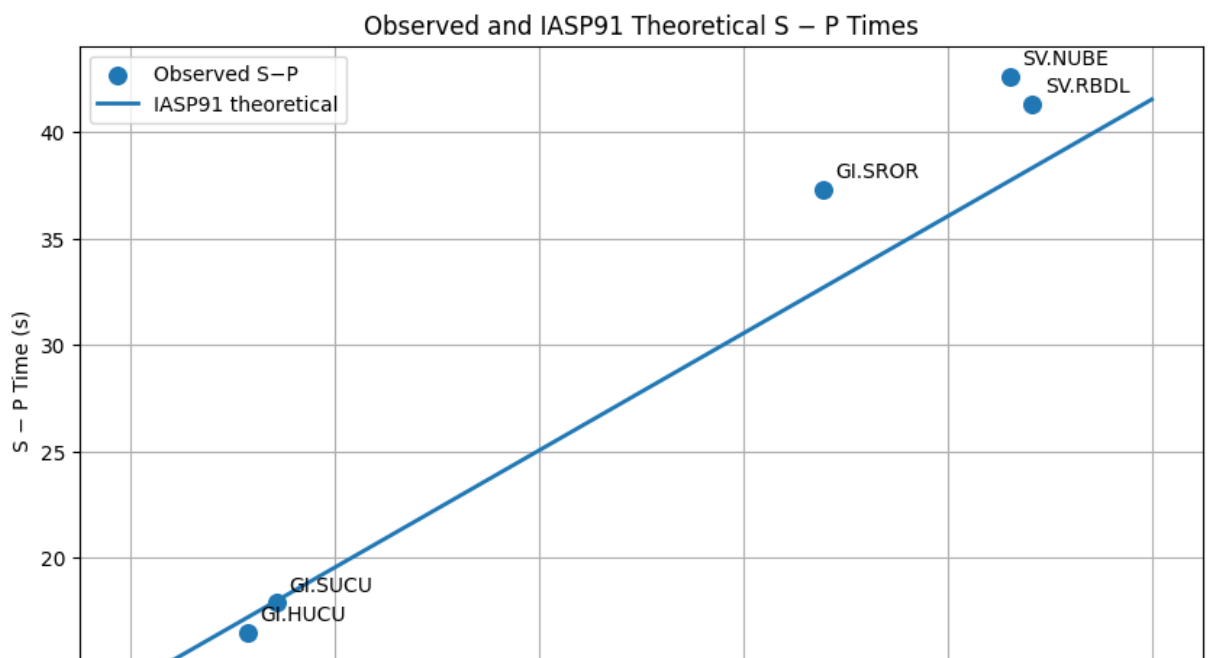
# Station labels
for _, row in sp_data.iterrows():
    plt.annotate(
        row["Station"],
        (row["Distance_deg"], row["S_minus_P_s"]),
        xytext=(6, 6),
        textcoords="offset points"
    )

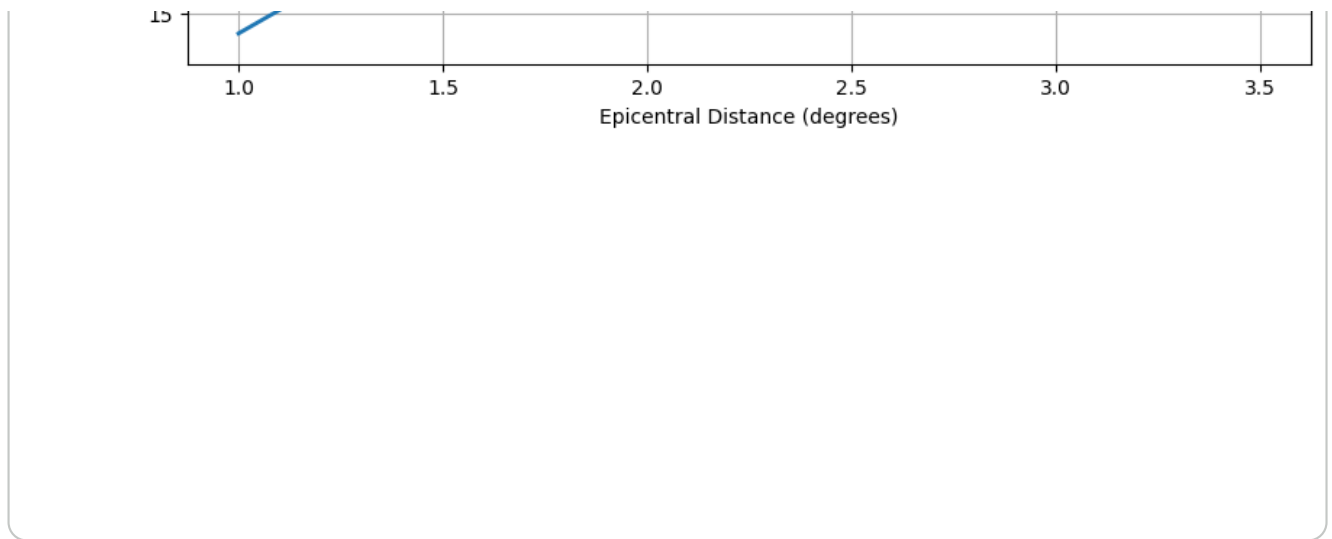
# IASP91 theoretical curve
plt.plot(
    dist_range,
    theoretical_sp,
    linewidth=2,
    label="IASP91 theoretical"
)

plt.xlabel("Epicentral Distance (degrees)")
plt.ylabel("S - P Time (s)")
plt.title("Observed and IASP91 Theoretical S - P Times")
plt.legend()
plt.grid()

plt.show()

```





```
# Calculating theoretical S-P time at each observed station distance

sp_comparison = sp_data.copy()

theoretical_values = []

for distance in sp_comparison["Distance_deg"]:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P", "S"]
    )

    p_arrivals = [a.time for a in arrivals if a.name.startswith("P")]
    s_arrivals = [a.time for a in arrivals if a.name.startswith("S")]

    p_theory = min(p_arrivals)
    s_theory = min(s_arrivals)

    theoretical_values.append(s_theory - p_theory)

sp_comparison["SP_theory_s"] = theoretical_values

sp_comparison["SP_residual_s"] = (
    sp_comparison["S_minus_P_s"]
    - sp_comparison["SP_theory_s"]
)

sp_comparison
```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	SP_theory_s
0	GI.HUCU	1.2881	24.5	41.0	16.5	17.230732
1	GI.SUCU	1.3579	25.8	43.7	17.9	17.997358
5	GI.SROR	2.6962	47.5	84.8	37.3	32.694360
7	SV.NUBE	3.1539	55.0	97.6	42.6	37.718580
8	SV.RBDL	3.2079	56.3	97.6	41.3	38.311239

Next steps: [Generate code with sp_comparison](#)

[New interactive sheet](#)

```
# Residual statistics
```

```
mean_residual = sp_comparison["SP_residual_s"].mean()
mean_abs_residual = sp_comparison["SP_residual_s"].abs().mean()
rmse = np.sqrt(
    np.mean(sp_comparison["SP_residual_s"]**2)
)

print(f"Mean residual      = {mean_residual:.3f} s")
print(f"Mean absolute error = {mean_abs_residual:.3f} s")
print(f"RMSE                = {rmse:.3f} s")
```

```
Mean residual      = 2.330 s
Mean absolute error = 2.661 s
RMSE               = 3.302 s
```

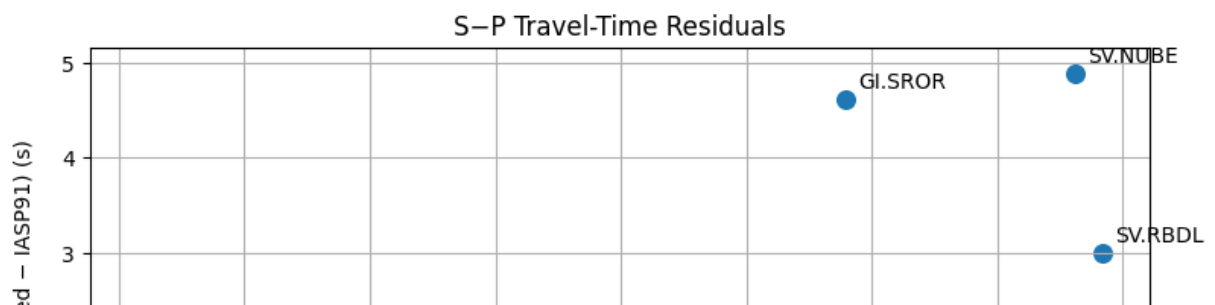
```
plt.figure(figsize=(9, 5))

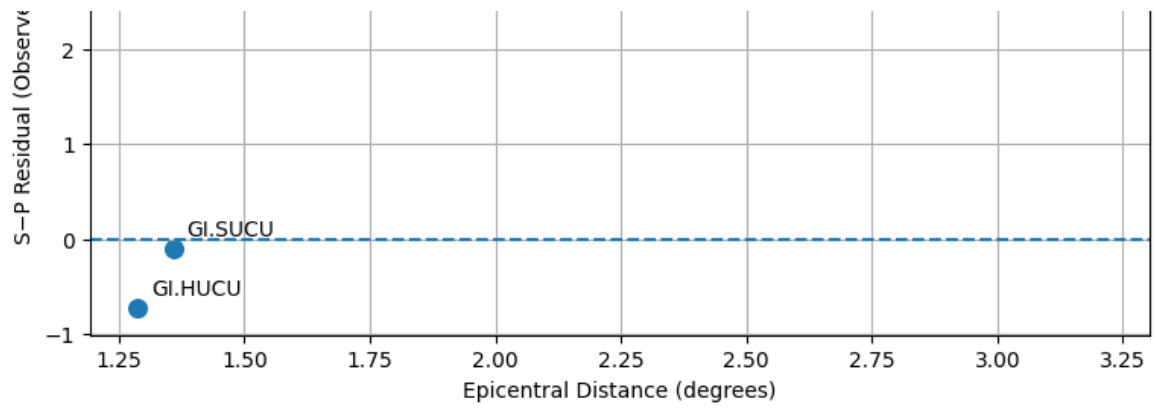
plt.axhline(0, linestyle="--")

plt.scatter(
    sp_comparison["Distance_deg"],
    sp_comparison["SP_residual_s"],
    s=70
)

for _, row in sp_comparison.iterrows():
    plt.annotate(
        row["Station"],
        (row["Distance_deg"], row["SP_residual_s"]),
        xytext=(6, 6),
        textcoords="offset points"
    )

plt.xlabel("Epicentral Distance (degrees)")
plt.ylabel("S-P Residual (Observed - IASP91) (s)")
plt.title("S-P Travel-Time Residuals")
plt.grid()
plt.show()
```





```

# Stations with reliable observed P picks

p_data = results.dropna(subset=["P_obs_s"]).copy()

p_theory = []

for distance in p_data["Distance_deg"]:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P"]
    )

    # Earliest theoretical P arrival
    p_time = min(a.time for a in arrivals)

    p_theory.append(p_time)

p_data["P_theory_s"] = p_theory

p_data["P_residual_s"] = (
    p_data["P_obs_s"] - p_data["P_theory_s"]
)

p_data

```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	P_theory_s
0	GI.HUCU	1.2881	24.5	41.0	16.5	22.655713
1	GI.SUCU	1.3579	25.8	43.7	17.9	23.615735
4	GI.GUSP	2.5584	45.1	NaN	NaN	40.125884
5	GI.SROR	2.6962	47.5	84.8	37.3	42.020904
6	GI.AVCS	2.8735	50.6	NaN	NaN	44.458944
7	SV.NUBE	3.1539	55.0	97.6	42.6	48.314495
8	SV.RBDL	3.2079	56.3	97.6	41.3	49.056987

Next steps:

[Generate code with p_data](#)[New interactive sheet](#)

```

# Generate smooth IASP91 P-wave travel-time curve

```

```

dist_range = np.linspace(1.0, 3.5, 150)

```

```
p_curve = []

for distance in dist_range:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P"]
    )

    p_curve.append(
        min(a.time for a in arrivals)
    )

plt.figure(figsize=(10, 6))

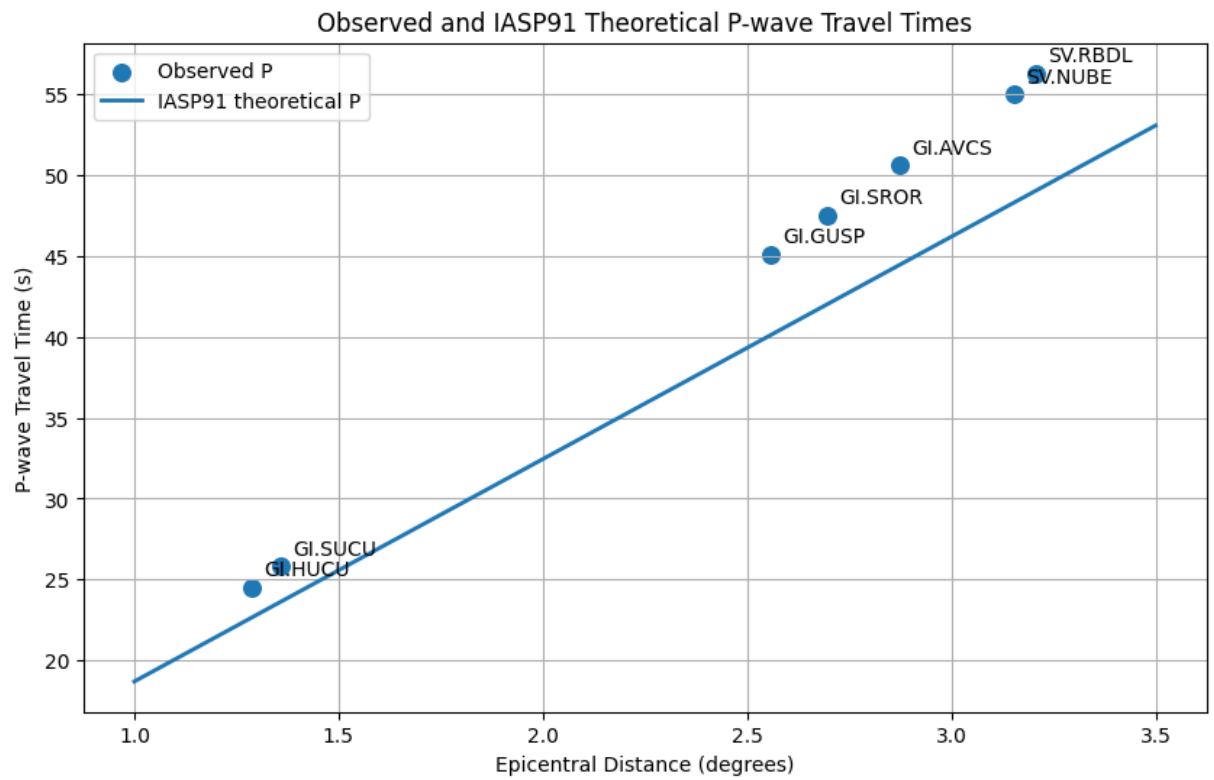
# Observed P picks
plt.scatter(
    p_data["Distance_deg"],
    p_data["P_obs_s"],
    s=70,
    label="Observed P"
)

# Station names
for _, row in p_data.iterrows():
    plt.annotate(
        row["Station"],
        (row["Distance_deg"], row["P_obs_s"]),
        xytext=(6, 6),
        textcoords="offset points"
    )

# IASP91 curve
plt.plot(
    dist_range,
    p_curve,
    linewidth=2,
    label="IASP91 theoretical P"
)

plt.xlabel("Epicentral Distance (degrees)")
plt.ylabel("P-wave Travel Time (s)")
plt.title("Observed and IASP91 Theoretical P-wave Travel Times")
plt.legend()
plt.grid()
```

```
plt.show()
```



```

# Generate the SECOND IASP91 P-arrival curve

dist_range = np.linspace(1.0, 3.5, 150)

p_second_curve = []

for distance in dist_range:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P"]
    )

    p_arrivals = sorted(
        [a.time for a in arrivals if a.name.startswith("P")]
    )

    if len(p_arrivals) >= 2:
        p_second_curve.append(p_arrivals[1])
    else:
        p_second_curve.append(np.nan)

```

```

plt.figure(figsize=(10, 6))

# Observed P
plt.scatter(
    p_data["Distance_deg"],
    p_data["P_obs_s"],
    s=70,
    label="Observed P"
)

# Station labels
for _, row in p_data.iterrows():
    plt.annotate(
        row["Station"],
        (row["Distance_deg"], row["P_obs_s"]),
        xytext=(6, 6),
        textcoords="offset points"
    )

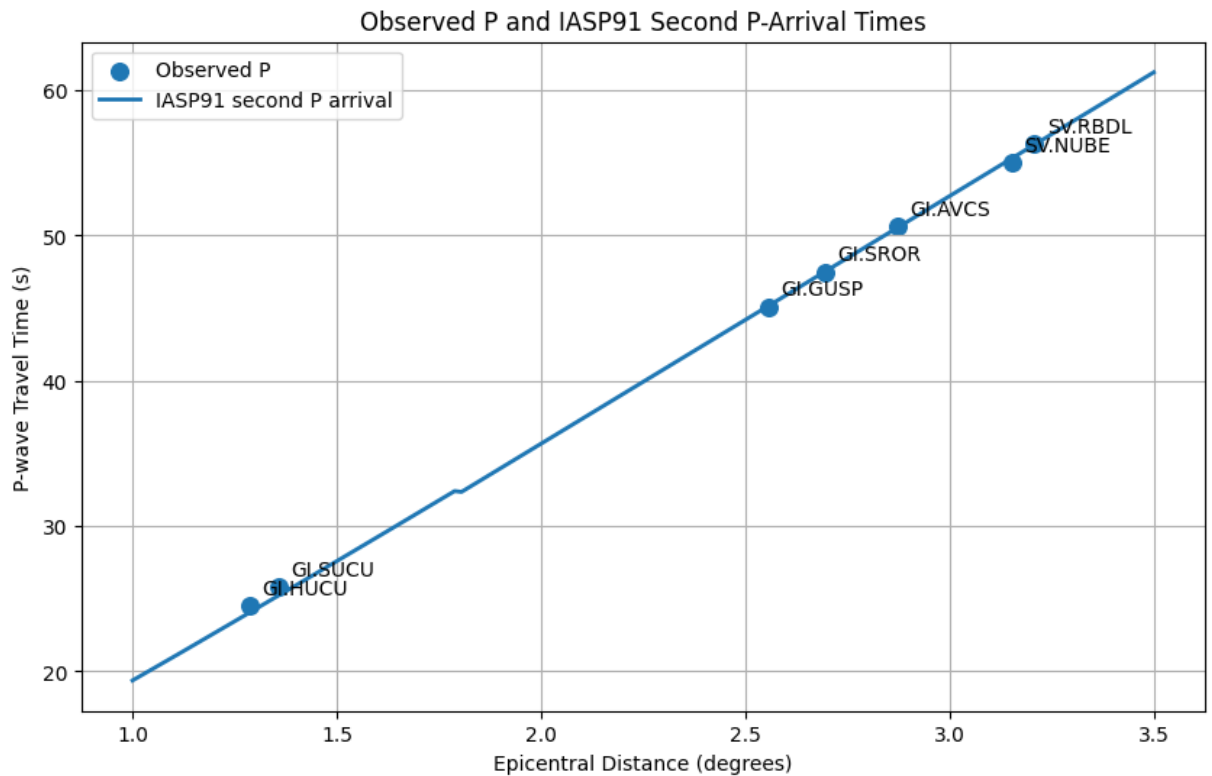
# Second theoretical P branch
plt.plot(
    dist_range,
    p_second_curve,
    linewidth=2,

```

```
        label="IASP91 second P arrival")
    )

plt.xlabel("Epicentral Distance (degrees)")
plt.ylabel("P-wave Travel Time (s)")
plt.title("Observed P and IASP91 Second P-Arrival Times")
plt.legend()
plt.grid()

plt.show()
```



```

p_comparison = p_data.copy()

second_p_values = []

for distance in p_comparison["Distance_deg"]:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P"]
    )

    p_arrivals = sorted(
        [a.time for a in arrivals if a.name.startswith("P")]
    )

    second_p_values.append(p_arrivals[1])

p_comparison["P_theory_second_s"] = second_p_values

p_comparison["P_residual_second_s"] = (
    p_comparison["P_obs_s"]
    - p_comparison["P_theory_second_s"]
)

p_comparison

```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	P_theory_s	
0	GI.HUCU	1.2881	24.5	41.0	16.5	22.655713	
1	GI.SUCU	1.3579	25.8	43.7	17.9	23.615735	
4	GI.GUSP	2.5584	45.1	NaN	NaN	40.125884	
5	GI.SROR	2.6962	47.5	84.8	37.3	42.020904	
6	GI.AVCS	2.8735	50.6	NaN	NaN	44.458944	
7	SV.NUBE	3.1539	55.0	97.6	42.6	48.314495	
8	SV.RBDL	3.2079	56.3	97.6	41.3	49.056987	

Next steps:

[Generate code with p_comparison](#)[New interactive sheet](#)

```
p_mean_residual = p_comparison["P_residual_second_s"].mean()

p_mae = p_comparison["P_residual_second_s"].abs().mean()

p_rmse = np.sqrt(
    np.mean(p_comparison["P_residual_second_s"]**2)
)

print(f"Mean P residual      = {p_mean_residual:.3f} s")
print(f"Mean absolute P error = {p_mae:.3f} s")
print(f"P-wave RMSE          = {p_rmse:.3f} s")
```

```
Mean P residual      = 0.089 s
Mean absolute P error = 0.222 s
P-wave RMSE          = 0.304 s
```

```
# Compare observed S picks with the second IASP91 S arrival

s_data = results.dropna(subset=["S_obs_s"]).copy()

s_theory_second = []

for distance in s_data["Distance_deg"]:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["S"]
    )

    s_arrivals = sorted(
        [a.time for a in arrivals if a.name.startswith("S")]
    )

    if len(s_arrivals) >= 2:
        s_theory_second.append(s_arrivals[1])
    else:
        s_theory_second.append(np.nan)

s_data["S_theory_second_s"] = s_theory_second

s_data["S_residual_second_s"] = (
    s_data["S_obs_s"]
    - s_data["S_theory_second_s"]
)

s_data
```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	S_theory_sec
0	GI.HUCU	1.2881	24.5	41.0	16.5	41.6
1	GI.SUCU	1.3579	25.8	43.7	17.9	43.6
5	GI.SROR	2.6962	47.5	84.8	37.3	82.3
7	SV.NUBE	3.1539	55.0	97.6	42.6	95.8
8	SV.RBDL	3.2079	56.3	97.6	41.3	97.4

Next steps:

[Generate code with s_data](#)[New interactive sheet](#)

```

sp_data = results.dropna(subset=["P_obs_s", "S_obs_s"]).copy()

model = TauPyModel(model="iasp91")

sp_theory = []

for distance in sp_data["Distance_deg"]:

    arrivals = model.get_travel_times(
        source_depth_in_km=22.0,
        distance_in_degree=distance,
        phase_list=["P", "S"]
    )

    p_arrivals = sorted([a.time for a in arrivals if a.name.startswith("P")])
    s_arrivals = sorted([a.time for a in arrivals if a.name.startswith("S")])

    P2 = p_arrivals[1]
    S2 = s_arrivals[1]

    sp_theory.append(S2 - P2)

sp_data["SP_theory_s"] = sp_theory

sp_data["SP_residual_s"] = (
    sp_data["S_minus_P_s"] - sp_data["SP_theory_s"]
)

sp_data

```

	Station	Distance_deg	P_obs_s	S_obs_s	S_minus_P_s	SP_theory_s
0	GI.HUCU	1.2881	24.5	41.0	16.5	17.602228
1	GI.SUCU	1.3579	25.8	43.7	17.9	18.447567
5	GI.SROR	2.6962	47.5	84.8	37.3	34.805987
7	SV.NUBE	3.1539	55.0	97.6	42.6	40.527411
8	SV.RBDL	3.2079	56.3	97.6	41.3	41.202390

Next steps:

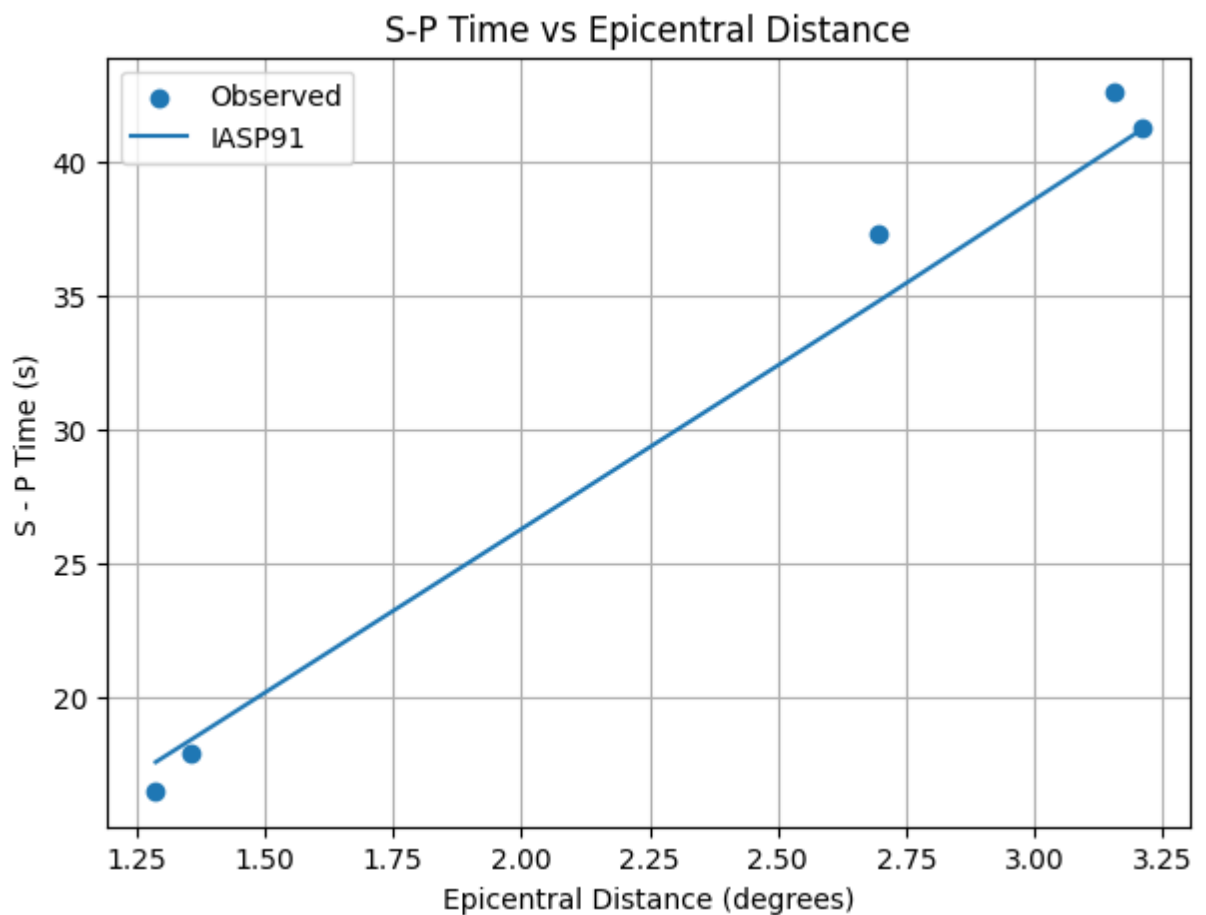
[Generate code with sp_data](#)[New interactive sheet](#)

```
plt.figure(figsize=(7,5))

plt.scatter(
    sp_data["Distance_deg"],
    sp_data["S_minus_P_s"],
    label="Observed"
)

plt.plot(
    sp_data["Distance_deg"],
    sp_data["SP_theory_s"],
    label="IASP91"
)

plt.xlabel("Epicentral Distance (degrees)")
plt.ylabel("S - P Time (s)")
plt.title("S-P Time vs Epicentral Distance")
plt.legend()
plt.grid()
plt.show()
```



```
tr = st_processed[0]

data = tr.data
n = len(data)
dt = tr.stats.delta

freq = np.fft.rfftfreq(n, d=dt)
spectrum = np.abs(np.fft.rfft(data))

plt.figure(figsize=(8,5))
plt.plot(freq, spectrum)

plt.xlim(0, 10)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.title("Frequency Spectrum - GI.HUCU")
plt.grid()
plt.show()
```

